

Table des matières

Liste des abréviations	1
Introduction générale	3
1 Contexte, problématique et positionnement du projet	4
Introduction	5
1.1 Présentation de l'organisme d'accueil	5
1.1.1 Présentation de Toulouse INP-ENSEEIH	5
1.1.2 Le service des relations internationales	5
1.2 Analyse du système d'information interne	6
1.2.1 Les outils en place	6
1.2.1.1 QS MoveON	6
1.2.1.2 Pégase	6
1.2.1.3 Eudonet	6
1.2.1.4 Les fichiers Excel	7
1.2.2 Limites structurelles	7
1.3 Problématique	11
1.4 Positionnement par rapport aux solutions existantes	11
1.4.1 Grille d'évaluation	11
1.4.2 Analyse des solutions existantes	12
1.4.2.1 QS MoveON	12
1.4.2.2 Mobility-Online	13
1.4.2.3 ErasmusJET	14
1.4.2.4 Travaux académiques : Elahi et al. (2022)	14
1.4.3 Tableau comparatif de synthèse	15
1.5 Solution envisagée	15
1.6 Organisation et cadrage du projet	16
1.6.1 Contraintes du projet	16
1.6.2 Méthodologie de gestion de projet	17
1.6.2.1 Méthode en cascade	17
1.6.2.2 Kanban	17
1.6.2.3 Scrum	18
1.6.2.4 Comparaison et choix retenu	18
1.6.3 Planification générale	21
1.6.4 Outils et environnement de travail	22
Conclusion	22

2	Besoins fonctionnels et choix technologiques	23
	Introduction	26
2.1	Besoins du système	26
2.1.1	Besoins fonctionnels	26
2.1.1.1	Fonctionnalités opérationnelles	26
2.1.1.2	Fonctionnalités intelligentes	27
2.1.2	Besoins non fonctionnels	27
2.1.3	Profil utilisateurs et périmètre de réalisation	28
2.1.3.1	Profil utilisateurs	28
2.1.3.2	Périmètre de réalisation	28
2.2	Choix du frontend	29
2.2.1	Besoins spécifiques	29
2.2.2	Alternatives évaluées	29
2.2.2.1	Applications monopage : React + Vite	29
2.2.2.2	Angular	30
2.2.2.3	Next.js 16	30
2.2.3	Synthèse comparative	31
2.2.4	Décision	31
2.3	Choix du backend	32
2.3.1	Besoins spécifiques	32
2.3.2	Choix du framework backend	32
2.3.2.1	Alternatives évaluées	32
2.3.2.1.1	Node.js	32
2.3.2.1.2	FastAPI	33
2.3.2.1.3	Django	33
2.3.2.2	Synthèse comparative	34
2.3.2.3	Décision	34
2.3.3	Choix de la couche API	35
2.3.3.1	Besoins spécifiques	35
2.3.3.2	Alternatives évaluées	35
2.3.3.2.1	Django REST Framework (DRF)	35
2.3.3.2.2	Django Ninja	35
2.3.3.3	Synthèse comparative	36
2.3.3.4	Décision	36
2.4	Choix de la base de données	37
2.4.1	Besoins spécifiques	37
2.4.2	Alternatives évaluées	37
2.4.2.1	MySQL	37
2.4.2.2	MongoDB	38
2.4.2.3	PostgreSQL 16	38

2.4.3	Synthèse comparative	38
2.4.4	Décision	39
2.5	Choix du format de stockage des imports bruts	39
2.5.1	Besoins spécifiques	39
2.5.2	Alternatives évaluées	40
2.5.2.1	Fichiers plats (JSON sur le système de fichiers)	40
2.5.2.2	Base NoSQL dédiée (MongoDB)	40
2.5.2.3	Type JSONB dans PostgreSQL	40
2.5.3	Synthèse comparative	41
2.5.4	Décision	41
2.6	Choix du gestionnaire de tâches asynchrones	41
2.6.1	Besoins spécifiques	42
2.6.2	Alternatives évaluées	42
2.6.2.1	Celery + Redis	42
2.6.2.2	Django-Q2 + PostgreSQL	43
2.6.3	Synthèse comparative	43
2.6.4	Décision	44
2.7	Choix du stockage de fichiers	44
2.7.1	Besoins spécifiques	44
2.7.2	Alternatives évaluées	44
2.7.2.1	Système de fichiers local	44
2.7.2.2	MinIO (stockage objet S3-compatible)	44
2.7.3	Synthèse comparative	45
2.7.4	Décision	45
2.8	Couche de routage des requêtes	46
2.8.1	Besoins spécifiques	46
2.8.2	Alternatives évaluées	46
2.8.2.1	Reverse proxy	46
2.8.2.2	API Gateway	46
2.8.3	Synthèse comparative	47
2.8.4	Décision	47
2.8.5	Choix de l'implémentation : Nginx vs Traefik	47
2.8.5.1	Besoins spécifiques	47
2.8.5.2	Alternatives évaluées	47
2.8.5.2.1	Nginx	48
2.8.5.2.2	Traefik	48
2.8.5.3	Synthèse comparative	48
2.8.5.4	Décision	49
2.9	Stratégie de performance base de données	49
2.9.1	Contexte et enjeux	49

2.9.2	Alternatives évaluées	49
2.9.2.1	Partitionnement par année	50
2.9.2.2	Indexation ciblée	50
2.9.3	Synthèse comparative	50
2.9.4	Décision	51
2.10	Synthèse des choix technologiques	51
	Conclusion	52
3	Conception du système	53
	Introduction	55
3.1	Product Backlog	55
3.1.1	Priorités MoSCoW	55
3.1.2	Story points	55
3.1.3	Tableau du Product Backlog	56
3.2	Planification des sprints	59
3.2.1	Diagramme de Gantt	60
3.3	Approches de conception : organisation du code et style architectural	63
3.3.1	Organisation du code : approche modulaire par domaine métier	63
3.3.1.1	Découpage en modules Django	63
3.3.1.2	Langage commun	64
3.3.1.3	Encapsulation des règles métier	64
3.3.2	Style architectural : architecture en couches	65
3.4	Architecture logicielle	65
3.4.1	Vue d'ensemble	65
3.4.2	Couche présentation : Next.js 16	66
3.4.3	Couche applicative : Django 5 + Django Ninja	67
3.4.4	Couche traitement asynchrone : Django-Q2	67
3.4.5	Couche données : PostgreSQL 16	67
3.4.6	Couche intégration : Adaptateurs ETL	68
3.4.6.1	Stratégie d'upsert	68
3.4.6.2	Clé de jointure : l'INE	69
3.4.6.3	Stratégie de synchronisation	69
3.4.7	Sécurité et contrôle d'accès	70
3.4.7.1	Flux d'authentification CAS	70
3.4.7.2	Contrôle d'accès (RBAC)	71
3.4.7.3	Gestion du renouvellement des tokens JWT	72
3.4.7.4	Journal d'audit	72
3.4.7.5	Protection des données (RGPD)	72
3.5	Architecture physique et déploiement	73
3.5.1	Vue d'ensemble	73

3.5.2	Containers Docker	74
3.5.3	Flux réseau	75
3.5.4	Environnements	75
3.5.5	CI/CD	76
3.5.6	Stratégie de migration de schéma	77
3.5.7	Sauvegarde de la base de données	77
3.5.8	Gestion des secrets	77
3.6	Modèle de données	77
3.6.1	Organisation par packages	77
3.6.2	Principes structurants	78
3.6.2.1	L'année universitaire comme axe temporel	78
3.6.2.2	L'INE comme identifiant pivot	78
3.6.2.3	Hierarchie quotas INP / quotas N7	79
3.7	Flux de données : de la source au workflow	79
3.7.1	Détail de l'étape de nettoyage et validation	80
3.7.1.1	Description du processus de transformation	80
3.7.1.1.1	Standardisation des formats	80
3.7.1.1.2	Normalisation sémantique	80
3.7.1.1.3	Gestion des valeurs manquantes	81
3.7.1.1.4	Détection et gestion des doublons	81
3.7.1.1.5	Validation métier	81
3.7.1.1.6	Traçabilité et gestion des erreurs	81
3.7.1.2	Architecture du pipeline de nettoyage	82
3.7.1.3	Technologies et outils utilisés	83
3.8	Gestion des risques	83
	Conclusion	84
4	Choix algorithmiques et machines à états	85
	Introduction	87
4.1	Choix de l'algorithme d'affectation	87
4.1.1	Besoins spécifiques	87
4.1.2	Alternatives évaluées	87
4.1.2.1	Algorithme glouton	87
4.1.2.2	Gale-Shapley	88
4.1.3	Synthèse comparative	88
4.1.4	Décision	89
4.2	Choix de la méthode de répartition des quotas par département	89
4.2.1	Besoins spécifiques	90
4.2.2	Alternatives évaluées	90
4.2.2.1	Méthode Hamilton (plus grands restes)	90

4.2.2.2	Méthode Jefferson (plus grandes moyennes)	91
4.2.2.3	Méthode Webster	91
4.2.3	Synthèse comparative	91
4.2.4	Décision	92
4.3	Choix du moteur de recommandation	92
4.3.1	Besoins spécifiques	92
4.3.2	Alternatives évaluées	92
4.3.2.1	Approche basée sur des règles (rule-based)	92
4.3.2.2	Approche par similarité (collaborative filtering)	93
4.3.2.3	Approche par apprentissage supervisé	93
4.3.2.3.1	Régression logistique	93
4.3.2.3.2	XGBoost	93
4.3.3	Synthèse comparative	94
4.3.4	Décision	94
4.4	Choix de l'orchestration RAG	95
4.4.1	Besoins spécifiques	95
4.4.2	Modèle de langage	96
4.4.3	Alternatives évaluées	96
4.4.3.1	LangChain	96
4.4.3.2	LlamaIndex	96
4.4.3.3	Haystack	97
4.4.3.4	Approche directe (sans framework)	97
4.4.4	Synthèse comparative	97
4.4.5	Décision	98
4.5	Synthèse des choix algorithmiques	98
4.6	Machines à états finis (FSM)	99
4.6.1	Principe et implémentation	99
4.6.2	FSM de l'année universitaire	99
4.6.3	FSM des entités métier	102
	Conclusion	102
5	Réalisation	103
	Introduction	105
5.1	Sprint 1 : Socle technique et référentiels	105
5.1.1	Objectif du sprint	105
5.1.2	Sprint Backlog : Sprint 1	105
5.1.3	Conception détaillée	107
5.1.3.1	Diagramme de classes	108
5.1.4	Implémentation	110
5.1.5	Tests et validation du sprint	111

5.2	Sprint 2 : Données et traçabilité	112
5.2.1	Objectif du sprint	112
5.2.2	Sprint Backlog : Sprint 2	113
5.2.3	Conception détaillée	115
5.2.3.1	Diagramme de classes	115
5.2.3.2	Diagramme de séquence	116
5.2.4	Implémentation	117
5.2.5	Tests et validation du sprint	118
5.3	Sprint 3 : Algorithme d'affectation	119
5.3.1	Objectif du sprint	119
5.3.2	Sprint Backlog : Sprint 3	120
5.3.3	Conception détaillée	122
5.3.3.1	Diagramme de classes	122
5.3.3.2	Diagrammes de séquence	123
5.3.4	Implémentation	125
5.3.5	Tests et validation du sprint	128
5.4	Sprint 4 : Sources complémentaires et mobilités	128
5.4.1	Objectif du sprint	128
5.4.2	Sprint Backlog : Sprint 4	129
5.4.3	Conception détaillée	133
5.4.3.1	Diagramme de classes	133
5.4.3.2	Diagrammes de séquence	135
5.4.4	Implémentation	136
5.4.5	Tests et validation du sprint	139
5.5	Sprint 5 : Conformité CTI et authentification	140
5.5.1	Objectif du sprint	140
5.5.2	Sprint Backlog : Sprint 5	140
5.5.3	Conception détaillée	143
5.5.3.1	Diagrammes de séquence	144
5.5.4	Implémentation	144
5.5.5	Tests et validation du sprint	147
5.6	Sprint 6 : Fonctionnalités intelligentes (V2)	149
5.6.1	Objectif du sprint	149
5.6.2	Sprint Backlog : Sprint 6	149
5.6.3	Conception détaillée	151
5.6.3.1	Diagramme de séquence	152
5.6.4	Implémentation	152
5.6.5	Tests et validation du sprint	153
	Conclusion	154

6 Tests et validation du système	155
Introduction	156
6.1 Stratégie de validation	156
6.2 Synthèse des tests réalisés	156
6.3 Validation End-to-End	157
6.4 Validation des machines à états	158
6.5 Validation des imports, exports et intégrations	159
6.6 Alertes et tâches asynchrones	160
6.7 Statistiques et tableaux de bord	160
6.8 Validation de la sécurité	160
6.9 Performance et charge	161
6.10 Intégration continue et conteneurisation	161
6.11 Analyse de qualité de code	162
6.12 Couverture des besoins non fonctionnels	162
6.13 Matrice de couverture fonctionnelle	163
Conclusion	164
Conclusion générale	166
Bibliographie	167
Netographie	168

Table des figures

1.1	Système d'information existant du service RI	7
1.2	Processus AS-IS du calcul de la durée de mobilité	9
1.3	Processus AS-IS de préparation de l'enquête CTI	10
1.4	Milestones du dépôt, un par sprint	20
1.5	Issues du dépôt, priorisées et rattachées à leur Milestone	20
1.6	Tableau de suivi GitHub Projects, par statut d'avancement	21
1.7	Pull requests fermées, nommées par sprint et par user story	21
3.1	Architecture logicielle	66
3.2	Diagramme de séquence : authentification CAS et rafraîchissement du JWT	71
3.3	Architecture physique	74
3.4	Exécution des étapes de lint, tests et build sur GitHub Actions	76
3.5	Pipeline global de traitement des données	80
3.6	Pipeline détaillé de l'étape de nettoyage des données, avec chemins de rejet	82
4.1	Machine à états : AcademicYear	100
5.1	Diagramme de classes : Sprint 1	109
5.2	Interface d'administration des référentiels	110
5.3	Suivi du cycle de vie de l'année universitaire	111
5.4	Consultation des accords et des quotas	111
5.5	Diagramme de classes : Sprint 2	116
5.6	Déroulement général de l'import et de la synchronisation des données	117
5.7	Journal d'audit	118
5.8	Consultation des données étudiants	118
5.9	Diagramme de classes : Sprint 3	123
5.10	Lancement et génération des affectations	124
5.11	Validation, correction et publication des affectations	125
5.12	Tableau de bord des affectations	126
5.13	Supervision individuelle des propositions d'affectation	126
5.14	Statistiques détaillées de l'affectation	127
5.15	Consultation des vœux et du résultat d'affectation (espace étudiant)	127
5.16	Diagramme de classes : Sprint 4	134
5.17	Réconciliation d'un dossier sans identifiant valide	135
5.18	Déclaration et traitement d'une mobilité complémentaire	136
5.19	Consultation des stages internationaux	137

5.20 Consultation des mobilités entrantes	137
5.21 Réconciliation d'un dossier sans identifiant valide	138
5.22 Déclaration d'une mobilité complémentaire (espace étudiant)	138
5.23 Validation d'une mobilité complémentaire	139
5.24 Finalisation CTI et génération du rapport	144
5.25 Tableau de bord étudiant : durée de mobilité cumulée	145
5.26 Rapport de conformité CTI	145
5.27 Tableau de bord analytique : évolution des flux	146
5.28 Tableau de bord analytique : répartition par département	146
5.29 Tableau de bord analytique : répartition par université partenaire	147
5.30 Tableau de bord analytique : répartition par pays	147
5.31 Calcul et consultation d'une recommandation de destination	152
5.32 Espace étudiant : destinations recommandées	153

Liste des tableaux

1.1	Grille d'évaluation des solutions existantes	12
1.2	Synthèse comparative des solutions selon la grille d'évaluation	15
1.3	Couverture prévue des critères d'évaluation	16
1.4	Comparaison des approches de gestion de projet	18
1.5	Planification générale du projet	22
1.6	Outils et environnement de travail	22
2.1	Comparaison des frameworks frontend	31
2.2	Comparaison des frameworks backend	34
2.3	Comparaison DRF et Django Ninja	36
2.4	Comparaison des systèmes de gestion de base de données	39
2.5	Comparaison des formats de stockage pour <code>RawImport</code>	41
2.6	Comparaison des gestionnaires de tâches asynchrones	43
2.7	Comparaison des solutions de stockage de fichiers	45
2.8	Comparaison des approches de routage des requêtes	47
2.9	Comparaison des outils de reverse proxy	49
2.10	Comparaison des stratégies de performance base de données	51
2.11	Synthèse des choix technologiques	51
3.1	Échelle des story points	56
3.2	Product Backlog	56
3.3	Planification des sprints	60
3.4	Diagramme de Gantt	61
3.5	Correspondance entre les apps Django et les packages du modèle de données	64
3.6	Stratégie de synchronisation par source	69
3.7	Containers Docker et leurs rôles	75
3.8	Packages du modèle de données	78
3.9	Matrice des risques	83
4.1	Comparaison des algorithmes d'affectation	89
4.2	Comparaison des méthodes de répartition des quotas	91
4.3	Comparaison des approches de recommandation	94
4.4	Variables d'entrée du moteur de recommandation	95
4.5	Comparaison des frameworks d'orchestration RAG	98
4.6	Synthèse des choix algorithmiques	98
4.7	Conditions préalables (guard conditions) des transitions FSM	101

4.8	FSM des entités métier	102
5.1	Sprint Backlog du Sprint 1	106
5.2	Validation du Sprint 1	112
5.3	Sprint Backlog du Sprint 2	113
5.4	Validation du Sprint 2	119
5.5	Sprint Backlog du Sprint 3	120
5.6	Validation du Sprint 3	128
5.7	Sprint Backlog du Sprint 4	129
5.8	Validation du Sprint 4	139
5.9	Sprint Backlog du Sprint 5	140
5.10	Validation du Sprint 5	148
5.11	Sprint Backlog du Sprint 6	149
5.12	Validation de l'US-22 (moteur de recommandation)	154
6.1	Synthèse des tests disponibles par couche	157
6.2	Couverture End-to-End des principaux parcours métier	158
6.3	Couverture de test des machines à états	159
6.4	Validation des imports, exports et intégrations	159
6.5	Couverture des besoins non fonctionnels	162
6.6	Matrice de couverture fonctionnelle	163

Liste des abréviations

ACID	Atomicity, Consistency, Isolation, Durability
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BDD	Base de Données
BPMN	Business Process Model and Notation
CAS	Central Authentication Service
CDC	Cahier Des Charges
CI/CD	Intégration Continue et Déploiement Continu
CQRS	Command Query Responsibility Segregation
CRM	Customer Relationship Management
CRUD	Create, Read, Update, Delete
CSR	Client-Side Rendering
CSS	Cascading Style Sheets
CTI	Commission des Titres d'Ingénieur
DDD	Domain-Driven Design
DRF	Django REST Framework
E2E	End-to-End
ETL	Extract, Transform, Load
FSM	Finite State Machine
GPA	Grade Point Average
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
IA	Intelligence Artificielle
INE	Identifiant National Étudiant
INP	Institut National Polytechnique
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
JSONB	JSON Binary (format PostgreSQL)
JWT	JSON Web Token
KPI	Key Performance Indicator

MVP	Minimum Viable Product
OIDC	OpenID Connect
ORM	Object-Relational Mapping
OWASP	Open Worldwide Application Security Project
PFE	Projet de Fin d'Études
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
RGPD	Règlement Général sur la Protection des Données
RI	Relations Internationales
RTO	Recovery Time Objective
SGBD	Système de Gestion de Base de Données
SP	Story Point
SQL	Structured Query Language
SSE	Server-Side Encryption
SSL	Secure Sockets Layer
SSO	Single Sign-On
SSR	Server-Side Rendering
TLS	Transport Layer Security
UAT	User Acceptance Testing
UML	Unified Modeling Language
URL	Uniform Resource Locator
US	User Story
UTC	Temps Universel Coordonné
WAL	Write-Ahead Log
XML	eXtensible Markup Language

Introduction générale

Contexte, problématique et positionnement du projet

Plan

Introduction	5
1.1 Présentation de l'organisme d'accueil	5
1.1.1 Présentation de Toulouse INP-ENSEEIH	5
1.1.2 Le service des relations internationales	5
1.2 Analyse du système d'information interne	6
1.2.1 Les outils en place	6
1.2.1.1 QS MoveON	6
1.2.1.2 Pégase	6
1.2.1.3 Eudonet	6
1.2.1.4 Les fichiers Excel	7
1.2.2 Limites structurelles	7
1.3 Problématique	11
1.4 Positionnement par rapport aux solutions existantes	11
1.4.1 Grille d'évaluation	11
1.4.2 Analyse des solutions existantes	12
1.4.2.1 QS MoveON	12
1.4.2.2 Mobility-Online	13
1.4.2.3 ErasmusJET	14
1.4.2.4 Travaux académiques : Elahi et al. (2022)	14
1.4.3 Tableau comparatif de synthèse	15
1.5 Solution envisagée	15
1.6 Organisation et cadrage du projet	16
1.6.1 Contraintes du projet	16
1.6.2 Méthodologie de gestion de projet	17
1.6.2.1 Méthode en cascade	17
1.6.2.2 Kanban	17
1.6.2.3 Scrum	18
1.6.2.4 Comparaison et choix retenu	18
1.6.3 Planification générale	21
1.6.4 Outils et environnement de travail	22
Conclusion	22

Introduction

Ce chapitre présente l’organisme d’accueil et la façon dont les mobilités internationales sont gérées aujourd’hui. Il en tire la problématique du stage, situe le projet par rapport aux solutions existantes et pose le cadrage retenu pour la suite.

1.1 Présentation de l’organisme d’accueil

Ce stage a été effectué au sein de Toulouse INP-ENSEEIH, école d’ingénieurs publique toulousaine, et plus précisément dans son service des Relations Internationales. Cette section présente d’abord l’établissement d’accueil, puis le service dans lequel le stage a été réalisé.

1.1.1 Présentation de Toulouse INP-ENSEEIH

Toulouse INP-ENSEEIH est une école publique d’ingénieurs, située à Toulouse et rattachée à l’Institut national polytechnique de Toulouse. Elle forme chaque année des ingénieurs, des étudiants en master et en doctorat dans trois spécialités : Sciences du Numérique (SN), Électronique–Énergie électrique–Automatique (EEEA) et Mécanique des Fluides, Énergétique & Environnement (MFEE).

L’école collabore avec plusieurs laboratoires de recherche, notamment IMFT, IRIT, LAAS et LAPLACE, et entretient des liens avec de nombreux partenaires industriels. Le développement à l’international fait partie de ses priorités : elle agrandit régulièrement son réseau d’universités partenaires et favorise les séjours à l’étranger pour ses étudiants [1].

1.1.2 Le service des relations internationales

Le service des Relations Internationales (RI) gère les échanges entre l’ENSEEIH et ses partenaires à l’étranger. Son rôle est à la fois stratégique et opérationnel : il s’occupe des accords de partenariat, mais aussi du suivi quotidien des mobilités étudiantes. Ses activités se répartissent en six domaines [2].

La gestion des partenariats académiques : gestion d’un réseau d’accords bilatéraux avec des universités étrangères, incluant les programmes d’échange et les doubles diplômes, avec suivi des places disponibles par accord et par département.

L’organisation des mobilités sortantes : organisation de la campagne annuelle de sélection des étudiants de l’ENSEEIH partant à l’étranger, depuis la publication des destinations jusqu’à la validation des affectations [3].

L’accueil des mobilités entrantes : inscription et suivi des étudiants étrangers accueillis à l’ENSEEIH dans le cadre des accords de partenariat [4].

L’accompagnement administratif et pédagogique : aide aux étudiants pour les procédures de départ, suivi des aspects financiers et vérification de la conformité des dossiers.

Le reporting CTI : suivi des mobilités entrantes et sortantes pour les audits de la Commission des Titres d’Ingénieur, qui exige une durée minimale de séjour à l’étranger pour l’obtention du diplôme.

La gestion des programmes courts : suivi des étudiants participant à des summer schools, séjours linguistiques ou autres programmes de courte durée, qui peuvent être comptabilisés dans la durée de mobilité sous certaines conditions [5].

Ces activités nécessitent de traiter chaque jour des données venant de plusieurs sources différentes, dans des formats qui ne sont pas toujours compatibles. C’est ce problème qui est au cœur du projet présenté dans ce rapport.

1.2 Analyse du système d’information interne

Avant de proposer une solution, il faut d’abord comprendre comment le service RI fonctionne et quels outils il utilise. Cette section décrit les systèmes en place, puis présente les problèmes que leur utilisation crée au quotidien.

1.2.1 Les outils en place

Le service RI utilise quatre outils distincts, qui ne communiquent pas entre eux.

1.2.1.1 QS MoveON

MoveON est l’outil principal du service RI. Il permet de gérer les accords bilatéraux, les quotas INP et les candidatures des étudiants. Il dispose d’une API REST qui permet d’extraire les données [6]. Mais il ne prend pas en compte les résultats académiques des étudiants, ne propose pas d’aide pour la sélection, et ne couvre ni les stages ni les autres éléments du parcours individuel.

1.2.1.2 Pégase

Pégase est le système d’information académique de Toulouse INP. Il regroupe les données des étudiants : inscription, département, niveau, parcours pédagogique et notes, en particulier le GPA, qui sert à classer les candidatures. Ces informations sont essentielles pour la gestion des mobilités. Les données sont accessibles via une API, mais il n’existe aucun lien direct avec MoveON.

1.2.1.3 Eudonet

Eudonet est l’outil utilisé pour gérer les conventions de stage. Il contient les informations sur les stages nationaux et internationaux : entreprise d’accueil, dates et durée [7]. Ces données entrent dans le calcul de la durée de mobilité, mais elles ne sont pas reliées aux autres systèmes.

1.2.1.4 Les fichiers Excel

Pour pallier les manques des trois outils précédents, le service RI utilise des fichiers Excel qu'il gère lui-même : suivi des quotas par département, gestion des mobilités entrantes, calcul des durées et suivi des programmes courts. Même avec une personne rigoureuse, cette méthode reste risquée : erreurs de saisie, oublis de mise à jour, fautes de frappe, incohérences entre fichiers ; ces problèmes sont difficiles à repérer et peuvent avoir des conséquences directes sur le déroulement de la campagne.

1.2.2 Limites structurelles

La figure 1.1 montre l'organisation actuelle du service RI : quatre outils utilisés séparément, sans aucune connexion automatique entre eux. Chaque transfert de données se fait manuellement.

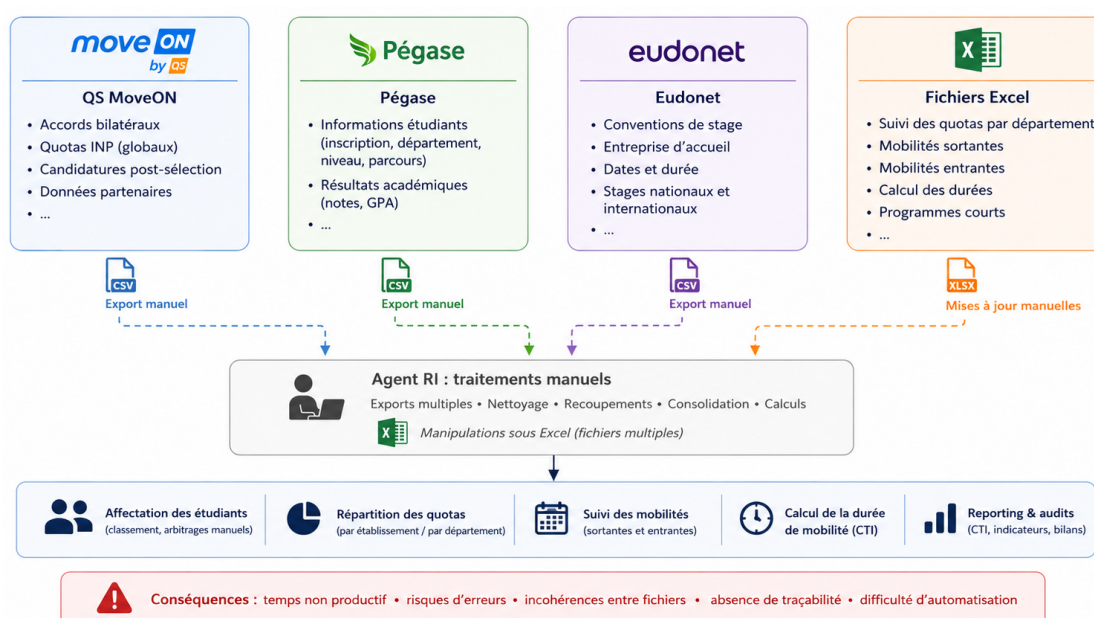


FIGURE 1.1 – Système d'information existant du service RI

Cette absence de communication entre les outils crée plusieurs problèmes qui reviennent à chaque campagne.

- **Des exports manuels à chaque étape.** Comme les outils ne communiquent pas entre eux, chaque consolidation de données nécessite des exports depuis plusieurs plateformes, puis une mise en forme dans Excel. Ce travail est long et source d'erreurs.
- **Une sélection sans règles fixes.** Les candidats sont classés manuellement à partir d'un tableau de GPA. En cas d'égalité ou de cas particulier, c'est l'agent RI qui tranche, et sa décision peut changer d'une année à l'autre sans qu'il y ait de trace écrite.
- **Des quotas insuffisamment détaillés.** MoveON ne gère les places qu'à l'échelle de l'INP global, sans distinguer les établissements qui le composent. La répartition

par établissement (dont l'ENSEEIH), puis par département pédagogique, est faite manuellement dans Excel, sans vérification de cohérence.

- **Des mobilités complémentaires non suivies.** Les summer schools, séjours linguistiques et programmes courts n'ont pas de place dans les outils existants. Pour les inclure dans le calcul de la durée de mobilité, l'agent doit s'en souvenir lui-même, ce qui n'est pas fiable.
- **Un calcul de durée de mobilité entièrement manuel et coûteux en temps.** Pour calculer la durée de mobilité d'un étudiant, il faut rassembler des données réparties dans trois systèmes différents (MoveON, Pégase, Eudonet) et les recouper à la main dans Excel. Selon les échanges avec le service RI, cette consolidation représente de l'ordre d'une dizaine de minutes par dossier, soit plusieurs journées de traitement cumulées à l'échelle d'une promotion de 300 étudiants, sans compter le risque d'oubli d'une période de mobilité à chaque manipulation. Cette opération ne peut pas être automatisée tant qu'il n'existe pas de source de données unifiée (figure 1.2).

Deux processus concrets illustrent particulièrement cette fragmentation : le calcul de la durée de mobilité, présenté précédemment, et la préparation de l'enquête annuelle CTI. Les figures 1.2 et 1.3 en proposent une modélisation selon la notation BPMN 2.0.

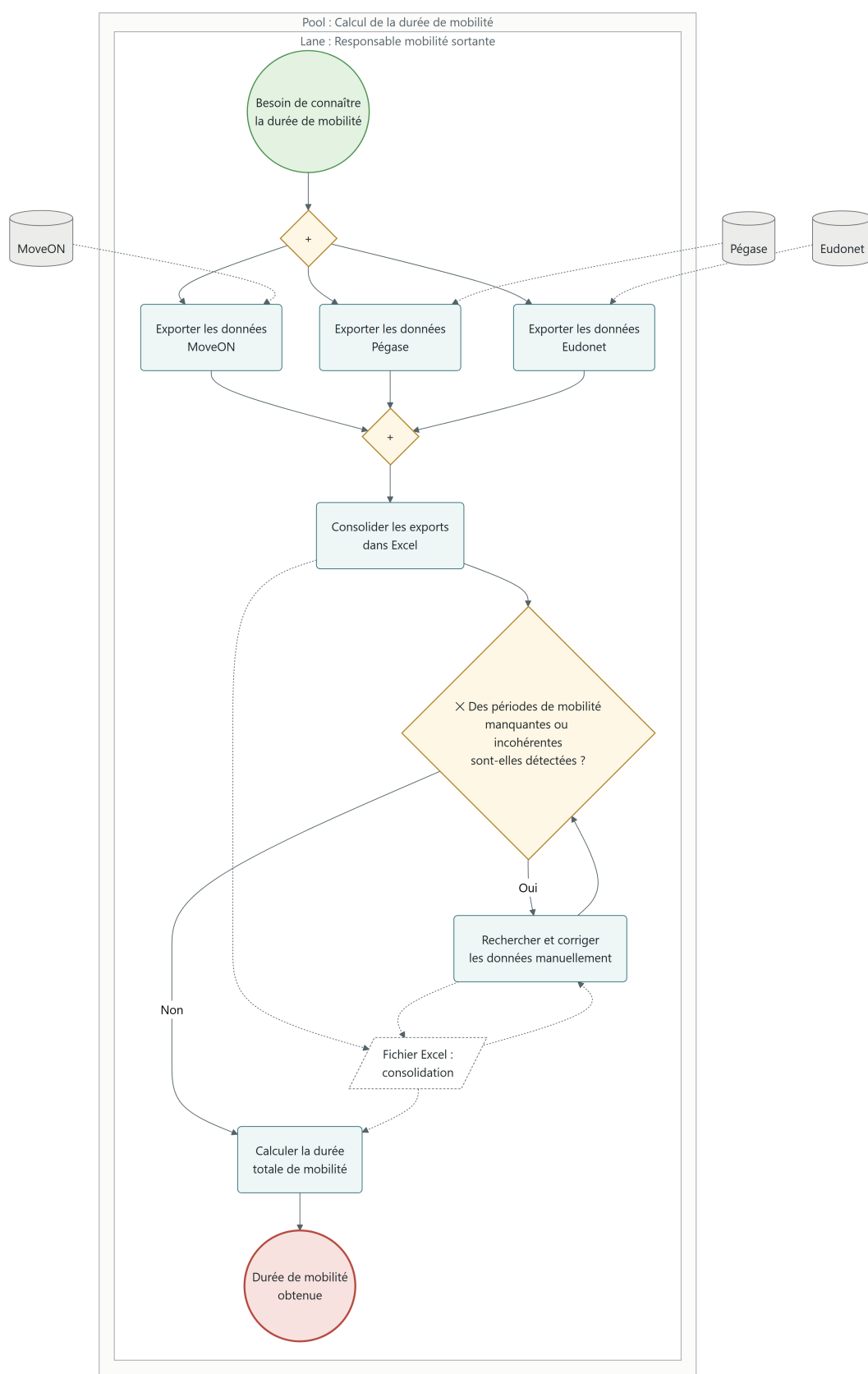


FIGURE 1.2 – Processus AS-IS du calcul de la durée de mobilité

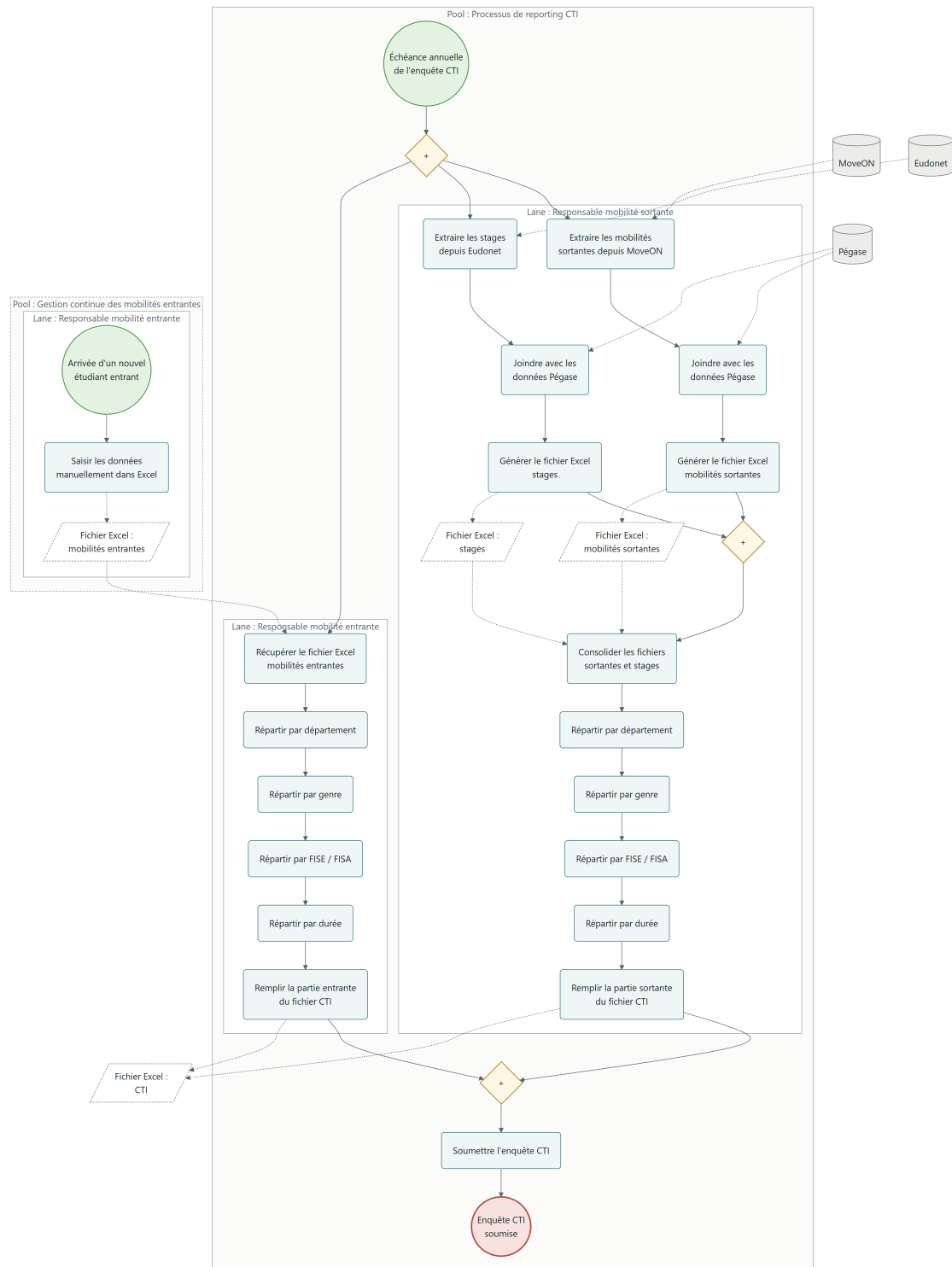


FIGURE 1.3 – Processus AS-IS de préparation de l'enquête CTI

1.3 Problématique

Le service RI utilise plusieurs outils qui ne communiquent pas entre eux. Chaque transfert de données se fait à la main, l'affectation des étudiants n'a pas de règle fixe, et il n'existe pas de moyen simple de calculer la durée de mobilité. Cinq problèmes restent sans solution :

- Les données sont dispersées entre plusieurs systèmes et ne peuvent pas être consultées depuis un seul endroit.
- Les quotas ne descendent pas jusqu'au niveau département, alors que c'est ce niveau qui détermine qui peut partir et où.
- La sélection des candidats se fait à la main, sans règle définie à l'avance.
- Le calcul de la durée de mobilité oblige à rassembler des données venant de plusieurs sources différentes.
- Les étudiants n'ont pas d'aide pour choisir leurs destinations.

C'est à partir de ce constat que se formule la question centrale du projet :

Comment concevoir une plateforme intégrée qui automatise le processus de gestion des mobilités internationales à l'ENSEEIH, depuis la centralisation des données multi-sources jusqu'à l'affectation algorithmique, tout en garantissant la conformité CTI et l'aide à la décision des étudiants ?

Avant de proposer une solution, il reste à vérifier qu'aucun outil du marché ne répond déjà à ce constat.

1.4 Positionnement par rapport aux solutions existantes

Cette section évalue les solutions existantes au regard des besoins et de la problématique formulés ci-dessus, afin de vérifier qu'aucune n'y répond déjà avant d'engager le développement d'une solution sur mesure.

1.4.1 Grille d'évaluation

Pour comparer les solutions de façon rigoureuse, une grille de neuf critères a été définie à partir des problèmes concrets identifiés ci-dessus. Chaque critère traduit une lacune précise du système actuel. Le tableau 1.1 présente cette grille.

TABLE 1.1 – Grille d'évaluation des solutions existantes

#	Critère	Ce que cela signifie concrètement pour l'ENSEEIH
C1	Centralisation des données de mobilité	Regrouper dans une seule plateforme l'ensemble des données relatives aux mobilités sortantes, entrantes, complémentaires et aux stages internationaux, ainsi que les données académiques des étudiants
C2	Quotas par école et par département	Gestion des places allouées d'abord à l'échelle de l'ENSEEIH, puis par département pédagogique, et non uniquement à l'échelle globale INP
C3	Aide à la décision et auto-affectation	Présence d'un mécanisme automatique de classement et de pré-affectation des candidats.
C4	Calcul de la durée de mobilité	Agrégation automatique de toutes les périodes de mobilité d'un étudiant (académique, stage, complémentaire) pour produire la durée réglementaire
C5	Mobilités complémentaires	Prise en charge des expériences hors accord bilatéral (summer schools, séjours linguistiques, programmes courts) avec dépôt et validation de justificatifs
C6	Adaptabilité aux règles internes	Capacité du système à fonctionner selon les règles propres à l'établissement : priorités d'affectation, seuils de sélection, critères locaux
C7	Recommandation de destinations	Suggestion de destinations adaptées au profil de l'étudiant (département, GPA, préférences) pour l'accompagner dans son choix
C8	Reporting CTI	Production automatique des indicateurs clés (KPIs) exigés par la CTI, notamment : taux de mobilité par promotion, durée moyenne de mobilité par étudiant, et répartition géographique des destinations
C9	Réponse aux questions des étudiants	Capacité du système à répondre directement aux questions procédurales des étudiants, sans passer par le service RI

1.4.2 Analyse des solutions existantes

Quatre solutions ont été examinées : QS MoveON, déjà utilisé par le service RI, Mobility-Online, une plateforme européenne largement déployée dans les services de relations internationales, ErasmusJET, développé dans le cadre du programme Erasmus+, et les travaux académiques d'Elahi et al. sur la recommandation de destinations.

1.4.2.1 QS MoveON

MoveON est l'outil central du service RI pour la gestion des accords bilatéraux et des candidatures. Il est examiné ici non pas pour le remplacer, mais pour identifier précisément ce qu'il ne couvre pas [6].

Points forts : le référentiel des accords bilatéraux, le suivi des candidatures post-sélection et l'administration des dossiers étudiants. Son API REST est un point d'appui utile pour extraire les données.

Limites par rapport aux critères définis :

- MoveON se limite aux mobilités sortantes et aux accords. Les données académiques (GPA, département, parcours), les stages internationaux et les mobilités complémentaires en sont totalement absents. Le service RI est donc contraint d’exporter depuis MoveON, de compléter à la main dans Excel, puis de travailler sur ce fichier hybride ; c’est exactement ce que ce projet cherche à éliminer.
- Les quotas ne sont gérés qu’à l’échelle INP. La répartition par école (ENSEEIHT) puis par département pédagogique, qui conditionne l’affectation interne, est entièrement tenue dans des fichiers Excel sans aucune contrainte de cohérence.
- Aucun classement automatique n’existe. La sélection est intégralement manuelle, ce qui signifie que les règles appliquées varient selon l’agent et la campagne.
- Sans les stages ni les expériences complémentaires, le calcul de la durée de mobilité est impossible à automatiser depuis MoveON seul.
- Aucune suggestion de destinations adaptée au profil de l’étudiant.
- Les exports MoveON ne produisent pas les indicateurs CTI structurés (taux de mobilité par promotion, durées moyennes, répartition géographique) dans un format directement exploitable pour les audits.

MoveON reste un bon outil pour ce qu’il fait, mais sa conception ne prévoit pas de couvrir les besoins opérationnels internes d’un établissement.

1.4.2.2 Mobility-Online

Mobility-Online est une plateforme de gestion de la mobilité étudiante largement utilisée par des services de relations internationales en Europe, notamment pour la gestion administrative des échanges Erasmus+ et des accords bilatéraux.

Points forts : un module de gestion documentaire complet (conventions, learning agreements, relevés de notes) et une bonne couverture du suivi administratif des mobilités entrantes et sortantes, sur un périmètre européen large.

Limites par rapport aux critères définis :

- La plateforme est conçue comme un outil de gestion administrative générique, sans granularité de quotas par département pédagogique propre à un établissement français comme l’ENSEEIHT.
- Aucun mécanisme d’auto-affectation basé sur le classement académique n’est proposé : la sélection reste manuelle.
- Le calcul de durée de mobilité selon les critères CTI, spécifiques aux écoles d’ingénieurs françaises, n’est pas pris en charge, la plateforme n’étant pas conçue pour ce cadre réglementaire.
- Ni recommandation de destinations personnalisée, ni reporting structuré selon les indicateurs CTI.

Mobility-Online confirme qu’une plateforme généraliste, même mature et largement déployée, ne couvre pas les besoins spécifiques d’un établissement soumis aux exigences de la CTI.

1.4.2.3 ErasmusJET

ErasmusJET est une plateforme développée dans le cadre des initiatives de digitalisation du programme Erasmus+. Son objectif est de standardiser les échanges de données entre établissements partenaires du réseau européen.

Points forts : la gestion complète du cycle Erasmus : dépôt de candidatures en ligne, génération et signature des conventions, échange de relevés de notes entre institutions. Pour un établissement dont toutes les mobilités relèvent du programme Erasmus, c’est une solution adaptée.

Limites par rapport aux critères définis :

- ErasmusJET ne fonctionne que pour Erasmus+. Les accords hors Europe, qui représentent une grande partie des mobilités de l’ENSEEIH, restent en dehors de son périmètre, tout comme les données académiques, les stages et les mobilités complémentaires.
- Les quotas par département, l’auto-affectation et le calcul de durée CTI ne sont pas prévus dans la plateforme.
- Son format est standardisé pour les échanges européens, ce qui ne laisse pas de place pour les règles propres à chaque établissement.
- Ni recommandation de destinations, ni reporting CTI.

ErasmusJET peut être utile pour la partie Erasmus+, mais il ne règle aucun des problèmes identifiés.

1.4.2.4 Travaux académiques : Elahi et al. (2022)

Les travaux d’Elahi et al. proposent un système qui combine recommandation de destinations et processus d’affectation étudiante. Les tests sont réalisés sur un jeu de données simulé [8].

Points forts : c’est la seule des solutions analysées qui aborde sérieusement le critère C7 : la recommandation personnalisée de destinations. L’article démontre la faisabilité technique de coupler un algorithme de suggestion avec un processus de sélection, ce qui en fait une référence directement pertinente pour la partie intelligence du projet.

Limites par rapport aux critères définis :

- Le modèle repose sur un dataset simulé et uniforme, sans connexion avec des systèmes institutionnels réels. Il ne propose aucune intégration avec MoveON, Pégase ou Eudonet.

- La gestion des quotas par département, l’affectation sous contraintes, le calcul de durée de mobilité et les expériences complémentaires sont entièrement hors du périmètre ; le modèle ne traite que la recommandation.
- C’est un prototype de recherche : il n’y a aucun moyen de configurer des règles propres à un établissement.
- Le reporting CTI n’est pas abordé.

Ces travaux sont une référence utile pour la partie recommandation, mais ils ne forment pas une solution prête à être utilisée à l’ENSEEIH.

1.4.3 Tableau comparatif de synthèse

Le tableau 1.2 résume la couverture de chaque solution pour les neuf critères de la grille d’évaluation. La légende suivante s’applique : ✓ = critère satisfait, ✗ = critère non satisfait, *Partiel* = critère partiellement couvert.

TABLE 1.2 – Synthèse comparative des solutions selon la grille d’évaluation

Critère	QS MoveON	Mobility-Online	ErasmusJET	Elahi et al.
C1 – Centralisation des données	Partiel	Partiel	✗	✗
C2 – Quotas par école et département	✗	✗	✗	✗
C3 – Aide à la décision et auto-affectation	✗	✗	✗	✗
C4 – Calcul durée de mobilité	✗	✗	✗	✗
C5 – Mobilités complémentaires	✗	✗	✗	✗
C6 – Adaptabilité métier	Partiel	✗	✗	✗
C7 – Recommandation de destinations	✗	✗	✗	Partiel
C8 – Reporting CTI	✗	✗	✗	✗
C9 – Réponse aux questions des étudiants	✗	✗	✗	✗

Aucune des quatre solutions étudiées ne répond à l’ensemble des neuf critères retenus, en particulier la granularité des quotas par département et la conformité CTI. Ce constat, obtenu sur un panel volontairement diversifié, confirme la nécessité d’une solution propre au service RI, dont les besoins sont précisés au chapitre suivant.

1.5 Solution envisagée

La réponse envisagée est le développement d’une plateforme propre au service RI, qui se connecte aux outils existants pour en récupérer les données plutôt que de les rempla-

cer. Le tableau 1.3 reprend chacun des neuf critères de la grille d'évaluation et indique la fonctionnalité, parmi celles qui viennent d'être présentées, qui le couvre.

TABLE 1.3 – Couverture prévue des critères d'évaluation

Critère	Fonctionnalité prévue	Périmètre
C1 – Centralisation des données	BF01 : intégration des données multi-sources	MVP garanti
C2 – Quotas par école et département	BF02 : gestion des accords et quotas	MVP garanti
C3 – Aide à la décision et auto-affectation	BF03 : algorithme d'affectation	MVP garanti
C4 – Calcul de la durée de mobilité	BF04 : suivi de la durée de mobilité	MVP garanti
C5 – Mobilités complémentaires	BF05 : mobilités complémentaires	MVP garanti
C6 – Adaptabilité aux règles internes	Configuration transverse des quotas, seuils et priorités (BF02, BF03)	MVP garanti
C7 – Recommandation de destinations	BF09 : moteur de recommandation	V2 conditionnel
C8 – Reporting CTI	BF08 : reporting et enquêtes CTI	MVP garanti
C9 – Réponse aux questions des étudiants	BF10 : assistant virtuel (chatbot RAG)	V2 conditionnel

Donc, sept critères sur neuf sont couverts dès la première version (MVP). Les deux critères restants, la recommandation de destinations (C7) et la réponse aux questions des étudiants (C9), sont repoussés en V2, si le temps le permet, car ils n'ont pas d'impact sur la conformité CTI.

Côté architecture, la plateforme est organisée en couches : présentation, API, logique métier et données, avec une couche dédiée à l'intégration des imports MoveON, Pégase, Eudonet et Excel. Le choix des technologies pour chaque couche est expliqué au chapitre 2, et les algorithmes de sélection, d'affectation et de recommandation, ainsi que les machines à états, au chapitre 4. Leur mise en œuvre concrète (organisation du code, modèle de données) est détaillée au chapitre 3.

1.6 Organisation et cadrage du projet

Cette section regroupe les éléments qui cadrent la conduite du projet : les contraintes qui l'encadrent, la méthodologie de gestion de projet retenue, la planification générale et l'environnement de travail utilisé.

1.6.1 Contraintes du projet

Trois types de contraintes ont encadré la conduite du projet.

Contraintes techniques. La plateforme ne remplace aucun des outils existants : elle doit s'intégrer à MoveON, Pégase et Eudonet via leurs API respectives, avec un mécanisme de repli par fichier Excel lorsque l'une d'elles est indisponible.

Contraintes réglementaires. Le système doit respecter le RGPD dans le traitement des données étudiantes et produire des indicateurs conformes aux exigences de la Commission des Titres d'Ingénieur en matière de durée de mobilité.

Contraintes organisationnelles. Le développement est assuré par une seule personne sur une durée fixe de six mois, ce qui limite le temps disponible aux fonctionnalités non prioritaires. Certaines règles métier, notamment la hiérarchie des quotas, n'étaient pas encore stabilisées au démarrage du projet, et la disponibilité du service RI pour les validations reste limitée aux points d'avancement de fin de sprint.

1.6.2 Méthodologie de gestion de projet

Le projet s'étend sur six mois dans un contexte où certaines règles métier n'étaient pas complètement formalisées dès le départ. Trois approches ont été évaluées avant de retenir la méthodologie la mieux adaptée.

1.6.2.1 Méthode en cascade

La méthode en cascade organise le projet en phases séquentielles et strictement ordonnées : analyse, conception, développement, test, déploiement. Chaque phase doit être complètement terminée avant de passer à la suivante.

Avantages : Elle offre une planification claire et une documentation exhaustive dès le départ, ce qui facilite le suivi et la traçabilité des décisions.

Limites dans notre contexte : Elle impose de geler un cahier des charges complet avant tout développement. Or, certaines règles métier, notamment la hiérarchie des quotas et les critères de priorité dans l'affectation, n'étaient pas entièrement définies au démarrage du projet, rendant cette approche inadaptée.

1.6.2.2 Kanban

Kanban est une méthode de gestion de flux continu, sans itérations fixes. Le travail est visualisé sur un tableau et les tâches avancent au fur et à mesure de la disponibilité de l'équipe.

Avantages : Sa flexibilité permet de s'adapter rapidement aux changements et de traiter les tâches par ordre de priorité sans contrainte de cycle.

Limites dans notre contexte : L'absence d'itérations fixes et de jalons structurés le rend peu adapté à un projet avec un périmètre défini et une date de livraison fixe, où une planification rigoureuse est nécessaire.

1.6.2.3 Scrum

Scrum est un cadre agile qui organise le développement en sprints de durée fixe, avec des événements régulières (planning, review, retrospective) et un backlog priorisé.

Avantages : Il offre un bon équilibre entre structure et flexibilité : les priorités peuvent être révisées à chaque sprint, les livraisons sont incrémentales et validables régulièrement par le service RI.

Limites dans notre contexte : Scrum est conçu pour des équipes. Certains événements, comme le Daily Scrum ou la rétrospective d'équipe, supposent une coordination quotidienne entre plusieurs personnes et ne s'appliquent pas tels quels à un projet porté par une seule développeuse ; ils doivent être adaptés ou remplacés par des pratiques équivalentes.

1.6.2.4 Comparaison et choix retenu

Le tableau 1.4 compare les trois approches sur des critères objectifs.

TABLE 1.4 – Comparaison des approches de gestion de projet

Critère	Cascade	Kanban	Scrum
Planification	Rigide dès le début	Continue, sans itérations fixes	Sprints fixes avec rétroplanification
Adaptation aux changements	Difficile	Bonne, mais sans rythme structuré	Bonne, intégrée à chaque sprint
Visibilité client	Faible en cours de projet	Modérée	Élevée (Sprint Review)

Au regard de ces trois critères, Scrum est le seul à combiner une planification structurée et une capacité d'adaptation intégrée à chaque itération, tout en offrant la meilleure visibilité pour le service RI. C'est pourquoi **Scrum** est retenu, pour son équilibre entre structure (sprints de durée fixe, backlog priorisé) et flexibilité (révision des priorités à chaque sprint, livraisons incrémentales validables par le service RI). Ce cadre étant conçu pour une équipe, son application est toutefois adaptée au contexte d'un développement individuel : certains rôles et événements sont ajustés ou remplacés par des pratiques équivalentes, comme précisé ci-dessous.

Scrum s'appuie sur des rôles, des artefacts et des événements, présentés ci-dessous et adaptés au contexte d'un stage individuel.

Les rôles Scrum.

- **Le Product Owner** : responsable de la priorisation du backlog et de la validation des livraisons.
- **Le Scrum Master** : chargé du suivi de l'avancement, de la levée des blocages et de l'orientation technique.

- ***L'équipe de développement (Development Team)*** : réalise les incréments du produit. Ce rôle est habituellement porté par plusieurs personnes ; dans ce projet, il est assuré par une seule personne, responsable de l'intégralité du développement.

Les artefacts Scrum.

- ***Le backlog du produit*** : liste des fonctionnalités du système, priorisées selon la méthode MoSCoW et estimées en story points ; détaillé au chapitre 3 (Product Backlog).
- ***Le backlog de sprint*** : sous-ensemble du backlog produit retenu pour un sprint donné ; présenté pour chaque sprint réalisé au chapitre 5.
- ***L'incrément*** : ensemble des fonctionnalités terminées à l'issue d'un sprint, qui s'ajoute aux incréments précédents.

Les événements Scrum.

- ***Le Sprint*** : période de développement à durée fixe (deux à trois semaines dans ce projet), à l'issue de laquelle un incrément fonctionnel est livré.
- ***Sprint Planning*** : réunion de début de sprint au cours de laquelle les user stories du sprint sont sélectionnées et décomposées en tâches.
- ***Sprint Review*** : réunion de fin de sprint au cours de laquelle les fonctionnalités livrées sont démontrées au service RI (Product Owner), qui valide ou ajuste les priorités du backlog.
- ***Daily Scrum*** : dans sa forme standard, synchronisation quotidienne entre les membres de l'équipe de développement. Cet événement suppose une coordination entre plusieurs personnes et n'a donc pas de sens pour un projet porté par une seule développeuse ; il n'est pas mis en œuvre.
- ***Sprint Retrospective*** : dans sa forme standard, réunion d'équipe de fin de sprint destinée à identifier les points à améliorer. Dans le cadre de ce projet individuel, elle est remplacée par un bilan personnel de fin de sprint, réalisé sans réunion dédiée, permettant de vérifier les tâches restantes et d'ajuster en conséquence le contenu du sprint suivant.

Le suivi du projet s'appuie sur les Milestones et les Issues de GitHub : chaque sprint correspond à un Milestone, et chaque user story est déclarée comme une issue, priorisée (label `priority`) et rattachée au Milestone du sprint qui la traite. La figure 1.4 montre les Milestones du dépôt, chacun avec son taux d'avancement.

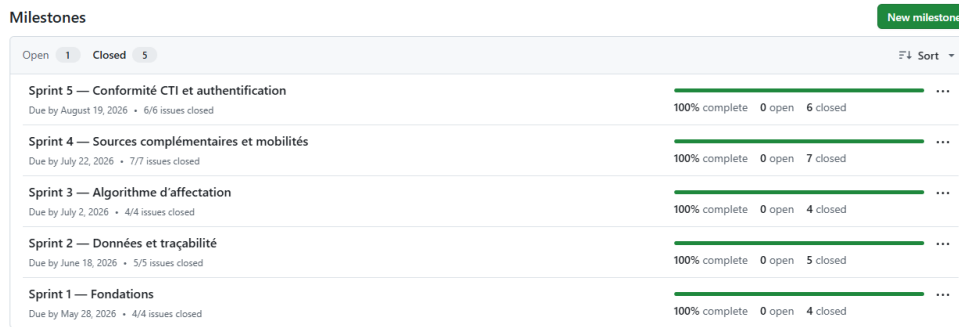


FIGURE 1.4 – Milestones du dépôt, un par sprint

La figure 1.5 détaille les issues d'un Milestone, avec leurs labels de priorité, de sprint et de type.

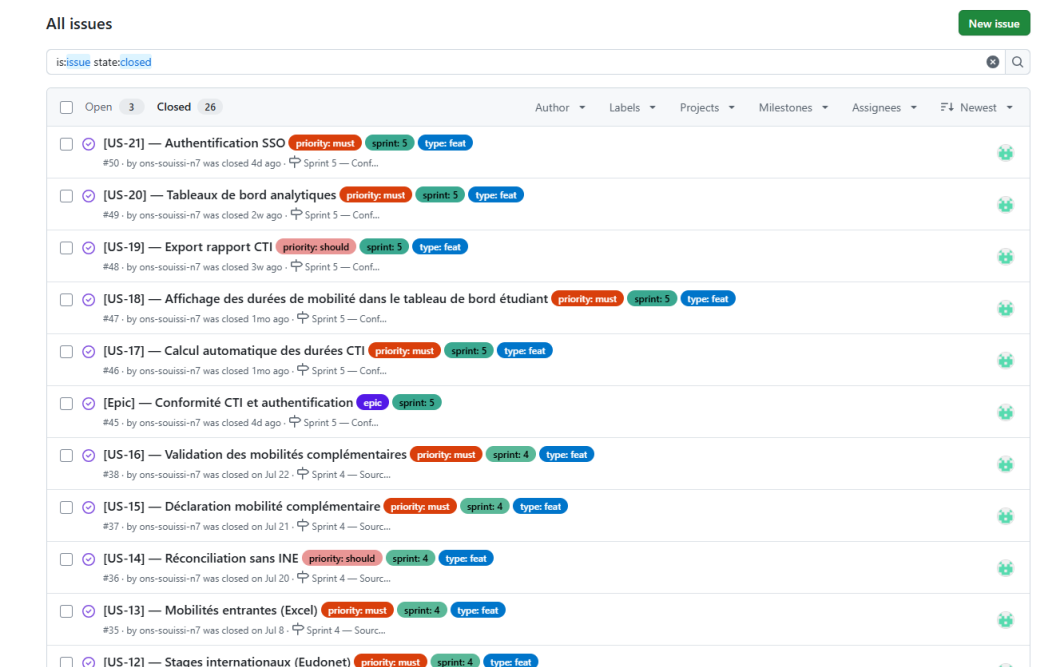


FIGURE 1.5 – Issues du dépôt, priorisées et rattachées à leur Milestone

L'avancement de ces issues est ensuite suivi via un tableau GitHub Projects, organisé par statut. La figure 1.6 en montre la vue par défaut.

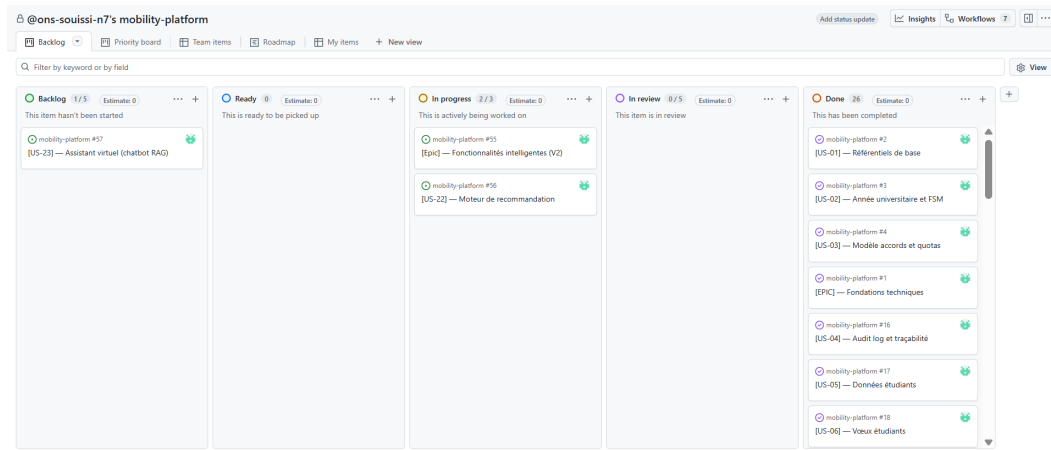


FIGURE 1.6 – Tableau de suivi GitHub Projects, par statut d’avancement

Cette vue centralisée permet de suivre en un coup d’œil l’avancement du projet et d’identifier rapidement les tâches encore ouvertes.

La traçabilité entre une user story et son implémentation est assurée par les pull requests : chaque branche suit la convention `feature/sprintN_usXX`, et sa pull request ferme l’issue correspondante une fois mergée. La figure 1.7 montre les pull requests fermées du dépôt.

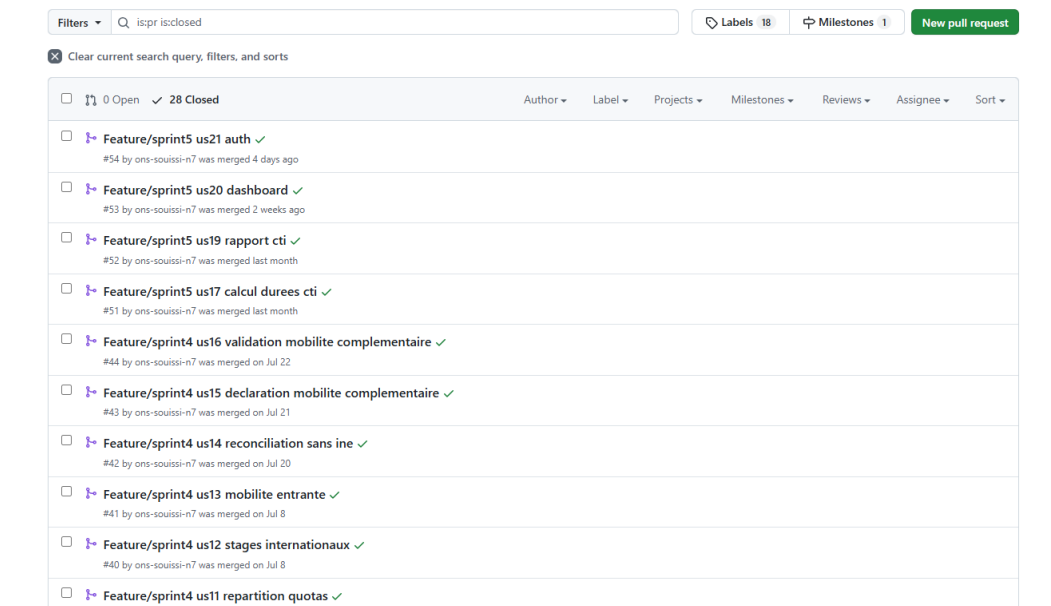


FIGURE 1.7 – Pull requests fermées, nommées par sprint et par user story

Ce lien direct entre l’expression du besoin et son implémentation facilite la revue du travail réalisé et la compréhension du contexte de chaque modification.

1.6.3 Planification générale

Le projet s’est déroulé sur six mois, organisés en cinq grandes phases dont l’enchaînement est résumé dans le tableau 1.5. La planification détaillée sprint par sprint, sous forme

de product backlog et de diagramme de Gantt, ne pouvait être établie qu’une fois le périmètre fonctionnel et les choix technologiques stabilisés ; elle est donc présentée au chapitre 3 (Conception du système).

TABLE 1.5 – Planification générale du projet

Phase	Contenu	Durée indicative
1	Cadrage, analyse de l’existant et positionnement par rapport aux solutions existantes (chapitre 1)	3 à 4 semaines
2	Choix technologiques et algorithmiques (chapters 2 et 4)	2 semaines
3	Conception détaillée : product backlog, architecture, modèle de données (chapitre 3)	2 à 3 semaines
4	Réalisation itérative en sprints Scrum (chapitre 5)	environ 3 mois
5	Tests de charge et de sécurité, documentation, livraison	2 à 3 semaines

1.6.4 Outils et environnement de travail

Le développement s’appuie sur un environnement de travail volontairement simple, adapté à un projet porté par une seule personne. Le tableau 1.6 en résume les principaux éléments.

TABLE 1.6 – Outils et environnement de travail

Catégorie	Outil(s)	Usage
Éditeur de code	Visual Studio Code	Développement de l’application
Conteneurisation	Docker Desktop	Exécution locale des conteneurs de l’application
Modélisation UML	StarUML	Diagrammes de classes, de séquence et machines à états
Gestion de versions	Git, GitHub	Versionnement du code ; suivi du backlog via GitHub Projects
Communication avec le service RI	Email, Zoom, réunions en présentiel	Échanges courants et points d’avancement, notamment lors des Sprint Review

Conclusion

Ce chapitre a posé le contexte, la problématique et le cadrage du projet. Le chapitre suivant détaille les besoins du système et les choix technologiques retenus pour y répondre.

Besoins fonctionnels et choix technologiques

Plan

Introduction	26
2.1 Besoins du système	26
2.1.1 Besoins fonctionnels	26
2.1.1.1 Fonctionnalités opérationnelles	26
2.1.1.2 Fonctionnalités intelligentes	27
2.1.2 Besoins non fonctionnels	27
2.1.3 Profils utilisateurs et périmètre de réalisation	28
2.1.3.1 Profils utilisateurs	28
2.1.3.2 Périmètre de réalisation	28
2.2 Choix du frontend	29
2.2.1 Besoins spécifiques	29
2.2.2 Alternatives évaluées	29
2.2.2.1 Applications monopage : React + Vite	29
2.2.2.2 Angular	30
2.2.2.3 Next.js 16	30
2.2.3 Synthèse comparative	31
2.2.4 Décision	31
2.3 Choix du backend	32
2.3.1 Besoins spécifiques	32
2.3.2 Choix du framework backend	32
2.3.2.1 Alternatives évaluées	32
2.3.2.1.1 Node.js	32
2.3.2.1.2 FastAPI	33
2.3.2.1.3 Django	33
2.3.2.2 Synthèse comparative	34
2.3.2.3 Décision	34
2.3.3 Choix de la couche API	35
2.3.3.1 Besoins spécifiques	35
2.3.3.2 Alternatives évaluées	35
2.3.3.2.1 Django REST Framework (DRF)	35
2.3.3.2.2 Django Ninja	35
2.3.3.3 Synthèse comparative	36

2.3.3.4	Décision	36
2.4	Choix de la base de données	37
2.4.1	Besoins spécifiques	37
2.4.2	Alternatives évaluées	37
2.4.2.1	MySQL	37
2.4.2.2	MongoDB	38
2.4.2.3	PostgreSQL 16	38
2.4.3	Synthèse comparative	38
2.4.4	Décision	39
2.5	Choix du format de stockage des imports bruts	39
2.5.1	Besoins spécifiques	39
2.5.2	Alternatives évaluées	40
2.5.2.1	Fichiers plats (JSON sur le système de fichiers)	40
2.5.2.2	Base NoSQL dédiée (MongoDB)	40
2.5.2.3	Type JSONB dans PostgreSQL	40
2.5.3	Synthèse comparative	41
2.5.4	Décision	41
2.6	Choix du gestionnaire de tâches asynchrones	41
2.6.1	Besoins spécifiques	42
2.6.2	Alternatives évaluées	42
2.6.2.1	Celery + Redis	42
2.6.2.2	Django-Q2 + PostgreSQL	43
2.6.3	Synthèse comparative	43
2.6.4	Décision	44
2.7	Choix du stockage de fichiers	44
2.7.1	Besoins spécifiques	44
2.7.2	Alternatives évaluées	44
2.7.2.1	Système de fichiers local	44
2.7.2.2	MinIO (stockage objet S3-compatible)	44
2.7.3	Synthèse comparative	45
2.7.4	Décision	45
2.8	Couche de routage des requêtes	46
2.8.1	Besoins spécifiques	46
2.8.2	Alternatives évaluées	46
2.8.2.1	Reverse proxy	46
2.8.2.2	API Gateway	46
2.8.3	Synthèse comparative	47
2.8.4	Décision	47
2.8.5	Choix de l'implémentation : Nginx vs Traefik	47
2.8.5.1	Besoins spécifiques	47

2.8.5.2	Alternatives évaluées	47
2.8.5.2.1	Nginx	48
2.8.5.2.2	Traefik	48
2.8.5.3	Synthèse comparative	48
2.8.5.4	Décision	49
2.9	Stratégie de performance base de données	49
2.9.1	Contexte et enjeux	49
2.9.2	Alternatives évaluées	49
2.9.2.1	Partitionnement par année	50
2.9.2.2	Indexation ciblée	50
2.9.3	Synthèse comparative	50
2.9.4	Décision	51
2.10	Synthèse des choix technologiques	51
	Conclusion	52

Introduction

Le chapitre précédent a montré qu’aucune solution existante ne couvrait les besoins du service RI. Ce chapitre précise ces besoins, puis justifie les choix technologiques retenus pour y répondre.

2.1 Besoins du système

Le constat précédent conduit à formuler des exigences précises. Cette section décrit les principales fonctionnalités attendues du système, les exigences non fonctionnelles associées, ainsi que les profils utilisateurs et le périmètre de réalisation retenu pour ce projet.

2.1.1 Besoins fonctionnels

Les besoins fonctionnels décrivent les principales fonctionnalités attendues du système afin de répondre aux problématiques identifiées au sein du service RI. Ils couvrent à la fois les fonctionnalités opérationnelles et les fonctionnalités intelligentes.

2.1.1.1 Fonctionnalités opérationnelles

Le système prend en charge les opérations suivantes, qui couvrent l’ensemble du cycle de gestion des mobilités internationales, de l’import des données jusqu’à l’archivage des résultats.

- **BF01 — Intégration des données** : import depuis MoveON, Pégase et Eudonet via API REST, et depuis des fichiers Excel pour les sources sans API. Chaque import inclut une détection de doublons, une validation des formats et un mécanisme de fallback Excel en cas d’indisponibilité d’une API.
- **BF02 — Gestion des accords et quotas** : import des accords depuis MoveON et gestion de la répartition locale des places par département N7, avec possibilité d’ajustement manuel par le service RI.
- **BF03 — Algorithme d’affectation** : pré-affectation automatique basée sur le GPA, les vœux ordonnés et les quotas par département, avec interface de supervision et d’ajustement.
- **BF04 — Suivi de la durée de mobilité** : calcul automatique de la durée totale de mobilité de chaque étudiant, par agrégation des séjours académiques, stages et expériences complémentaires validées.
- **BF05 — Mobilités complémentaires** : les étudiants déclarent leurs expériences et déposent les justificatifs nécessaires, puis le service RI se charge de la validation des dossiers.

- **BF06 — Mobilités entrantes** : import via template Excel et suivi administratif granulaire.
- **BF07 — Historisation** : les données d’une campagne clôturée passent en lecture seule. Aucune exécution ultérieure ne peut modifier les résultats d’une année passée.
- **BF08 — Reporting et enquêtes CTI** : calcul automatique des indicateurs requis pour les enquêtes annuelles de la Commission des Titres d’Ingénieur, à savoir le nombre et le pourcentage d’étudiants mobiles, la durée moyenne de mobilité et la répartition des destinations, ainsi que l’export des rapports de conformité correspondants.

2.1.1.2 Fonctionnalités intelligentes

En complément des fonctionnalités opérationnelles, le système intègre deux modules intelligents qui améliorent l’expérience étudiant sans solliciter le service RI.

- **BF09 — Moteur de recommandation** : suggestions de destinations basées sur le GPA, le département et l’historique des affectations, pour orienter les choix de l’étudiant avant la saisie dans MoveON.
- **BF10 — Assistant virtuel (chatbot RAG)** : agent conversationnel répondant aux questions procédurales à partir de la documentation institutionnelle, sans solliciter le service RI.

2.1.2 Besoins non fonctionnels

Au-delà des fonctionnalités métier, le système doit également répondre à plusieurs exigences techniques et organisationnelles. Ces besoins non fonctionnels concernent notamment la sécurité, la performance, l’intégrité des données et l’intégration avec l’environnement existant.

- **BNF01 — Interopérabilité** : le système doit s’intégrer sans rupture dans l’écosystème existant via des adaptateurs REST dédiés à chaque source. En cas d’indisponibilité d’une API, un fallback par import Excel garantit la continuité opérationnelle.
- **BNF02 — Intégrité des données** : les données provenant de sources hétérogènes peuvent être incohérentes ou dupliquées. Une validation stricte est appliquée à chaque import : contrôle de format, détection de doublons, cohérence inter-sources. Toute donnée rejetée est tracée dans un rapport d’import avec la raison précise, pour correction ciblée.
- **BNF03 — Sécurité et RGPD** : authentification via le SSO institutionnel, gestion stricte des rôles (étudiant vs administrateur), chiffrement des données sensibles. Un étudiant n’accède qu’à son propre dossier.
- **BNF04 — Performance** : le temps de réponse de l’API pour les endpoints courants doit être inférieur à 200 ms sous une charge de 50 utilisateurs simultanés. L’algorithme

d'affectation doit s'exécuter en moins de 30 secondes pour une promotion de 300 étudiants. Les traitements lourds, tels que l'algorithme d'affectation ou les calculs CTI, sont exécutés de façon asynchrone en arrière-plan pour ne pas bloquer l'interface.

- **BNF05 — Disponibilité** : s'agissant d'un système institutionnel sollicité lors de périodes de campagne critiques (ouverture des vœux, publication des résultats), une disponibilité cible de 99 % hors fenêtres de maintenance planifiées est visée, appuyée par une politique de sauvegarde permettant une reprise rapide en cas d'incident.
- **BNF06 — Scalabilité** : séparation entre données actives et données archivées en lecture seule pour absorber la croissance annuelle du volume historique et les pics de connexion lors des moments clés de campagne.
- **BNF07 — Ergonomie** : l'interface adapte sa navigation et ses fonctionnalités au rôle de l'utilisateur connecté, réduisant les erreurs de manipulation.
- **BNF08 — Traçabilité** : toute action importante, telle qu'un import, une modification d'affectation, une validation ou une transition de phase, est enregistrée avec l'identité de l'auteur et l'horodatage dans un journal d'audit persistant en écriture seule.

2.1.3 Profils utilisateurs et périmètre de réalisation

Cette sous-section présente les profils d'utilisateurs du système, puis définit le périmètre de réalisation retenu pour ce projet.

2.1.3.1 Profils utilisateurs

Deux profils utilisent le système, avec des besoins très différents.

L'administrateur, c'est-à-dire le service RI, pilote la campagne de bout en bout. Il supervise les imports, configure les quotas, ajuste les propositions d'affectation, instruit les demandes de mobilités complémentaires et produit les rapports CTI. Le système prend en charge les tâches répétitives, mais toute décision finale reste sous sa responsabilité, et chaque intervention manuelle est tracée.

L'étudiant consulte les destinations disponibles, reçoit des recommandations personnalisées avant de saisir ses vœux dans MoveON, suit l'état de sa candidature, déclare ses expériences complémentaires et vérifie à tout moment si sa durée totale de mobilité atteint le seuil requis pour l'obtention du diplôme. L'assistant virtuel répond à ses questions procédurales sans solliciter le service RI.

2.1.3.2 Périmètre de réalisation

Le développement est organisé en deux niveaux de priorité.

MVP : fonctionnalités prioritaires (BF01 à BF08, BNF01 à BNF08) :

- Intégration des données multi-sources (MoveON, Pégase, Eudonet, Excel)
 - Gestion des accords et quotas par département
 - Algorithme d'affectation avec validation humaine
 - Calcul automatique de la durée de mobilité
 - Gestion des mobilités complémentaires et entrantes
 - Reporting et enquêtes CTI : indicateurs chiffrés, tableaux de bord analytiques et exports de conformité
 - Espace étudiant : catalogue, suivi de candidature, déclarations
 - Sécurité : SSO, contrôle d'accès, audit log
- V2 : fonctionnalités secondaires (BF09, BF10) :**
- Moteur de recommandation de destinations personnalisé
 - Assistant virtuel (chatbot RAG)

2.2 Choix du frontend

Cette section présente les besoins spécifiques du frontend, évalue les alternatives disponibles, puis justifie le choix retenu.

2.2.1 Besoins spécifiques

L'interface du projet doit couvrir deux espaces bien distincts : d'un côté, un espace dédié au service RI pour piloter les campagnes, de l'autre, un espace étudiant pour consulter les destinations, suivre sa candidature et interagir avec le chatbot. Certaines pages affichent des données sensibles (résultats d'affectation, GPA) et doivent être construites côté serveur pour éviter qu'elles ne soient accessibles avant que le rôle de l'utilisateur soit vérifié. D'autres, comme le suivi de candidature ou le chatbot, nécessitent une navigation fluide et réactive. Gérer ces deux types de pages dans le même projet, avec une séparation fiable des accès selon le rôle, est le principal critère de choix.

2.2.2 Alternatives évaluées

Trois frameworks ont été évalués au regard des besoins identifiés.

2.2.2.1 Applications monopage : React + Vite

React permet de construire des interfaces entièrement rendues dans le navigateur [9].

Avantages :

- Simple à configurer et à démarrer

- Très bien documenté, avec de nombreuses ressources disponibles en ligne

Limites :

- Tout est chargé et affiché dans le navigateur : les données sensibles comme les affectations ou les GPA y arrivent avant même que le rôle de l'utilisateur soit contrôlé, ce qui fragilise la séparation entre l'espace étudiant et l'espace RI
- La protection des pages selon le rôle doit être codée manuellement, sans mécanisme intégré pour l'imposer au niveau du routing
- Les appels à l'API Django passent tous par le navigateur, ce qui expose les tokens d'authentification au client

2.2.2.2 Angular

Angular est un framework TypeScript développé par Google, utilisé principalement pour des applications d'entreprise avec beaucoup de logique interne [10].

Avantages :

- Architecture très structurée, adaptée aux applications avec de nombreux écrans et règles métier
- Typage TypeScript natif, ce qui réduit les erreurs au moment du développement
- Mécanisme de protection des routes intégré, utile pour séparer les accès selon le rôle utilisateur
- Bien adapté à des interfaces d'administration complexes comme le tableau de bord RI

Limites :

- Pas de rendu côté serveur sans Angular Universal, qui nécessite une configuration supplémentaire non négligeable
- Angular demande un temps de prise en main important : son architecture impose de nombreux concepts spécifiques (modules, décorateurs, injection de dépendances) qui ralentissent le démarrage
- Moins d'exemples disponibles que React/Next.js pour l'intégration avec le SSO CAS utilisé par Toulouse INP

2.2.2.3 Next.js 16

Next.js est un framework basé sur React qui permet de combiner rendu côté serveur et rendu côté client selon les besoins de chaque page [11].

Avantages :

- Rendu hybride : pages sensibles rendues côté serveur, composants interactifs côté client, les deux coexistant dans le même projet

- Middleware intégré pour bloquer l'accès aux pages selon le rôle utilisateur avant même leur chargement
- Appels à l'API Django réalisables côté serveur, sans exposer les tokens dans le navigateur
- App Router et React Server Components : chaque composant est rendu côté serveur par défaut, avec des layouts partagés par espace (RI, étudiant) qui structurent naturellement la séparation des interfaces
- TypeScript, Tailwind CSS et intégration SSO CAS bien documentés

Limites :

- Configuration initiale un peu plus complexe qu'une application monopage classique

2.2.3 Synthèse comparative

Le tableau 2.1 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu. Chaque tableau comparatif de ce chapitre introduit une colonne **Poids** : un critère **bloquant** est une exigence dont le non-respect suffit à écarter une solution, tandis qu'un critère **secondaire** renforce ou affaiblit un choix sans être, à lui seul, éliminatoire.

TABLE 2.1 – Comparaison des frameworks frontend

Critère	Poids	React + Vite	Angular	Next.js 16
Rendu côté serveur (SSR)	Bloquant	✗	Via Angular Universal (config requise)	✓
Protection des routes par rôle	Bloquant	Manuelle	Mécanisme intégré	Middleware intégré
Données sensibles côté serveur	Bloquant	✗	✗	✓
TypeScript natif	Secondaire	Optionnel	✓	✓
Intégration SSO CAS	Bloquant	✓	Peu d'exemples avec le SSO CAS de Toulouse INP	✓
Rendu hybride SSR + CSR	Bloquant	✗	✗	✓

2.2.4 Décision

Next.js 16 est retenu. Son modèle de React Server Components permet de gérer naturellement les deux espaces du projet : les pages sensibles du tableau de bord RI sont rendues côté serveur avant toute vérification de rôle, tandis que les composants interactifs

de l'espace étudiant s'exécutent côté client. Angular pourrait, en théorie, répondre au même besoin grâce à Angular Universal. Mais avec le temps disponible pour ce projet, Next.js permettait d'arriver au même résultat plus rapidement et avec moins de configuration à mettre en place.

2.3 Choix du backend

Cette section présente les besoins spécifiques du backend, évalue les alternatives disponibles, puis justifie le choix retenu.

2.3.1 Besoins spécifiques

Le backend de ce projet ne se limite pas à une simple gestion de données. Il doit orchestrer de nombreuses entités métier liées entre elles et gérer des processus parfois complexes, comme le suivi des candidatures, des affectations ou des mobilités étudiantes.

L'application doit également assurer plusieurs fonctions essentielles : gérer les droits d'accès selon les profils (étudiants et administrateurs RI), conserver une trace des actions importantes via un journal d'audit, accompagner l'évolution de la base de données pendant le développement, et s'intégrer au système d'authentification institutionnel CAS de Toulouse INP.

Dans ce contexte, le choix du framework ne repose pas uniquement sur les performances, mais aussi sur la qualité des outils qu'il apporte pour structurer, sécuriser et faire évoluer l'application.

2.3.2 Choix du framework backend

Cette sous-section évalue les frameworks backend candidats au regard des besoins définis ci-dessus, puis justifie le choix retenu.

2.3.2.1 Alternatives évaluées

Trois frameworks ont été évalués en regard des besoins identifiés précédemment.

2.3.2.1.1 Node.js

Environnement d'exécution JavaScript côté serveur, reconnu pour ses performances sur les applications qui traitent beaucoup de requêtes simultanées [12].

Avantages :

- Bonnes performances sur les traitements non bloquants
- Grand nombre de bibliothèques disponibles
- Gestion native des opérations asynchrones

Limites :

- Aucun outil natif pour la base de données, l’administration ou la gestion des droits : il faut tout assembler manuellement à partir de bibliothèques séparées, ce qui alourdit la configuration et la maintenance.
- Des bibliothèques de machine à états comme XState existent et sont matures, mais rien d’équivalent à `django-fsm` n’est directement intégré à un ORM Node.js : il faudrait câbler soi-même la protection des transitions et la traçabilité fine des modifications au niveau des modèles, ce qui est indispensable dans ce projet.
- L’utilisation de Node.js implique de travailler en JavaScript, alors que le reste du système est conçu en Python, ce qui introduit une complexité supplémentaire en termes de cohérence et de maintenance.
- La gestion des évolutions du schéma de base de données au fil des sprints demande davantage d’efforts d’organisation et de suivi, car elle n’est pas aussi intégrée nativement dans l’écosystème.

2.3.2.1.2 FastAPI

Framework Python récent, conçu pour construire des APIs performantes avec une validation automatique des données [13].

Avantages :

- Très bonnes performances grâce au support natif des requêtes asynchrones
- Validation automatique des données entrantes
- Documentation de l’API générée automatiquement

Limites :

- Pas d’outil intégré pour la base de données, l’administration ou la gestion des droits
- Les cycles de vie des entités, le journal d’audit et les permissions seraient à construire de zéro
- L’outil de migration de schéma demande davantage de configuration et d’intervention manuelle, ce qui peut ralentir les cycles d’itération en développement.
- FastAPI est surtout adapté à des APIs légères et rapides, et moins à des applications contenant beaucoup de logique métier.

2.3.2.1.3 Django

Framework Python complet, pensé pour développer rapidement des applications web avec beaucoup de logique interne [14].

Avantages :

- Gestion de la base de données, authentification, permissions et interface d’administration fournis nativement, sans aucune configuration supplémentaire

- `django-fsm` permet de définir des cycles de vie sécurisés sur chaque entité, avec une option qui empêche toute modification directe du statut en dehors des transitions autorisées
- `django-auditlog` enregistre automatiquement toutes les modifications avec l’auteur et la date, ce qui est indispensable pour les décisions réglementaires
- La gestion des évolutions de schéma est intégrée nativement, ce qui facilite les changements à chaque sprint
- S’intègre facilement avec le SSO CAS via des bibliothèques existantes et maintenues

Limites :

- Django était historiquement limité sur les traitements asynchrones, mais cette lacune a été corrigée depuis la version 5
- Légèrement moins performant que FastAPI sur des calculs très complexes, ce qui peut être contourné en passant par des requêtes SQL directes pour les agrégations de durées CTI

2.3.2.2 Synthèse comparative

Le tableau 2.2 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.2 – Comparaison des frameworks backend

Critère	Poids	Node.js	FastAPI	Django 5
ORM et gestion BDD intégrés	Bloquant	✗	✗	✓
Interface d’administration	Secondaire	✗	✗	✓
Machines à états (FSM)	Bloquant	✗	✗	✓
Migrations de schéma	Secondaire	Manuel	nécessite config	✓
Authentification SSO	Bloquant	Manuel	Manuel	✓
Journal d’audit automatique	Bloquant	Manuel	Manuel	✓
Support ASGI / async	Secondaire	✓	✓	✓

2.3.2.3 Décision

Django 5 est retenu car le projet repose sur de nombreuses entités liées entre elles, avec des cycles de vie distincts, un journal d’audit obligatoire et une gestion fine des droits entre deux profils utilisateurs.

Utiliser une autre solution aurait impliqué de reconstruire une grande partie de ces mécanismes.

Django permet de disposer de ces fonctionnalités directement, ce qui est un avantage important dans ce projet.

2.3.3 Choix de la couche API

Une fois le framework backend retenu, il reste à choisir la bibliothèque la mieux adaptée pour construire l'API. Cette section évalue les deux solutions compatibles avec Django et justifie le choix retenu.

2.3.3.1 Besoins spécifiques

L'API doit permettre les échanges entre Next.js et Django, valider des structures de données parfois complexes (vœux ordonnés, quotas par département, rapports d'import), générer automatiquement une documentation utilisable par le développeur frontend, et répondre de façon non bloquante aux requêtes qui déclenchent des traitements longs (lancement de l'algorithme, recalcul).

2.3.3.2 Alternatives évaluées

Deux solutions ont été évaluées pour exposer l'API entre le frontend Next.js et le backend Django.

2.3.3.2.1 Django REST Framework (DRF)

Django REST Framework est la bibliothèque la plus utilisée pour construire des APIs avec Django. Elle fournit un ensemble complet d'outils pour la sérialisation, la validation et la gestion des permissions [15].

Avantages :

- Très mature et bien documenté, avec une grande communauté
- Outillage complet : gestion des tokens JWT, filtres, pagination, génération de documentation via `drf-spectacular`
- Fonctionne parfaitement avec l'ORM et les permissions Django

Limites :

- Les vues DRF sont synchrones par défaut : les rendre asynchrones demande une configuration supplémentaire, utile uniquement si une vue devait un jour appeler elle-même une API externe sans passer par une tâche de fond
- Les sérialiseurs DRF demandent beaucoup de code répétitif, surtout quand on travaille avec plusieurs entités
- La documentation de l'API nécessite une configuration externe séparée, à maintenir en parallèle

2.3.3.2.2 Django Ninja

Framework API pour Django, basé sur une validation des données structurée et typée via Pydantic [16].

Avantages :

- Chaque route peut être synchrone ou asynchrone selon le besoin, nativement et sans configuration supplémentaire.
- La validation des données via Pydantic est plus concise que DRF
- La documentation de l’API est générée automatiquement, sans aucune configuration supplémentaire
- Conserve entièrement l’écosystème Django : ORM, administration, cycles de vie, permissions

Limites :

- Plus récent que DRF, avec moins d’extensions disponibles
- Moins de ressources en ligne pour des cas très spécifiques

2.3.3.3 Synthèse comparative

Le tableau 2.3 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.3 – Comparaison DRF et Django Ninja

Critère	Poids	DRF	Django Ninja
Validation Pydantic (type strict)	Secondaire	✗	✓
Réduction du code répétitif	Secondaire	✗	✓
Documentation auto-générée	Secondaire	✗	✓
Intégration ORM Django	Bloquant	✓	✓
Support async natif (vues)	Secondaire	Non natif, configuration requise	✓
Maturité et communauté	Secondaire	✓	Framework récent, moins de ressources

2.3.3.4 Décision

Django Ninja est retenu. L’argument principal est la validation par schémas Pydantic : les entrées et sorties de chaque endpoint sont typées explicitement, ce qui réduit nettement le code répétitif de sérialisation par rapport à DRF, en particulier sur les nombreuses entités du domaine (accords, quotas, vœux, affectations). La documentation OpenAPI générée automatiquement à partir de ces schémas est également appréciable pour un projet développé avec un frontend séparé.

Le support natif de l’asynchrone n’est pas le critère décisif : les endpoints critiques du projet (déclenchement de l’algorithme, recalcul des quotas, synchronisations ETL) répondent immédiatement en déléguant le traitement à une tâche Django-Q2, ce qu’une vue synchrone

classique fait tout aussi bien puisque c'est la mise en file d'attente qui est asynchrone, pas la vue elle-même. Ce support reste néanmoins un atout pour d'éventuels futurs endpoints qui appelleraient directement une API externe (Eudonet, MoveON, Pégase) sans passer par une tâche de fond.

La stack backend retenue est donc : **Django 5 + Django Ninja**.

2.4 Choix de la base de données

Cette section présente les besoins spécifiques liés au stockage des données, évalue les alternatives disponibles, puis justifie le choix retenu.

2.4.1 Besoins spécifiques

La base de données doit supporter un modèle avec plusieurs entités reliées entre elles par de nombreuses clés étrangères, stocker les réponses brutes des APIs externes avant transformation (**RawImport**), et garantir la cohérence des données lors des opérations qui touchent plusieurs entités à la fois (transitions de cycle de vie, affectations, journal d'audit).

2.4.2 Alternatives évaluées

Trois systèmes de gestion de base de données ont été évalués en regard des besoins identifiés précédemment.

2.4.2.1 MySQL

MySQL est un système de gestion de base de données relationnelle très répandu, bien supporté par l'écosystème Django et largement utilisé en production [17].

Avantages :

- Système très répandu, avec une grande communauté et une documentation abondante
- Bonne intégration avec l'écosystème Django
- Largement utilisé en production, avec des outils de sauvegarde matures

Limites :

- Support moins adapté pour le stockage et l'exploitation de données au format JSON, notamment pour les imports bruts (**RawImport**)
- Fonctionnalités limitées pour gérer des mécanismes de file d'attente de tâches
- Moins adapté aux opérations transactionnelles complexes impliquant plusieurs entités liées

2.4.2.2 MongoDB

MongoDB est une base de données orientée documents, conçue pour offrir une grande flexibilité dans la structure des données et particulièrement adaptée au stockage JSON [18].

Avantages :

- Grande flexibilité dans la structure des données
- Adapté au stockage de documents JSON
- Bonnes performances sur les lectures de documents non structurés

Limites :

- Le modèle de données du projet repose fortement sur des relations entre entités (quotas, affectations, mobilités), ce qui rend l'absence de relations structurées moins adaptée
- Intégration limitée avec les outils ORM utilisés dans le projet
- Absence de support direct pour certains mécanismes de traitement asynchrone utilisés dans l'architecture

2.4.2.3 PostgreSQL 16

PostgreSQL est un système de gestion de base de données relationnelle open source, reconnu pour sa robustesse, la richesse de ses fonctionnalités et son respect strict des standards SQL [19].

Avantages :

- Gestion robuste des transactions, adaptée aux opérations impliquant plusieurs entités liées
- Support natif du format JSON avec indexation, utile pour le stockage des données brutes (`RawImport`)
- Possibilité d'utiliser la base comme support de file d'attente pour les traitements asynchrones
- Bon niveau d'intégration avec les outils de développement du projet
- Outils de sauvegarde et de restauration fiables

Limites :

- Mise en place initiale plus complète, nécessitant une configuration structurée dès le début du projet

2.4.3 Synthèse comparative

Le tableau 2.4 récapitule les critères évalués pour chacune des trois solutions, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.4 – Comparaison des systèmes de gestion de base de données

Critère	Poids	MySQL / MariaDB	MongoDB	PostgreSQL 16
Transactions ACID multi-tables	Bloquant	✓	ACID ajouté en v4.0, moins fiable qu'un SGBD relationnel	✓
Support JSONB natif + indexation	Bloquant	JSON simple, pas d'indexation avancée	✓	✓
File d'attente de tâches	Secondaire	✗	✗	✓
Intégration ORM Django	Bloquant	✓	Support via PyMongo sans migration Django	✓
Extension pgcrypto (chiffrement)	Secondaire	✗	✗	✓
Outils de sauvegarde fiables	Secondaire	✓	✓	✓

2.4.4 Décision

PostgreSQL 16 est retenu. Il répond de manière cohérente à l'ensemble des besoins identifiés, à savoir la gestion des relations, le support du JSON natif, les traitements asynchrones et le chiffrement, dans un seul service, sans introduire de composant supplémentaire dans l'infrastructure.

2.5 Choix du format de stockage des imports bruts

Cette section précise comment sont stockées les réponses brutes des APIs externes avant leur traitement, en évaluant les alternatives disponibles et en justifiant le format retenu.

2.5.1 Besoins spécifiques

L'entité `RawImport` doit conserver la réponse complète de chaque appel d'API externe (MoveON, Pégase, Eudonet) avant toute transformation. Ces données sont semi-structurées et variables d'une source à l'autre. Le format retenu doit permettre de relire, d'interroger et de tracer ces imports tout en restant cohérent avec le reste du modèle de données.

2.5.2 Alternatives évaluées

Deux options ont été comparées pour répondre à ces besoins.

2.5.2.1 Fichiers plats (JSON sur le système de fichiers)

Un fichier plat est un fichier texte brut qui stocke des données sans structure hiérarchique ni mécanisme d'indexation. Dans ce contexte, chaque réponse d'API serait sérialisée en JSON et enregistrée dans un répertoire dédié sur le serveur, un fichier par import.

Avantages :

- Mise en place immédiate, sans configuration particulière
- Lecture directe par n'importe quel éditeur de texte

Limites :

- Aucune possibilité d'interroger le contenu avec des requêtes SQL
- Intégrité référentielle avec les autres entités non garantie
- Gestion des accès concurrents et des transactions non assurée
- Sauvegarde dissociée du reste de la base de données

2.5.2.2 Base NoSQL dédiée (MongoDB)

Les imports bruts sont stockés dans une base documentaire séparée, spécialisée dans le stockage de documents JSON.

Avantages :

- Très bien adapté au stockage de documents JSON hétérogènes
- Bonne performance sur les lectures de documents non structurés

Limites :

- Service supplémentaire à déployer, superviser et sauvegarder
- Aucune relation possible avec les entités de la base relationnelle
- Surcoût d'infrastructure injustifié pour ce seul cas d'usage

2.5.2.3 Type JSONB dans PostgreSQL

Les réponses brutes sont stockées dans une colonne de type **JSONB** au sein de la table **RawImport**, dans la même base PostgreSQL.

Avantages :

- Stockage dans la base existante, sans service supplémentaire
- Indexation du contenu JSON pour des requêtes efficaces
- Jointures possibles avec les autres tables du modèle

- Mêmes garanties ACID et même périmètre de sauvegarde que le reste

Limites :

- Moins flexible que MongoDB pour des structures JSON très hétérogènes, ce qui reste sans impact ici compte tenu du faible nombre de sources

2.5.3 Synthèse comparative

Le tableau 2.5 récapitule les critères évalués pour les deux formats, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.5 – Comparaison des formats de stockage pour RawImport

Critère	Poids	Fichiers plats	MongoDB	JSONB (PostgreSQL)
Requêtes SQL sur le contenu	Bloquant	✗	✗	✓
Intégrité référentielle	Bloquant	✗	✗	✓
Garanties ACID	Bloquant	✗	Transactions ACID ajoutées en v4.0, moins fiables que PostgreSQL	✓
Aucun service supplémentaire	Secondaire	✓	✗	✓
Indexation du contenu JSON	Secondaire	✗	✓	✓
Sauvegarde unifiée	Secondaire	✗	✗	✓

2.5.4 Décision

Le type **JSONB** de PostgreSQL est retenu pour le stockage des imports bruts. Il évite toute fragmentation de l'infrastructure en réutilisant la base déjà en place, tout en offrant les capacités d'interrogation nécessaires sur les données semi-structurées. Les fichiers plats ne permettent pas de garantir la cohérence avec le reste du modèle, et l'ajout de MongoDB n'apporterait aucun avantage fonctionnel pour ce seul cas d'usage.

2.6 Choix du gestionnaire de tâches asynchrones

Cette section présente les besoins en traitement asynchrone du projet, évalue les solutions disponibles, puis justifie le choix retenu.

2.6.1 Besoins spécifiques

Certaines opérations du système ne peuvent pas s'exécuter de façon synchrone sans bloquer l'interface utilisateur. Deux catégories de tâches ont été identifiées.

La première catégorie regroupe les tâches déclenchées par des événements : le lancement de l'algorithme d'affectation, le recalcul des durées après modification d'un état, et le traitement des imports qui peuvent durer plusieurs secondes.

La seconde catégorie regroupe les tâches planifiées : une vérification quotidienne du calendrier de la campagne, qui déclenche automatiquement le passage à la candidature, la clôture des vœux et la clôture de l'année universitaire dès que les dates prévues sont atteintes. Les synchronisations des données métier (accords, vœux, étudiants, stages) sont, elles, déclenchées à la demande depuis l'interface d'administration.

Le volume reste modéré : quelques dizaines de tâches par jour en fonctionnement normal, quelques centaines lors des périodes de traitement, sans besoin de traitement intensif en continu.

2.6.2 Alternatives évaluées

Deux solutions ont été évaluées pour la gestion des tâches asynchrones dans l'écosystème Django.

2.6.2.1 Celery + Redis

Celery est la bibliothèque Python de référence pour l'exécution de tâches en arrière-plan. Elle s'appuie sur un broker de messages externe, généralement Redis ou RabbitMQ, pour acheminer les tâches vers les workers [20] [21].

Avantages :

- Très mature, bien documenté, avec une grande communauté
- Capable de traiter un très grand volume de tâches en parallèle
- Compatible avec plusieurs brokers (Redis, RabbitMQ)
- Outil de supervision dédié (Flower)

Limites :

- Trois services supplémentaires à déployer et maintenir : le worker Celery, le planificateur Celery Beat et Redis
- Redis consomme entre 50 et 200 Mo de mémoire pour un rôle que PostgreSQL, déjà présent, peut assurer seul
- Complexité disproportionnée par rapport au volume réel du projet

2.6.2.2 Django-Q2 + PostgreSQL

Django-Q2 est une bibliothèque de gestion de tâches asynchrones conçue spécifiquement pour Django. Elle utilise directement PostgreSQL comme file d’attente, sans nécessiter de broker externe [22].

Avantages :

- Aucun service supplémentaire : PostgreSQL joue le rôle de file d’attente sans configuration additionnelle
- Planificateur de tâches intégré, adapté aux synchronisations nocturnes et aux suppressions programmées
- Supervision accessible depuis l’interface d’administration Django existante, sans outils tiers
- Seulement deux services à opérer au lieu de cinq avec Celery

Limites :

- Moins performant que Celery pour des volumes très élevés, sans incidence pour ce projet
- Communauté plus restreinte que Celery

2.6.3 Synthèse comparative

Le tableau 2.6 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.6 – Comparaison des gestionnaires de tâches asynchrones

Critère	Poids	Celery + Redis	Django-Q2 + PostgreSQL
Aucun service supplémentaire	Secondaire	✗	✓
Planificateur intégré	Bloquant	Celery Beat disponible mais service séparé	✓
Supervision via admin Django	Secondaire	✗	✓
Adapté au volume du projet	Bloquant	Surdimensionné pour ce volume de tâches	✓
Maturité et communauté	Secondaire	✓	Communauté plus restreinte que Celery
Intégration native avec Django	Secondaire	Nécessite une configuration supplémentaire	✓

2.6.4 Décision

Django-Q2 avec PostgreSQL est retenu. Il réutilise la base de données déjà en place comme file d'attente, ce qui évite d'introduire un service supplémentaire dans l'infrastructure. Le volume de tâches du projet ne justifie pas la complexité opérationnelle de Celery. La supervision des traitements depuis l'interface d'administration Django existante simplifie également le suivi quotidien pour le service RI.

2.7 Choix du stockage de fichiers

Cette section présente les besoins en stockage de fichiers du projet, évalue les solutions disponibles, puis justifie le choix retenu.

2.7.1 Besoins spécifiques

Le projet requiert un mécanisme de dépôt et de stockage de fichiers pour les justificatifs soumis par les étudiants lors de la déclaration de mobilités complémentaires. Ces fichiers doivent être stockés de manière sécurisée et chiffrée, accessibles uniquement par le service RI et l'étudiant concerné, avec une suppression automatique conforme au RGPD.

2.7.2 Alternatives évaluées

Deux approches ont été évaluées pour répondre à ces besoins.

2.7.2.1 Système de fichiers local

Le stockage local consiste à enregistrer les fichiers directement sur le système de fichiers du serveur hébergeant l'application Django, dans un répertoire dédié.

Avantages :

- Simple à configurer, sans dépendance supplémentaire

Limites :

- Aucun chiffrement natif des fichiers stockés
- Non portable entre environnements (développement, staging, production)
- Difficile à sauvegarder et à migrer indépendamment de la base de données

2.7.2.2 MinIO (stockage objet S3-compatible)

MinIO est une solution de stockage objet open source, auto-hébergeable et compatible avec l'API S3 d'Amazon [23]. Elle s'intègre nativement dans une infrastructure Docker et ne nécessite aucun service cloud externe.

Avantages :

- Stockage objet auto-hébergé, compatible avec l’API S3 standard
- Chiffrement côté serveur (SSE-S3) des fichiers stockés
- Génération d’URLs pré-signées pour un accès temporaire et sécurisé, sans exposer les fichiers directement
- Déployable comme container Docker interne, sans aucune dépendance externe ni service cloud
- Politique de rétention et suppression automatique configurables, conformes aux exigences RGPD

Limites :

- Configuration initiale plus complexe qu’un stockage local

2.7.3 Synthèse comparative

Le tableau 2.7 récapitule les critères évalués pour chacune des deux solutions, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.7 – Comparaison des solutions de stockage de fichiers

Critère	Poids	Fichiers local	MinIO
Chiffrement natif (SSE-S3)	Bloquant	✗	✓
URLs pré-signées (accès sécurisé)	Bloquant	✗	✓
Portabilité multi-environnements	Secondaire	✗	✓
Suppression automatique (RGPD)	Bloquant	✗	✓
Déploiement Docker intégré	Secondaire	✗	✓
Simplicité de configuration	Secondaire	✓	Plus complexe qu’un stockage local

2.7.4 Décision

MinIO est retenu. MinIO peut être déployé selon deux modes : **standalone** (instance unique sur un seul serveur) ou **distributed** (plusieurs nœuds pour la haute disponibilité et la tolérance aux pannes). Le mode distribué est destiné aux infrastructures multi-serveurs nécessitant une réplication des données.

Dans le cadre de ce projet, le déploiement cible un **serveur unique on-premise** au sein de l’établissement. Le mode standalone est donc retenu : il correspond exactement à cette topologie, sans surcharger l’infrastructure avec des mécanismes de distribution inutiles. MinIO répond ainsi directement aux exigences de sécurité (chiffrement SSE-S3) et de conformité RGPD (rétention et suppression automatiques), tout en s’intégrant de façon transparente dans Docker Compose sans aucun service externe.

2.8 Couche de routage des requêtes

Cette section présente les besoins en matière de routage réseau, évalue les deux approches disponibles, puis justifie le choix retenu.

2.8.1 Besoins spécifiques

Le système doit disposer d'un composant chargé de recevoir les requêtes HTTP entrantes, d'assurer la terminaison SSL, de router le trafic vers le frontend Next.js et l'API Django Ninja, et de servir directement les fichiers statiques sans les faire transiter par l'application. Ce composant sera administré par les équipes IT de Toulouse INP ; sa configuration doit donc être lisible et maintenable sans expertise Docker particulière.

2.8.2 Alternatives évaluées

Deux approches ont été évaluées pour assurer ce rôle.

2.8.2.1 Reverse proxy

Un reverse proxy est un composant qui se place entre les clients et les services internes. Il reçoit les requêtes HTTP entrantes, assure la terminaison SSL, les achemine vers le service approprié selon des règles de routage, et sert les fichiers statiques directement sans solliciter l'application.

Avantages :

- Composant léger, dédié au routage et à la terminaison SSL
- Configuration statique (Nginx) ou dynamique (Traefik) selon le besoin
- Bien adapté à une architecture monolithique sur un seul serveur
- Maturité et documentation abondantes pour les deux outils

Limites :

- Ne centralise pas l'authentification ni le contrôle de débit (ce qui n'est pas requis ici)

2.8.2.2 API Gateway

Une API Gateway est un composant qui, en plus du routage, centralise la gestion de l'authentification, du contrôle de débit et de la transformation des requêtes. Elle est conçue pour exposer plusieurs microservices à des clients externes variés.

Avantages :

- Gestion centralisée de l'authentification et du contrôle de débit
- Adapté aux architectures microservices exposées à l'externe

Limites :

- Complexité et surcoût injustifiés pour une architecture déployée sur un seul serveur
- Le projet consomme des APIs externes, il n'en expose pas à des clients variés ; ce cas d'usage ne s'applique pas

2.8.3 Synthèse comparative

Le tableau 2.8 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.8 – Comparaison des approches de routage des requêtes

Critère	Poids	Reverse proxy	API Gateway
Adapté à un seul serveur	Bloquant	✓	✗
Terminaison SSL	Bloquant	✓	✓
Routage vers services internes	Bloquant	✓	✓
Gestion auth. centralisée	Secondaire	✗	✓
Complexité d'exploitation	Secondaire	Faible	Élevée
Pertinent pour ce projet	Bloquant	✓	✗

2.8.4 Décision

Le **reverse proxy** est retenu. L'architecture est déployée sur un seul serveur et le système consomme des APIs externes sans en exposer à des clients variés. Une API Gateway introduirait une complexité sans aucun bénéfice fonctionnel dans ce contexte.

2.8.5 Choix de l'implémentation : Nginx vs Traefik

Une fois le reverse proxy retenu, il reste à choisir l'outil d'implémentation. Cette partie évalue les deux solutions les plus utilisées dans l'écosystème Docker et justifie le choix retenu.

2.8.5.1 Besoins spécifiques

L'outil retenu doit assurer la terminaison SSL, le routage vers le frontend Next.js et l'API Django Ninja, et la distribution des fichiers statiques. Sa configuration doit être lisible et auditable par les équipes IT de Toulouse INP qui administreront le serveur, sans expertise Docker avancée.

2.8.5.2 Alternatives évaluées

Deux outils open source ont été comparés pour implémenter le reverse proxy.

2.8.5.2.1 Nginx

Nginx est un serveur web et reverse proxy très répandu, reconnu pour sa stabilité et la lisibilité de ses fichiers de configuration [24].

Avantages :

- Très stable et bien documenté, avec un comportement prévisible
- Configuration dans des fichiers texte simples, lisibles et faciles à auditer par n'importe quel administrateur système
- Gestion SSL/TLS fiable, excellentes performances sur le routage des requêtes

Limites :

- La gestion des certificats SSL nécessite un outil externe (Certbot)
- Configuration manuelle à mettre à jour lors de changements de services

2.8.5.2.2 Traefik

Traefik est un reverse proxy conçu pour les environnements conteneurisés. Il se configure dynamiquement à partir des labels des conteneurs Docker actifs [25].

Avantages :

- Configuration automatique à partir des conteneurs Docker actifs
- Gestion native des certificats SSL via Let's Encrypt
- Interface de supervision intégrée

Limites :

- Configuration déclarative moins lisible pour des équipes IT non familières avec Docker
- Comportement moins prévisible dans un environnement institutionnel avec des contraintes réseau spécifiques

2.8.5.3 Synthèse comparative

Le tableau 2.9 récapitule les critères évalués pour chacun des deux outils, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 2.9 – Comparaison des outils de reverse proxy

Critère	Poids	Nginx	Traefik
Configuration lisible (fichiers texte)	Bloquant	✓	Labels Docker, moins lisible pour les équipes IT
Gestion SSL native	Secondaire	Nécessite Certbot pour les certificats	✓
Automatisation Docker	Secondaire	Configuration manuelle à chaque changement	✓
Prévisibilité en environnement institutionnel	Bloquant	✓	Moins prévisible avec des contraintes réseau institutionnelles
Maintenabilité par équipes IT générales	Bloquant	✓	✗
Maturité et documentation	Secondaire	✓	✓

2.8.5.4 Décision

Nginx est retenu. Le serveur sera administré par les équipes IT de Toulouse INP. Disposer d’une configuration statique dans des fichiers texte qu’un administrateur système peut lire et modifier sans expertise Docker est, dans ce contexte institutionnel, plus important que l’automatisation dynamique proposée par Traefik.

2.9 Stratégie de performance base de données

Cette section présente les enjeux de performance liés à la croissance des données, évalue les deux approches disponibles, puis justifie la stratégie retenue.

2.9.1 Contexte et enjeux

Plusieurs tables du projet, telles que les mobilités, les affectations ou les imports, grossissent d’une campagne à l’autre. Le système doit pouvoir les filtrer rapidement par année universitaire, que ce soit pour afficher le tableau de bord RI, calculer les durées CTI ou lancer l’algorithme d’affectation. Deux approches ont été évaluées pour répondre à cet enjeu.

2.9.2 Alternatives évaluées

Ces deux approches sont détaillées ci-dessous.

2.9.2.1 Partitionnement par année

Le partitionnement consiste à découper physiquement une table en sous-tables, une par valeur de la colonne de partition, ici par année universitaire. PostgreSQL ne lit alors que la partition concernée lors d’une requête filtrée.

Avantages :

- Réduction significative du volume scanné pour les requêtes filtrées par année
- Efficace lorsque les tables atteignent plusieurs millions de lignes

Limites :

- La colonne de partitionnement doit être incluse dans toutes les contraintes d’unicité, ce qui peut créer des conflits avec les migrations Django au fil des sprints
- Complexité supplémentaire du schéma pour un volume qui ne le justifie pas

2.9.2.2 Indexation ciblée

L’indexation ciblée consiste à créer des index sur les colonnes les plus utilisées dans les filtres, sans modifier la structure physique des tables.

Avantages :

- Aucune modification du schéma ni des migrations Django
- Index déclarés directement dans les modèles, visibles dans le code et versionnés avec lui
- Transparent pour l’ORM, sans configuration PostgreSQL séparée

Limites :

- Moins efficace que le partitionnement pour des volumes de plusieurs millions de lignes, sans incidence ici

2.9.3 Synthèse comparative

Le tableau 2.10 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier la stratégie retenue.

TABLE 2.10 – Comparaison des stratégies de performance base de données

Critère	Poids	Partitionnement	Indexation ciblée
Adapté au volume du projet	Bloquant	✗	✓
Compatible avec les migrations Django	Bloquant	Contraintes d'unicité incompatibles avec certaines migrations	✓
Déclaration dans les modèles	Secondaire	✗	✓
Aucune configuration PostgreSQL séparée	Secondaire	✗	✓
Efficacité à très grande échelle	Secondaire	✓	Moins efficace pour plusieurs millions de lignes

2.9.4 Décision

L'**indexation ciblée** est retenue. L'ENSEEIHT compte environ **300 étudiants en mobilité par an**. Même en projetant ce volume sur dix ans, les tables restent très loin du seuil où le partitionnement apporterait un gain mesurable. Des index sont créés sur `academic_year_id` et sur la combinaison (`academic_year_id`, `student_id`) pour les tables les plus sollicitées, directement dans les modèles Django concernés.

2.10 Synthèse des choix technologiques

TABLE 2.11 – Synthèse des choix technologiques

Composant	Technologie retenue	Justification principale
Méthodologie de développement	Scrum	Développement itératif et adaptation progressive aux besoins du service RI
Frontend	Next.js 16	Rendu hybride SSR/CSR, séparation sécurisée des espaces RI et étudiant
Backend framework	Django 5	ORM, administration, gestion des droits et logique métier intégrés
Couche API	Django Ninja	Support natif de l'asynchrone, validation Pydantic, documentation automatique

Composant	Technologie retenue	Justification principale
Base de données	PostgreSQL 16	Transactions robustes, support JSON et bonne intégration avec Django
Gestionnaire de tâches asynchrones	Django-Q2 + PostgreSQL	Aucun service supplémentaire et supervision intégrée à Django
Stockage fichiers	MinIO (S3-compatible)	Stockage objet auto-hébergé, chiffrement SSE-S3, conforme RGPD, intégré comme container Docker sans service externe
Reverse proxy	Nginx	Configuration simple, stable et adaptée à un environnement institutionnel
Stratégie de performance BDD	Indexation ciblée	Solution simple et suffisante pour le volume de données du projet
Conteneurisation	Docker + Docker Compose	Déploiement reproductible et simplification de l'environnement
Gestion des états métier	django-fsm	Transitions sécurisées et intégration directe avec Django ORM
Journalisation et audit	Module d'audit applicatif développé en interne (app/audit/logger.py)	Traçabilité ciblée des actions administratives via <code>log_action()</code>
Authentification	CAS/OIDC	Exigence institutionnelle : SSO Toulouse INP avec détection automatique du rôle depuis le token

Conclusion

Ce chapitre a précisé les besoins du système et justifié les choix technologiques retenus pour le construire. Ces choix posent le socle de la conception détaillée, présentée au chapitre suivant.

Conception du système

Plan

Introduction	55
3.1 Product Backlog	55
3.1.1 Priorités MoSCoW	55
3.1.2 Story points	55
3.1.3 Tableau du Product Backlog	56
3.2 Planification des sprints	59
3.2.1 Diagramme de Gantt	60
3.3 Approches de conception : organisation du code et style architectural	63
3.3.1 Organisation du code : approche modulaire par domaine métier	63
3.3.1.1 Découpage en modules Django	63
3.3.1.2 Langage commun	64
3.3.1.3 Encapsulation des règles métier	64
3.3.2 Style architectural : architecture en couches	65
3.4 Architecture logicielle	65
3.4.1 Vue d'ensemble	65
3.4.2 Couche présentation : Next.js 16	66
3.4.3 Couche applicative : Django 5 + Django Ninja	67
3.4.4 Couche traitement asynchrone : Django-Q2	67
3.4.5 Couche données : PostgreSQL 16	67
3.4.6 Couche intégration : Adaptateurs ETL	68
3.4.6.1 Stratégie d'upsert	68
3.4.6.2 Clé de jointure : l'INE	69
3.4.6.3 Stratégie de synchronisation	69
3.4.7 Sécurité et contrôle d'accès	70
3.4.7.1 Flux d'authentification CAS	70
3.4.7.2 Contrôle d'accès (RBAC)	71
3.4.7.3 Gestion du renouvellement des tokens JWT	72
3.4.7.4 Journal d'audit	72
3.4.7.5 Protection des données (RGPD)	72
3.5 Architecture physique et déploiement	73
3.5.1 Vue d'ensemble	73
3.5.2 Containers Docker	74
3.5.3 Flux réseau	75

3.5.4	Environnements	75
3.5.5	CI/CD	76
3.5.6	Stratégie de migration de schéma	77
3.5.7	Sauvegarde de la base de données	77
3.5.8	Gestion des secrets	77
3.6	Modèle de données	77
3.6.1	Organisation par packages	77
3.6.2	Principes structurants	78
3.6.2.1	L'année universitaire comme axe temporel	78
3.6.2.2	L'INE comme identifiant pivot	78
3.6.2.3	Hierarchie quotas INP / quotas N7	79
3.7	Flux de données : de la source au workflow	79
3.7.1	Détail de l'étape de nettoyage et validation	80
3.7.1.1	Description du processus de transformation	80
3.7.1.1.1	Standardisation des formats	80
3.7.1.1.2	Normalisation sémantique	80
3.7.1.1.3	Gestion des valeurs manquantes	81
3.7.1.1.4	Détection et gestion des doublons	81
3.7.1.1.5	Validation métier	81
3.7.1.1.6	Traçabilité et gestion des erreurs	81
3.7.1.2	Architecture du pipeline de nettoyage	82
3.7.1.3	Technologies et outils utilisés	83
3.8	Gestion des risques	83
	Conclusion	84

Introduction

Le chapitre précédent a justifié les choix technologiques de la plateforme. Ce chapitre en présente la conception globale : backlog produit, architecture logicielle et modèle de données.

3.1 Product Backlog

Cette section présente le backlog produit du projet : la méthode de priorisation retenue, l'estimation en story points et le tableau complet des user stories.

3.1.1 Priorités MoSCoW

Chaque user story est priorisée selon la méthode MoSCoW, qui permet de séparer l'essentiel de l'accessoire et d'éviter que des fonctionnalités secondaires ne retardent la livraison du cœur du système.

- **M : Must Have** : fonctionnalités sans lesquelles le système ne peut pas remplir son objectif. Leur absence rendrait le projet inutilisable pour le service RI.
- **S : Should Have** : fonctionnalités importantes, mais dont le report à une version suivante reste acceptable sans compromettre le MVP.
- **C : Could Have** : fonctionnalités souhaitables, planifiées uniquement si la capacité restante du sprint le permet une fois les priorités supérieures traitées.
- **W : Won't Have (this time)** : fonctionnalités écartées de cette itération, non de façon définitive, mais documentées pour ne pas créer d'attente non maîtrisée ; elles pourront être réévaluées lors d'une itération future.

3.1.2 Story points

L'effort de chaque story est estimé en story points selon l'échelle de Fibonacci. Cette échelle non linéaire reflète le fait que l'incertitude croît avec la complexité : la différence entre 5 et 8 points n'est pas la même que celle entre 1 et 2. Le tableau 3.1 donne les correspondances retenues pour ce projet, avec des exemples concrets.

TABLE 3.1 – Échelle des story points

SP	Effort estimé	Exemples typiques
1	Très faible : quelques heures	Ajout d'un champ, endpoint simple
2	Faible : une demi-journée	Nouveau modèle Django, endpoint CRUD
3	Modéré : une journée	Service métier, FSM à 3 états
5	Significatif : deux jours	Pipeline d'import, interface multi-composants
8	Élevé : trois à quatre jours	Intégration SSO, import Excel avec rapport
13	Très élevé : une semaine	Algorithme d'affectation, moteur de recommandation
21	Exceptionnel : plus d'une semaine	Chatbot RAG complet

3.1.3 Tableau du Product Backlog

Le tableau ci-dessous liste l'ensemble des user stories au niveau produit, avec leur priorité MoSCoW, leur estimation en story points et le sprint auquel elles sont affectées. Les critères d'acceptation détaillés, tâche par tâche, ne sont pas répétés ici : ils sont présentés dans les sprint backlogs correspondants au chapitre 5, afin d'éviter toute redondance ou incohérence entre les deux niveaux de backlog.

TABLE 3.2 – Product Backlog

ID	Titre	User Story	Prior.	SP	Sprint
Fondations techniques					
US-01	Référentiels de base	En tant qu' administrateur RI , je veux disposer d'un référentiel stable des pays, départements et universités partenaires (synchronisé depuis MoveON) afin que toutes les entités du système s'appuient sur des données de référence cohérentes.	M	5	S1
US-02	Année universitaire et FSM	En tant qu' administrateur RI , je veux que le cycle de vie de l'année universitaire soit géré par une machine à états (initialisation → recommandation → candidature → import → pré-affectation → validation → publiée → finalisation CTI → clôturée) afin de contrôler chaque phase de la campagne de manière sécurisée.	M	5	S1
US-03	Accords et quotas	En tant qu' administrateur RI , je veux un modèle de données complet pour les accords et leurs quotas (INP global, estimation N7, ajustement N7, répartition par département), synchronisé depuis MoveON, afin d'avoir une base fiable pour l'algorithme d'affectation.	M	8	S1
Données et traçabilité					

ID	Titre	User Story	Prior.	SP	Sprint
US-04	Audit log et traçabilité	En tant qu' administrateur RI , je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.	M	3	S2
US-05	Données étudiants	En tant qu' administrateur RI , je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.	M	8	S2
US-06	Vœux étudiants	En tant qu' administrateur RI , je veux importer les dossiers de candidature et les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.	M	5	S2
US-07	Rapport d'import	En tant qu' administrateur RI , je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.	M	3	S2
Algorithme d'affectation					
US-08	Algorithme Gale-Shapley	En tant qu' administrateur RI , je veux lancer l'algorithme d'affectation (Deferred Acceptance) qui affecte chaque étudiant à son meilleur vœu disponible en respectant les quotas par département, afin de produire une proposition équitable et stable.	M	13	S3
US-09	Supervision des affectations	En tant qu' administrateur RI , je veux consulter les propositions d'affectation, les ajuster manuellement pour les cas particuliers et tracer chaque modification avec un motif, avant de valider et publier les résultats.	M	8	S3
US-10	Consultation des résultats	En tant qu' étudiant , je veux consulter mon affectation une fois les résultats publiés afin de connaître ma destination et les démarches à suivre.	M	3	S3
Sources complémentaires et mobilités					

ID	Titre	User Story	Prior.	SP	Sprint
US-11	Répartition des quotas par département	En tant qu' administrateur RI , je veux que le système calcule automatiquement la répartition des places de chaque accord entre les départements pédagogiques (SN, 3EA, MF2E) selon la méthode Hamilton (plus grands restes), afin que l'algorithme d'affectation dispose de quotas par département précis, équitables et vérifiables.	M	5	S4
US-12	Stages internationaux (Eudonet)	En tant qu' administrateur RI , je veux importer et synchroniser les conventions de stages internationaux depuis Eudonet afin de les comptabiliser dans le calcul CTI de chaque étudiant.	M	8	S4
US-13	Mobilités entrantes (Excel)	En tant qu' administrateur RI , je veux importer les mobilités entrantes via un template Excel standardisé afin de centraliser le suivi des étudiants internationaux accueillis à l'ENSEEIH.	M	8	S4
US-14	Réconciliation sans INE	En tant qu' administrateur RI , je veux que le système me propose des correspondances pour les enregistrements sans INE valide (via nom, prénom, e-mail) afin de résoudre manuellement les cas ambigus.	S	5	S4
US-15	Déclaration mobilité complémentaire	En tant qu' étudiant , je veux déclarer une expérience internationale hors cursus (summer school, séjour linguistique, programme court) avec dépôt de justificatifs, afin qu'elle soit prise en compte dans mon calcul de durée CTI.	M	8	S4
US-16	Validation des mobilités complémentaires	En tant qu' administrateur RI , je veux consulter les justificatifs soumis, valider ou rejeter chaque demande avec un motif, afin de maintenir la qualité des données comptabilisées dans le calcul CTI.	M	5	S4
Conformité CTI et authentification					
US-17	Calcul automatique des durées CTI	En tant qu' administrateur RI , je veux que le système calcule automatiquement la durée totale de mobilité de chaque étudiant (séjours académiques, stages internationaux, mobilités complémentaires validées) afin de vérifier la conformité CTI sans intervention manuelle.	M	8	S5
US-18	Affichage des durées de mobilité dans le tableau de bord étudiant	En tant qu' étudiant , je veux visualiser ma durée cumulée de mobilité afin de suivre mon avancement.	M	3	S5

ID	Titre	User Story	Prior.	SP	Sprint
US-19	Export rapport CTI	En tant qu' administrateur RI , je veux exporter un rapport CTI pour l'ensemble des étudiants de la promotion afin de le soumettre lors des audits.	S	5	S5
US-20	Tableaux de bord analytiques	En tant qu' administrateur RI , je veux disposer de tableaux de bord analytiques (taux d'affectation par département, évolution des flux entrants/sortants, destinations les plus demandées, indicateurs CTI) afin d'alimenter le pilotage opérationnel et la réflexion stratégique sur les partenariats.	M	8	S5
US-21	Authentification SSO	En tant qu' utilisateur , je veux m'authentifier via le SSO institutionnel (CAS) afin d'accéder à mon espace personnalisé (étudiant ou administrateur) sans créer de compte séparé.	M	8	S5
Fonctionnalités intelligentes					
US-22	Moteur de recommandation	En tant qu' étudiant , je veux recevoir des suggestions de destinations adaptées à mon profil (GPA, département, historique des affectations similaires) afin de cibler mes vœux de manière réaliste avant la saisie dans MoveON.	C	13	S6
US-23	Assistant virtuel (chatbot RAG)	En tant qu' étudiant , je veux poser des questions sur les procédures, critères d'éligibilité et démarches à un assistant virtuel afin d'obtenir des réponses immédiates sans solliciter le service RI.	C	21	S6

3.2 Planification des sprints

Le développement est découpé en six sprints de trois semaines chacun, suivant une progression de l'infrastructure vers les fonctionnalités visibles. Les sprints sont ordonnés par dépendances techniques : rien dans le Sprint 2 ne peut exister sans le Sprint 1, et l'algorithme du Sprint 3 dépend directement des données produites par le Sprint 2. Les cinq premiers sprints couvrent l'engagement MVP ; le Sprint 6, classé *Could Have*, couvre les fonctionnalités intelligentes (moteur de recommandation, chatbot RAG). Le tableau 3.3 détaille la répartition.

TABLE 3.3 – Planification des sprints

Sprint	Thème	User stories couvertes
Sprint 1	Fondations techniques	US-01 à US-03 : Référentiels de base (sync MoveON), année universitaire et FSM, accords et quotas (sync MoveON)
Sprint 2	Données et traçabilité	US-04 à US-07 : Audit log, données étudiants (import Pégase), vœux étudiants (import MoveON), rapport d'import
Sprint 3	Algorithme d'affectation	US-08 à US-10 : Algorithme Gale-Shapley, supervision et publication des résultats, consultation des résultats par l'étudiant
Sprint 4	Sources complémentaires et mobilités	US-11 à US-16 : Répartition des quotas par département (Hamilton), stages internationaux (import Eudonet), mobilités entrantes (import Excel), réconciliation sans INE, déclaration et validation des mobilités complémentaires
Sprint 5	Conformité CTI et authentification	US-17 à US-21 : Calcul des durées CTI, tableau de bord étudiant, export CTI, tableaux de bord analytiques, authentification SSO
Sprint 6	Fonctionnalités intelligentes (V2)	US-22 à US-23 : Moteur de recommandation de destinations, assistant virtuel (chatbot RAG)

3.2.1 Diagramme de Gantt

La figure 3.4 présente le calendrier de réalisation sur l'ensemble des sprints, en indiquant pour chaque tâche la semaine de début, la durée estimée et l'éventuel chevauchement avec les tâches adjacentes. Ce calendrier reste indicatif et prévisionnel : il sert à estimer la charge globale et à cadrer le projet en amont, mais le contenu précis de chaque sprint continue d'être ajusté à chaque itération, conformément à l'approche Scrum retenue.

TABLE 3.4 – Diagramme de Gantt

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Phase 1 : Cadrage et analyse																								
Analyse des besoins																								
Rédaction CDC / spécifications																								
Modélisation des données																								
Conception architecture																								
Phase 2 : Environnement et DevOps																								
Setup Docker / environnement																								
Pipeline CI/CD (Git-Lab/GitHub)																								
Configuration PostgreSQL																								
Setup Django-Q2 / MinIO																								
Monitoring (Sentry / Grafana)																								
Phase 3 : Développement itératif																								
Sprint 1																								
Sprint 2																								
Sprint 3																								
Sprint 4																								

Activité	Mois 1				Mois 2				Mois 3				Mois 4				Mois 5				Mois 6			
	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13	S14	S15	S16	S17	S18	S19	S20	S21	S22	S23	S24
Sprint 5																								
Sprint 6 (V2)																								
Phase 4 : Tests et qualité																								
Tests unitaires (continus)																								
Tests d'intégration (continus)																								
Tests de charge (Locust)																								
Audit sécurité (OWASP ZAP)																								
UAT avec service RI																								
Phase 5 : Livraison																								
Documentation technique																								
Déploiement staging																								
Livraison finale																								

3.3 Approches de conception : organisation du code et style architectural

Avant de détailler l'architecture technique, deux décisions structurantes doivent être présentées : l'approche retenue pour organiser le code, et le style architectural choisi pour l'ensemble du système.

3.3.1 Organisation du code : approche modulaire par domaine métier

Le code est organisé en modules Django (*apps*) alignés sur les domaines métier de la mobilité internationale, plutôt que sur des considérations purement techniques. Cette organisation s'inspire de certains principes du *Domain-Driven Design* (DDD), notamment un vocabulaire métier partagé, l'encapsulation des règles par module, et des dépendances orientées dans un seul sens. Elle n'en reprend cependant pas les techniques les plus avancées : pas de règles obligeant plusieurs objets liés à toujours rester cohérents entre eux (agrégats), pas d'historique de chaque modification plutôt que du seul résultat final (*event sourcing*), pas de séparation entre le chemin utilisé pour lire les données et celui utilisé pour les modifier (CQRS), ni de limites strictement définies entre les différentes parties du système (*bounded contexts*). Ces techniques auraient ajouté une complexité disproportionnée pour un système développé par une seule personne sur la durée d'un PFE ; l'objectif retenu est la clarté du découpage et l'indépendance des modules, pas la conformité à un modèle théorique.

3.3.1.1 Découpage en modules Django

Quatorze apps Django, chacune exposant sa propre API (Django Ninja), portent la logique métier effectivement déployée. Le tableau 3.5 les met en correspondance avec les sept packages du modèle de données présenté au tableau 3.8 : plusieurs apps peuvent alimenter un même package lorsque leurs entités sont fortement liées.

TABLE 3.5 – Correspondance entre les apps Django et les packages du modèle de données

Package	App(s) Django	Périmètre
Reference	<code>reference</code>	Référentiels stables : pays (avec région CTI), départements, niveaux d'études (dont le niveau terminal diplômé) et parcours
Academic Year	<code>academic</code>	Année universitaire et cycle de campagne, piloté par une FSM
Students	<code>students</code>	Données académiques importées de Pégase, inscriptions, parcours
Institutions	<code>institutions</code>	Universités partenaires et organismes d'accueil
Agreements & Quotas	<code>mobility</code>	Accords de mobilité et hiérarchie de quotas (INP, N7, département)
Mobilities	<code>outgoing,</code> <code>incoming,</code> <code>internships,</code> <code>complementary,</code> <code>cti</code>	Vœux et affectations issues de l'algorithme Gale-Shapley pour les mobilités sortantes, quatre types de mobilité au total, et durée CTI consolidée
Import	<code>imports</code>	Pipeline ETL de staging (MoveON, Pégase, Eudonet, Excel) : enregistrements bruts avant transformation et rapports de session d'import (compteurs succès / erreurs / doublons)

Au sein du package `Mobilities`, l'app `outgoing` porte à la fois la logique de vœux et d'affectations et l'entité *mobilité sortante académique* qui en résulte : les deux sont trop imbriquées pour être séparées en deux packages distincts sans complexifier inutilement le code. Cinq apps supplémentaires jouent un rôle transversal, hors de ce découpage par package : `audit` (traçabilité des actions via un module applicatif développé en interne), `alerts` (notifications système), `analytics` (tableaux de bord de flux de mobilité), `rag` (assistant conversationnel permettant aux étudiants d'interroger la base de connaissances de la mobilité internationale) et `recommendation` (moteur de recommandation de destinations fondé sur le profil académique et les vœux historiques de l'étudiant). S'y ajoutent `integrations` (adaptateurs MoveON, Pégase, Eudonet).

3.3.1.2 Langage commun

Les noms des classes, des champs et des méthodes reprennent directement le vocabulaire du service RI (accord, quota, vœu, affectation, mobilité complémentaire...), plutôt qu'un vocabulaire technique générique. Ce vocabulaire partagé facilite les échanges avec l'experte du domaine et réduit les risques d'ambiguïté lors de la validation des fonctionnalités.

3.3.1.3 Encapsulation des règles métier

Chaque app encapsule ses propres règles métier dans des fichiers `services.py` distincts des vues et de l'API, ce qui permet de les faire évoluer indépendamment du reste du système. Cette séparation rend le système plus robuste face aux évolutions de besoins, inévitables

dans un contexte de développement itératif où certaines règles, comme les critères de priorité dans l’affectation, se précisent au fil des sprints.

3.3.2 Style architectural : architecture en couches

Le système est structuré selon une architecture en couches, où chaque niveau a une responsabilité clairement délimitée. Ce style architectural a été préféré à une architecture microservices, qui aurait été surdimensionnée pour l’échelle du projet, les contraintes d’hébergement de Toulouse INP et une équipe réduite à une seule développeuse.

Le système s’articule autour de cinq couches distinctes :

- **La couche présentation (Next.js 16)** : responsable du rendu des interfaces et de la gestion des interactions utilisateur.
- **La couche applicative (Django 5 + Django Ninja)** : porte la logique métier, la validation et l’orchestration des apps.
- **La couche asynchrone (Django-Q2)** : prend en charge les traitements longs découplés des requêtes, tels que l’algorithme d’affectation ou les synchronisations nocturnes.
- **La couche données (PostgreSQL + MinIO)** : assure la persistance relationnelle et le stockage des fichiers justificatifs.
- **La couche intégration** (adaptateurs **MoveON**, **Pégase**, **Eudonet**) : gère la connexion avec les systèmes externes via des adaptateurs dédiés.

La section suivante détaille l’implémentation concrète de cette architecture en couches, avec les technologies retenues pour chacune d’entre elles.

3.4 Architecture logicielle

Cette section détaille la mise en œuvre concrète de l’architecture en couches présentée précédemment : une vue d’ensemble des flux entre couches, puis le rôle et les technologies retenues pour chacune d’entre elles.

3.4.1 Vue d’ensemble

La plateforme repose sur une architecture en cinq couches qui s’appuie sur le modèle 3-tiers classique tout en l’enrichissant d’une couche asynchrone dédiée aux traitements longs. Chaque couche a une frontière bien définie : aucun composant Next.js n’accède à la base de données directement, aucune tâche asynchrone ne peut bloquer une requête HTTP. La figure 3.1 donne une vue d’ensemble du système.

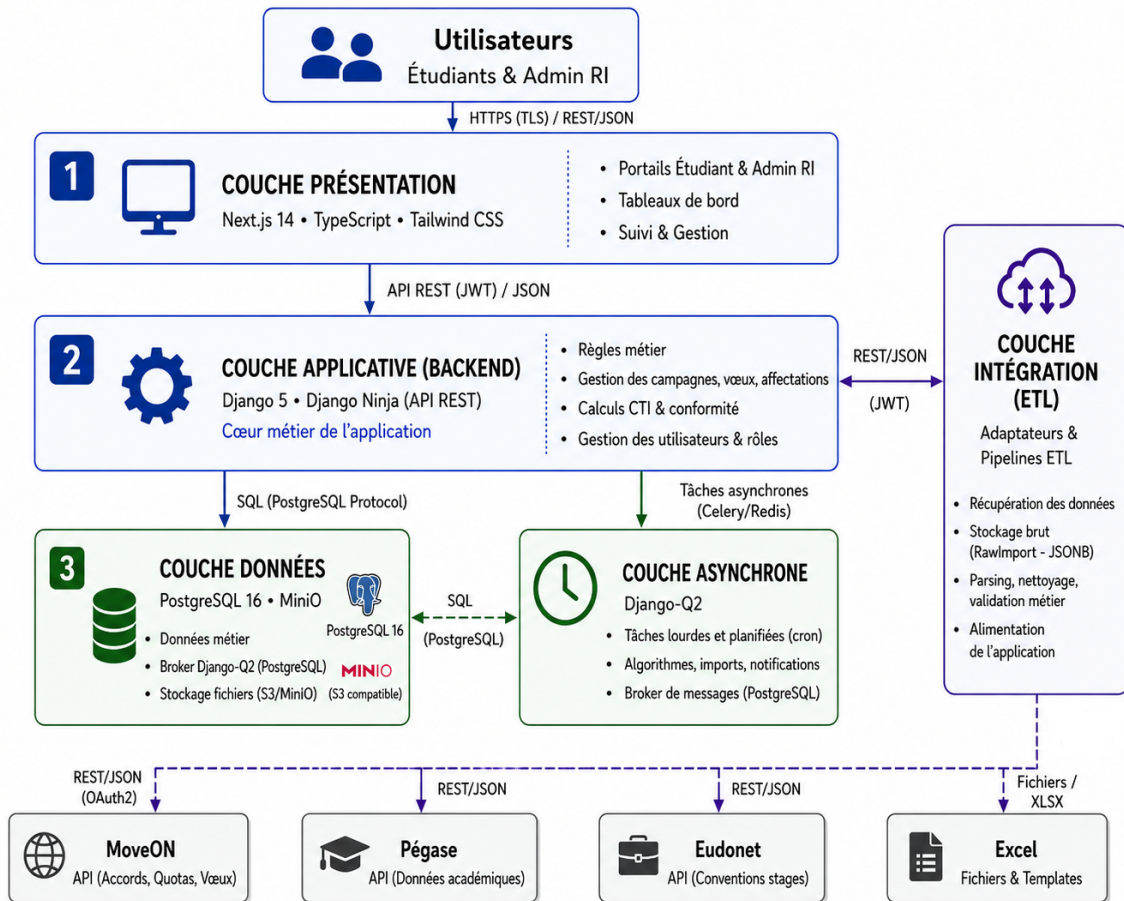


FIGURE 3.1 – Architecture logicielle

Les flux entre couches sont unidirectionnels et bien délimités. Le frontend Next.js 16 s'adresse exclusivement à l'API Django Ninja via des appels REST sur HTTPS, jamais directement à la base de données. Django-Q2 s'exécute dans un processus séparé et partage le même PostgreSQL que l'applicatif, ce qui lui permet de fonctionner comme file d'attente sans service externe supplémentaire. PostgreSQL et MinIO sont des services internes non accessibles depuis l'extérieur du réseau Docker. Les adaptateurs ETL sont physiquement séparés des modules métier, précisément parce qu'ils dépendent de sources externes (MoveON, Pégase, Eudonet) dont la disponibilité n'est pas garantie. L'authentification passe par le CAS de Toulouse INP : à chaque requête, le token JWT est validé et le rôle (ADMIN_RI ou ETUDIANT) est extrait pour contrôler les accès.

3.4.2 Couche présentation : Next.js 16

La couche présentation est développée en **Next.js 16** avec **TypeScript** et **Tailwind CSS**. Elle expose deux espaces fonctionnellement distincts, séparés dès le middleware de routage.

Espace administrateur RI : pilotage de campagne, gestion des quotas par accord et par département, supervision des affectations avant publication, instruction des demandes

de mobilités complémentaires, tableaux de bord de conformité CTI.

Espace étudiant : consultation du catalogue de destinations, suivi de l'état de candidature, déclaration d'expériences complémentaires, indicateur personnel de durée CTI.

Toutes les communications entre le frontend et le backend passent par l'API REST, en JSON sur HTTPS, avec un token JWT joint à chaque requête.

3.4.3 Couche applicative : Django 5 + Django Ninja

Le backend concentre la logique métier du système. Il est découpé en modules Django autonomes, chacun responsable d'un domaine précis, selon les principes DDD décrits en début de chapitre. Cette organisation garantit qu'une modification dans le domaine de l'affectation ne peut pas affecter le comportement du domaine CTI ou des imports.

Django Ninja est utilisé pour exposer l'API REST. Chaque module définit ses propres schémas pour valider les données en entrée et structurer les réponses. L'authentification repose sur le middleware JWT (CAS Toulouse INP), qui vérifie le token et associe l'utilisateur à chaque requête.

3.4.4 Couche traitement asynchrone : Django-Q2

Plutôt que de bloquer l'interface le temps qu'un calcul se termine, les traitements longs sont soumis à une file de tâches gérée par **Django-Q2**. Ce dernier utilise PostgreSQL comme file d'attente persistante (backend de tâches), ce qui évite d'introduire Redis dans l'infrastructure. Deux familles de tâches y transitent :

- **Tâches déclenchées par événement** : lancement de l'algorithme d'affectation, recalcul des durées CTI après chaque transition FSM, génération des rapports d'import.
- **Tâches planifiées (cron)** : vérification quotidienne du calendrier de la campagne, qui déclenche automatiquement le passage à la candidature, la clôture des vœux et la clôture de l'année universitaire dès que les dates prévues sont atteintes.

3.4.5 Couche données : PostgreSQL 16

Un seul service de données est utilisé dans l'infrastructure : **PostgreSQL 16**. Ce choix simplifie les sauvegardes et la supervision. PostgreSQL remplit ici trois fonctions différentes :

- **Base de données relationnelle** : toutes les entités métier y sont stockées, avec des index composites sur `academic_year_id` pour les performances de filtrage par campagne.
- **File d'attente Django-Q2** : la table `django_q_task` stocke les tâches en attente.
- **Stockage des données brutes (JSONB)** : la table `RawImport` conserve les réponses JSON brutes de chaque appel API avant transformation, garantissant la possibilité de relancer les imports.

Le stockage des fichiers (justificatifs de mobilités complémentaires, templates Excel) est assuré par **MinIO**, un serveur de stockage objet compatible S3, déployé comme container Docker interne.

3.4.6 Couche intégration : Adaptateurs ETL

Chaque source externe est encapsulée dans un adaptateur dédié, isolé du reste du code métier. Quatre adaptateurs ont été développés :

- `adapters/moveon.py` : extraction des accords bilatéraux, des quotas INP et des candidatures/vœux ;
- `adapters/pegase.py` : extraction des données académiques étudiants (INE, GPA annuel, département d'inscription) ;
- `adapters/eudonet.py` : extraction des conventions de stages à l'international ;
- `adapters/excel.py` : traitement des fichiers Excel standardisés pour chaque domaine (étudiants, accords, vœux, mobilités entrantes) en fallback lorsque l'API source est indisponible.

Tous les adaptateurs suivent le même enchaînement : récupération des données depuis la source → stockage brut en JSONB (`RawImport`) → parsing et standardisation → validation métier → persistance en base → génération du rapport d'import. En cas d'indisponibilité de la source, le client HTTP tente successivement plusieurs points d'entrée alternatifs (par exemple `/internships` puis `/api/internships` pour Eudonet) ; si tous échouent, une exception est levée et l'échec de la tâche est consigné par Django-Q2. Un import Excel de secours reste disponible pour chaque source, ce qui permet à l'administrateur de poursuivre le traitement en cas d'indisponibilité prolongée.

3.4.6.1 Stratégie d'upsert

À la phase de persistance, chaque enregistrement est traité selon une logique d'upsert. La clé métier (généralement l'INE ou l'identifiant de l'accord) permet de déterminer si l'enregistrement est nouveau ou déjà connu. S'il est nouveau, il est inséré. S'il existe, ses champs sont comparés un à un avec la version en base ; en l'absence de différence, l'enregistrement est ignoré.

La suite dépend de la source. Pour les données étudiants (Pégase) et les vœux (MoveON), une différence détectée, par exemple un GPA révisé, est appliquée directement, et le rapport d'import indique le nombre d'enregistrements créés et mis à jour. Pour les stages internationaux importés depuis Eudonet, en revanche, une différence n'est jamais appliquée automatiquement : l'ancienne et la nouvelle valeur sont conservées côte à côte dans le rapport, et c'est l'administrateur qui décide, depuis l'interface, d'appliquer la mise à jour, de conserver les deux enregistrements séparément, ou d'ignorer l'anomalie.

3.4.6.2 Clé de jointure : l'INE

L'identifiant national étudiant (INE) est la clé partagée entre MoveON, Pégase, Eudonet et le CAS institutionnel. Toutes les données importées sont croisées via l'INE pour construire une vue unifiée du dossier étudiant.

Lorsque l'INE est absent d'une source (notamment Eudonet), une stratégie de réconciliation par score de similarité est appliquée : chaque candidat est scoré selon une pondération fixe, 50 % sur le nom (similarité textuelle `SequenceMatcher`), 30 % sur le prénom (même mesure) et 20 % sur la correspondance exacte de l'e-mail. Les candidats dont le score composite est inférieur à 0,60 sont éliminés ; ceux dont il dépasse 0,85 sont classés confiance élevée. Les cas ambigus (confiance faible ou moyenne) sont remontés pour validation manuelle par le service RI.

3.4.6.3 Stratégie de synchronisation

Le tableau 3.6 récapitule, pour chaque source externe, le déclencheur de synchronisation et les particularités de traitement associées. Ces intégrations restent toutes en lecture seule : aucune synchronisation n'écrit vers le système source. De même, lorsqu'un enregistrement importé ne peut pas être rattaché à un étudiant faute d'INE valide, la réconciliation par similarité de nom s'applique quelle que soit la source concernée, et non à une seule d'entre elles.

TABLE 3.6 – Stratégie de synchronisation par source

Source	Déclencheur	Remarque
MoveON	À la demande via l'interface admin	Vœux et accords mis à jour automatiquement à chaque synchronisation
Pégase	Initialisation de campagne + à la demande	Les champs modifiés (GPA, département, niveau) sont mis à jour automatiquement à chaque synchronisation
Eudonet	À la demande via l'interface admin	Une différence détectée n'est pas appliquée automatiquement : l'administrateur choisit d'appliquer la mise à jour, de conserver les deux versions séparément, ou d'ignorer l'anomalie
Excel	Upload manuel par l'administrateur	Fallback pour toutes les sources en cas d'indisponibilité API ; pour les mobilités entrantes, un doublon détecté suit le même principe que pour Eudonet et reste soumis à la décision de l'administrateur

3.4.7 Sécurité et contrôle d'accès

Cette section présente les mécanismes qui protègent l'accès au système : l'authentification via le CAS institutionnel, l'attribution des rôles, le renouvellement des jetons, la traçabilité des actions et la protection des données personnelles.

3.4.7.1 Flux d'authentification CAS

L'application délègue entièrement l'authentification au CAS institutionnel de Toulouse INP (protocole CAS v2). Aucun mot de passe n'est stocké côté applicatif. Le flux se déroule en quatre étapes :

1. L'utilisateur accède à l'application et est redirigé vers le portail SSO institutionnel.
2. Après authentification CAS, un JWT signé est émis par le backend et transmis à l'application.
3. Le backend valide le ticket auprès du CAS, qui renvoie l'INE et le rôle de l'utilisateur (ADMIN_RI ou ETUDIANT) directement dans sa réponse.
4. Le frontend reçoit le rôle et affiche l'espace correspondant. Les tokens ont une durée de vie courte (15 minutes) et sont renouvelés silencieusement sans interrompre la navigation.

La figure 3.2 détaille ce flux sous forme de diagramme de séquence, ticket CAS et rafraîchissement du token inclus.

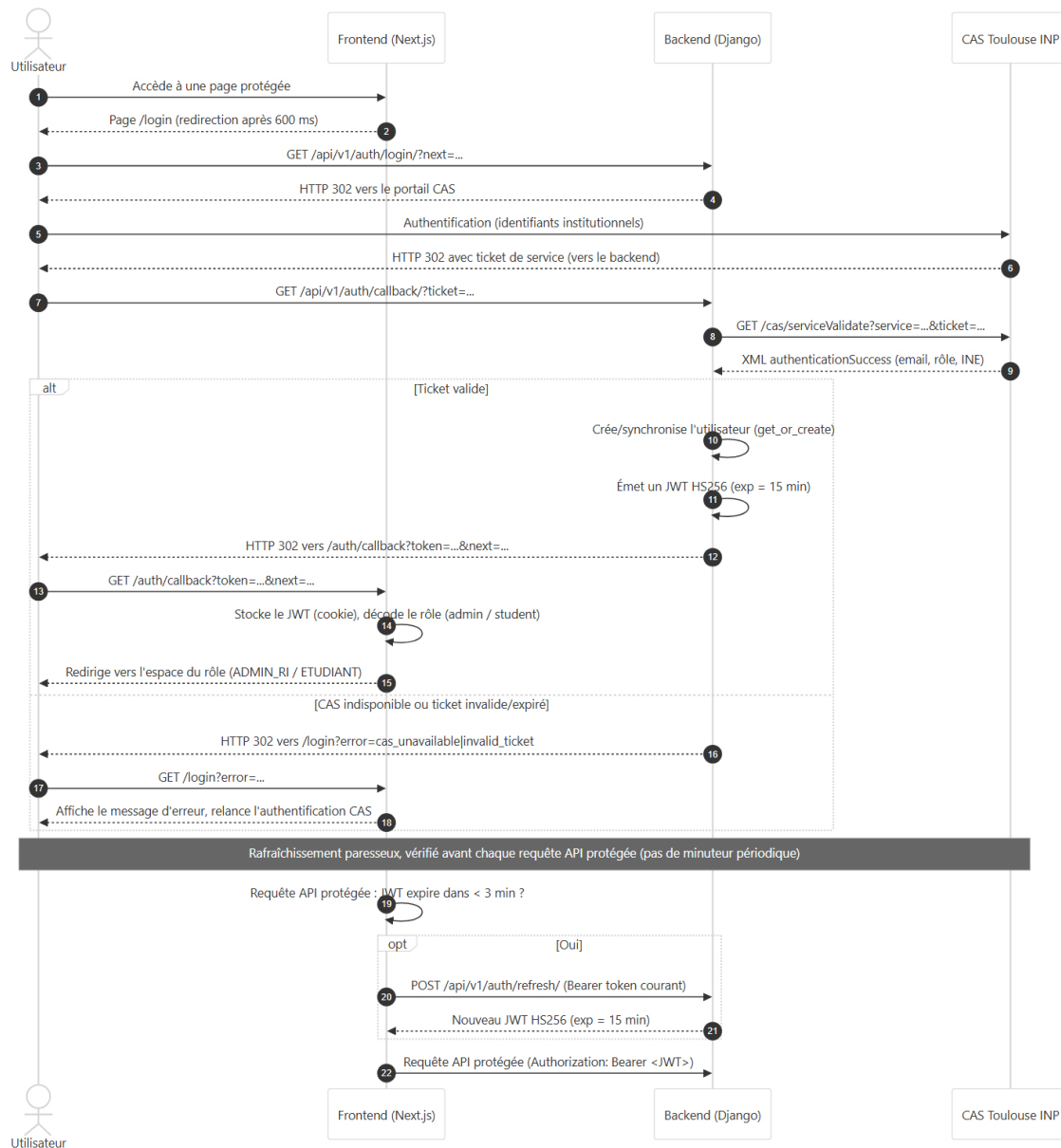


FIGURE 3.2 – Diagramme de séquence : authentification CAS et rafraîchissement du JWT

3.4.7.2 Contrôle d'accès (RBAC)

Les rôles sont déduits automatiquement depuis la réponse du CAS institutionnel, sans gestion manuelle dans l'application :

- **ADMIN_RI** : accès complet aux fonctions d'administration, à savoir les imports, le lancement de l'algorithme, les validations et les exports CTI. Toute action est enregistrée dans le journal d'audit.
- **ETUDIANT** : accès strictement limité à son propre dossier. Un contrôle au niveau de l'objet (*object-level permission*) est appliqué sur chaque endpoint exposant des données personnelles, de sorte qu'un étudiant ne puisse pas accéder au dossier d'un autre.

3.4.7.3 Gestion du renouvellement des tokens JWT

Les tokens JWT ont une durée de vie de 15 minutes pour limiter l'exposition en cas d'interception. Côté Next.js, un mécanisme de renouvellement proactif intégré au client HTTP vérifie l'expiration du token avant chaque requête (seuil à 3 minutes) et appelle l'endpoint `/api/v1/auth/refresh/` du backend pour obtenir un nouveau JWT sans interrompre la navigation. Côté Django, chaque requête est vérifiée par un middleware qui contrôle la signature et l'expiration du token ; un token expiré retourne un 401 et force le frontend à initier un nouveau cycle. La durée de vie courte (15 minutes) limite l'exposition en cas d'interception ; après expiration, un nouveau cycle d'authentification est nécessaire.

3.4.7.4 Journal d'audit

La traçabilité des actions administratives est assurée par un module applicatif développé en interne (`app/audit/logger.py`). La fonction `log_action()` est appelée explicitement dans les vues et les transitions FSM pour tracer les actions administratives. Les événements enregistrés couvrent :

- Toutes les transitions FSM (changements d'état d'une année universitaire, d'un dossier, d'une affectation) ;
- Modifications manuelles des affectations par le service RI (avec champ *motif* obligatoire) ;
- Lancements d'imports ETL et résultats (source, nombre d'enregistrements traités, erreurs) ;
- Validations ou rejets de mobilités complémentaires ;
- Exports de rapports CTI.

Chaque entrée contient l'identifiant de l'auteur, l'horodatage UTC, l'entité concernée (table et clé primaire), l'action effectuée, et les valeurs avant/après modification. Le journal est stocké dans une table PostgreSQL dédiée (`auditlog_logentry`), accessible en lecture seule : aucune entrée ne peut être modifiée ou supprimée depuis l'application, toute suppression nécessitant un accès direct à la base, lui-même tracé au niveau opérateur.

3.4.7.5 Protection des données (RGPD)

La plateforme traite des données personnelles étudiantes (INE, email, genre, nationalité, identifiant Pégase) soumises au RGPD. Quatre mesures techniques ont été intégrées dès la conception :

- **Chiffrement au repos** : l'e-mail et l'identifiant Pégase de l'étudiant sont chiffrés en base via `pgcrypto` ; leur lecture requiert la clé de déchiffrement, qui n'est jamais stockée dans le code mais fournie par variable d'environnement.

- **Chiffrement en transit** : TLS est obligatoire sur tous les endpoints ; les communications HTTP sont systématiquement redirigées vers HTTPS par Nginx.
- **Stockage des justificatifs** : les fichiers uploadés (justificatifs de mobilités complémentaires) sont chiffrés côté serveur dans MinIO (SSE-S3). Une tâche supprime les justificatifs dont le délai de conservation a expiré, activée par une planification Django-Q configurée par l'administrateur système.
- **Droit à l'oubli** [26] : sur demande d'un étudiant, le service RI peut anonymiser ses données nominatives (nom, prénom, e-mail, genre, nationalité, identifiant Pégase) via l'API dédiée, à tout moment et non uniquement après diplôme ; les données nécessaires à la cohérence de la campagne (inscriptions, résultats d'affectation, durées CTI) sont conservées sous forme dépersonnalisée. L'anonymisation est refusée tant que l'étudiant est inscrit à une campagne non clôturée, le temps que celle-ci se termine, conformément aux exceptions prévues par ce même règlement. Chaque anonymisation est tracée dans le journal d'audit.

3.5 Architecture physique et déploiement

Cette section présente le déploiement physique du système, les containers Docker qui le composent, les environnements maintenus et les stratégies de migration, de rollback, de sauvegarde et de gestion des secrets qui encadrent la mise en production.

3.5.1 Vue d'ensemble

L'application s'exécute sur un serveur on-premise hébergé par Toulouse INP. Tous les services sont conteneurisés via **Docker Compose**, ce qui garantit que l'environnement de développement et l'environnement de production sont structurellement identiques ; éliminer les surprises du déploiement était une contrainte dès le départ. La figure 3.3 détaille les flux réseau de ce déploiement.

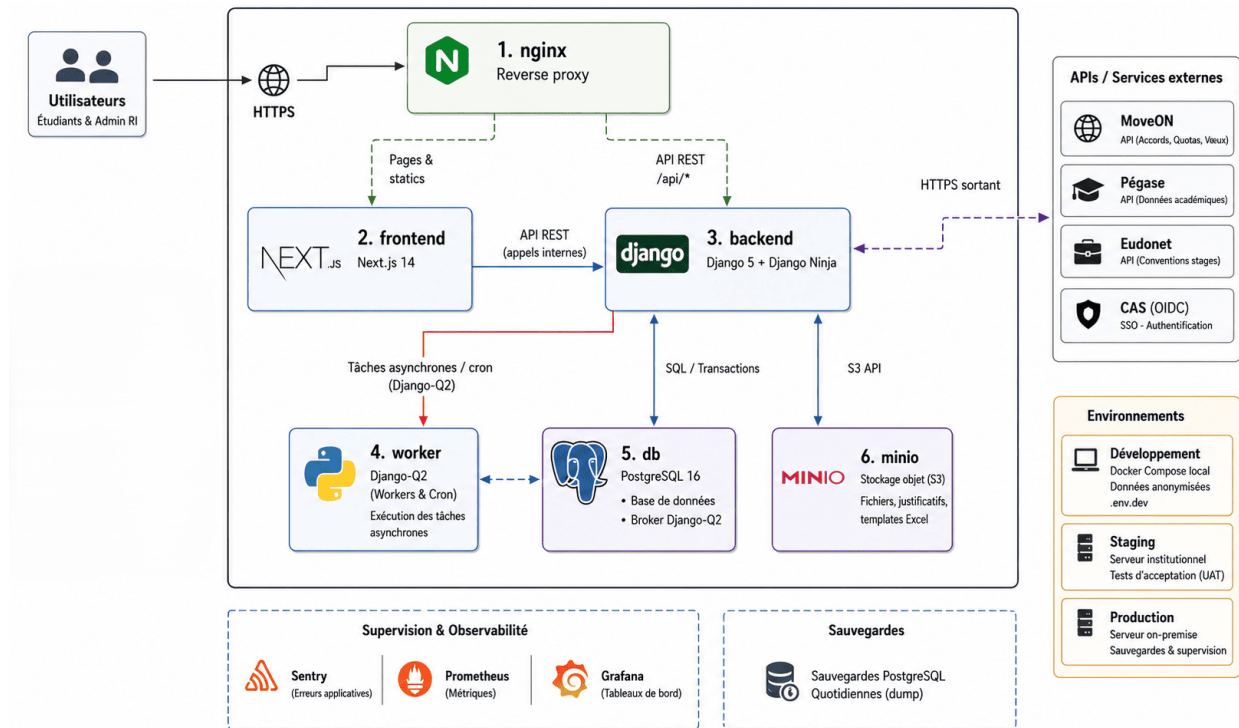


FIGURE 3.3 – Architecture physique

Tout le trafic entrant passe par **Nginx**, le seul container exposé à l'extérieur (ports 80 et 443). Nginx redirige le HTTP vers HTTPS, sert les fichiers statiques Next.js et route les appels API vers **backend:8000**. PostgreSQL, MinIO et le worker Django-Q2 sont confinés dans le réseau Docker interne ; aucune connexion directe depuis l'extérieur n'est possible. Les appels vers les APIs institutionnelles (MoveON, Pégase, Eudonet, CAS) partent exclusivement du container **backend** en HTTPS sortant. Sentry est externe à l'infrastructure et reçoit les rapports d'erreur du backend. Prometheus (collecte de métriques) et Grafana (visualisation) tournent en interne, accessibles uniquement depuis le réseau institutionnel.

3.5.2 Containers Docker

Neuf containers composent l'infrastructure. Le tableau 3.7 en donne le détail.

TABLE 3.7 – Containers Docker et leurs rôles

Container	Image	Rôle
nginx	nginx:alpine	Reverse proxy : SSL/TLS, fichiers statiques, routage vers Django et Next.js
frontend	node:22-slim	Application Next.js (SSR + API routes)
backend	python:3.12-slim	Django 5 + Django Ninja
worker	python:3.12-slim	Django-Q2 cluster (tâches asynchrones et cron)
db	postgres:16-alpine	PostgreSQL, base de données et file d'attente Django-Q2
minio	minio/minio	Stockage objet S3 (justificatifs, fichiers Excel)
prometheus	prom/prometheus	Collecte des métriques système et applicatives
grafana	grafana/grafana	Visualisation des métriques (tableaux de bord internes)
certbot	certbot/certbot	Renouvellement automatique des certificats TLS (Let's Encrypt)

3.5.3 Flux réseau

Nginx est le seul point d'entrée du système : les requêtes vers `/api/*` sont acheminées vers `backend:8000`, toutes les autres vers `frontend:3000`. Le container `backend` dialogue avec la base de données pour les opérations métier et avec MinIO pour la gestion des fichiers. Le `worker` Django-Q2 utilise le même PostgreSQL comme file de messages ; aucun Redis ni RabbitMQ n'est nécessaire. Les appels sortants vers MoveON, Pégase, Eudonet et le CAS passent uniquement par le container `backend`.

3.5.4 Environnements

Trois environnements coexistent, avec une parité de configuration maximale entre eux :

- **Développement** : Docker Compose local, données anonymisées, variables d'environnement dans `.env`.
- **Staging** : copie conforme de la production hébergée sur le serveur institutionnel, utilisée pour les tests d'acceptation (UAT) avec le service RI avant chaque mise en production.
- **Production** : déploiement final sur le serveur on-premise, avec sauvegardes PostgreSQL quotidiennes, alertes via Sentry (standard pour la capture d'exceptions appli-

catives) et métriques via Prometheus/Grafana (standard pour la supervision d'infrastructure conteneurisée).

3.5.5 CI/CD

Le pipeline CI/CD sur **GitHub Actions** repose sur deux workflows distincts. Le premier, `ci.yml`, s'exécute à chaque push sur une branche de travail et à chaque pull request vers `develop` ou `main`; il enchaîne deux étapes :

- **Lint et tests** : le linter `ruff` vérifie la qualité du code Python et `pytest` exécute la suite de tests backend, pendant qu'ESLint et le compilateur TypeScript contrôlent le code frontend et que Vitest exécute ses tests.
- **Build** : construction des images Docker du backend et du frontend, chacune analysée par Trivy pour détecter les vulnérabilités critiques ou élevées.

Le second workflow, `deploy-staging.yml`, ne se déclenche que lors d'un push sur `main`; il automatise le déploiement sur l'environnement de staging, sans intervention manuelle. Le déploiement en production, lui, n'est piloté par aucun workflow automatisé : il est déclenché manuellement par l'administrateur système une fois les tests d'acceptation validés sur staging, ce qui garantit une approbation explicite avant toute mise en production, compte tenu du caractère institutionnel du système et des données personnelles qu'il traite.

La figure 3.4 montre une exécution du workflow `ci.yml` (lint et tests, build) sur GitHub Actions.

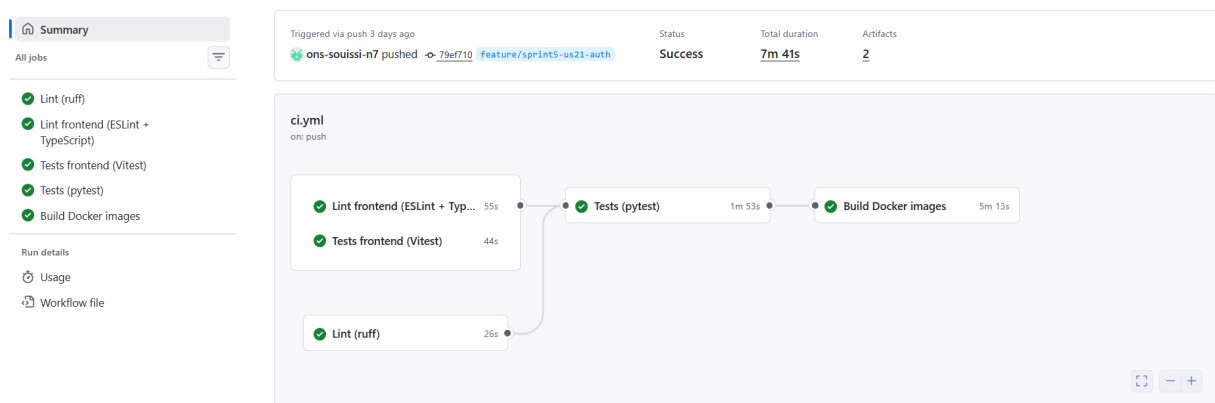


FIGURE 3.4 – Exécution des étapes de lint, tests et build sur GitHub Actions

Une exécution réussie garantit que le code fusionné respecte les règles de qualité définies, backend et frontend, et que les images produites ne contiennent pas de vulnérabilité critique ou élevée, avant tout déploiement.

3.5.6 Stratégie de migration de schéma

Les migrations Django sont appliquées automatiquement avant le démarrage du serveur applicatif, ce qui garantit que le code et le schéma de base de données sont toujours cohérents. La stratégie retenue est celle des migrations additives en deux temps : on ajoute d’abord le nouveau champ (migration non destructive), et on supprime les éléments devenus inutiles dans une migration ultérieure, une fois que le code de production n’y fait plus référence. Cette approche permet de déployer sans fenêtre de maintenance et sans risque de rupture entre le code et le schéma. Chaque fichier de migration est versionné dans le dépôt git au même titre que le code, et relu avant intégration.

3.5.7 Sauvegarde de la base de données

Une commande dédiée exécute `pg_dump` dans le conteneur PostgreSQL et exporte l’ensemble des données dans un fichier SQL horodaté.

3.5.8 Gestion des secrets

Aucun secret n’est présent dans le dépôt git. Les variables sensibles, telles que `SECRET_KEY`, les mots de passe PostgreSQL, les clés d’API MoveON, Pégase, Eudonet ou les tokens MinIO, sont injectées au runtime via un fichier `.env.prod` stocké uniquement sur le serveur de production. Le `.gitignore` exclut tous les fichiers `.env.*`, à l’exception d’un fichier de référence avec des valeurs fictives fourni pour le développement local.

3.6 Modèle de données

Le modèle de données traduit en structures relationnelles les entités métier identifiées dans l’analyse. Il est organisé en sept packages thématiques, chacun correspondant à un domaine métier distinct. Cette section en présente la logique d’ensemble ; les diagrammes de classes détaillés de chaque sprint sont présentés dans le chapitre de réalisation.

3.6.1 Organisation par packages

Le modèle s’articule autour de sept packages dont les responsabilités sont résumées dans le tableau suivant.

TABLE 3.8 – Packages du modèle de données

Package	Responsabilité
Reference	Référentiels stables partagés par l'ensemble du système : pays (avec région CTI), départements, niveaux d'études (dont le niveau terminal diplômante) et parcours.
Academic Year	Axe temporel central. Chaque campagne de mobilité est rattachée à une année universitaire dont le cycle de vie est piloté par une machine à états.
Students	Données académiques des étudiants importées depuis Pégase : Student , inscriptions annuelles (AnnualEnrollment), parcours pédagogique (Parcours), GPA mis à jour à chaque synchronisation.
Institutions	Universités partenaires (synchronisées depuis MoveON) et organismes d'accueil pour les stages.
Agreements & Quotas	Accords de mobilité (Agreement) et leur hiérarchie de quotas : déclinaison annuelle par accord (AgreementYear), liste des départements autorisés par accord (AgreementDepartment), et répartition annuelle du quota N7 par département (AgreementYearDepartment).
Mobilities	Quatre types de mobilités : sortantes académiques, dont les vœux ordonnés saisis par l'étudiant (StudentWish) et les affectations produites par l'algorithme ou ajustées manuellement (Assignment , AssignmentResult) ; stages internationaux (Eudonet) ; complémentaires déclarées par l'étudiant ; et entrantes (import Excel). La durée CTI consolidée est calculée automatiquement.
Import	Stockage brut (JSONB) des données reçues de chaque source avant transformation, garantissant la reprise sur erreur et la traçabilité des imports.

3.6.2 Principes structurants

Trois décisions de conception traversent l'ensemble du modèle et méritent d'être explicitées.

3.6.2.1 L'année universitaire comme axe temporel

Toutes les entités opérationnelles, telles que les dossiers, vœux, affectations, mobilités et imports, sont rattachées à une **AcademicYear**. Cette clé de partitionnement logique garantit deux propriétés : l'historique des campagnes passées est complet et consultable, et dès qu'une année est clôturée, ses données basculent automatiquement en lecture seule.

3.6.2.2 L'INE comme identifiant pivot

L'identifiant national étudiant (INE) est le seul identifiant commun à toutes les sources : MoveON, Pégase, Eudonet et le CAS. Tous les imports sont croisés via l'INE pour construire une vue unifiée du dossier étudiant. Quand un enregistrement arrive sans INE valide (cas courant dans Eudonet), une réconciliation par similarité est appliquée sur le nom et le prénom (distance d'édition), pondérée par la correspondance exacte de l'e-mail ; chaque

candidat est classé par un score de confiance (élevé, moyen, faible), et les cas ambigus sont remontés pour validation manuelle par le service RI.

3.6.2.3 Hiérarchie quotas INP / quotas N7

MoveON ne gère les quotas qu'à l'échelle de l'INP, au niveau global de l'accord, sans distinction d'année universitaire. Pour piloter une campagne à la fois et permettre à ce quota de varier d'une année sur l'autre, la hiérarchie de quotas s'appuie non pas sur **Agreement** mais sur sa version annuelle, **AgreementYear** : à sa validation, celle-ci enregistre le quota INP de l'année (**inp_total_places**), à partir duquel est calculé le quota N7 (**n7_places**). Ce quota N7 est ensuite réparti par département via **AgreementYearDepartment**, selon une estimation automatique dont la méthode de calcul est justifiée et détaillée au chapitre 4 ; le service RI peut ensuite modifier directement la valeur obtenue lorsque la répartition automatique ne correspond pas à la réalité du terrain. Cette organisation garantit la compatibilité avec MoveON, permet à un accord d'évoluer d'une campagne à l'autre, et fournit à l'algorithme d'affectation le niveau de détail dont il a besoin.

3.7 Flux de données : de la source au workflow

Chaque information importée dans le système suit un même parcours balisé, du fichier brut jusqu'à son intégration dans les entités métier. Ce parcours est délibérément découpé pour garantir qu'aucune donnée non validée ne peut déclencher un workflow, et que chaque étape est indépendamment traçable et rejouable.

1. **Récupération** : la tâche Django-Q2 interroge la source (API MoveON, Pégase, Eudonet ou fichier Excel) et construit un enregistrement **RawImport** en JSONB à partir des données normalisées (INE, chaînes, codes pays). Ce stockage intermédiaire permet de rejouer un traitement ultérieur sans rappeler la source en cas d'échec.
2. **Nettoyage et validation** : un pipeline Python normalise les chaînes, croise les INE et vérifie les règles métier. Les enregistrements qui ne passent pas la validation sont consignés dans le rapport d'import avec le motif de rejet, sans interrompre le reste du traitement.
3. **Persistance** : les enregistrements validés sont intégrés en base par upsert, rattachés à l'année universitaire en cours.
4. **Déclenchement des workflows** : une affectation créée par l'algorithme entre dans l'état initial de sa FSM, **proposed**. Toute progression ultérieure passe obligatoirement par une transition FSM explicite.
5. **Supervision** : l'administrateur consulte le rapport d'import et traite les anomalies signalées.

La figure 3.5 illustre ces cinq étapes successives.

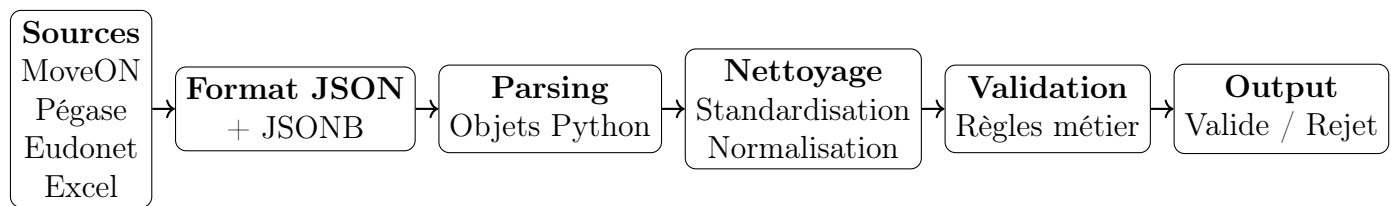


FIGURE 3.5 – Pipeline global de traitement des données

3.7.1 Détail de l'étape de nettoyage et validation

L'étape de nettoyage et validation est la plus délicate du pipeline : c'est là que les données brutes issues de sources hétérogènes deviennent exploitables par le système métier. Elle s'intercale entre la phase de récupération (stockage JSONB) et la persistance en base.

3.7.1.1 Description du processus de transformation

Les données importées arrivent sous des formats incompatibles : une API REST renvoie du JSON, les fichiers Excel du service RI utilisent des colonnes nommées différemment selon l'agent qui les a créés. Le pipeline de transformation commence donc par convertir toutes ces entrées dans une représentation intermédiaire commune, avant d'appliquer les mêmes règles de nettoyage quel que soit l'origine. Cette approche modulaire permet d'ajouter une nouvelle source sans modifier la logique de validation.

3.7.1.1.1 Standardisation des formats

- Normalisation des chaînes de caractères : suppression des espaces en début/fin, homogénéisation de la casse pour les codes (INE, codes ISO pays, codes département et niveau).
- **Gestion des accents :**
 - Les accents sont conservés pour les champs descriptifs (noms propres, universités, pays) afin de préserver l'intégrité sémantique des données.
 - Lors des opérations de recherche, de comparaison ou de détection de similarité (*matching*), les comparaisons sont effectuées de manière insensible aux accents afin d'éviter les divergences liées aux variations d'écriture.
- Uniformisation des formats de dates (ISO 8601).
- Conversion des types (ex : chaînes vers entiers pour les identifiants).

3.7.1.1.2 Normalisation sémantique

- Normalisation des pays selon un référentiel inspiré d’ISO 3166 : les variantes rencontrées dans les sources (codes ISO-2, ISO-3, noms en français ou en anglais) sont mappées vers un code ISO-2 de référence, par exemple “FR”, “France” ou “france” → “FR”.
- Une table de correspondance associe le nom d’offre d’un accord tel que renvoyé par MoveON à l’accord correspondant dans le système, ce qui absorbe les variations de libellé entre les deux référentiels.

3.7.1.1.3 Gestion des valeurs manquantes

- Identification des champs obligatoires (INE, université, type de mobilité).
- Si une valeur obligatoire est manquante, l’enregistrement est rejeté et consigné dans le rapport d’import avec la raison de l’erreur.
- Si les champs manquants ne sont pas critiques, l’enregistrement est conservé et peut être sauvegardé en base de données.

3.7.1.1.4 Détection et gestion des doublons

- Les doublons sont identifiés à partir d’une clé composite construite à partir de plusieurs attributs pertinents (par exemple : INE, entité concernée, période), permettant de représenter de manière unique un enregistrement métier.
- La composition de cette clé peut varier selon le type de donnée traité (étudiant, souhait, accord, etc.), afin de s’adapter aux spécificités de chaque entité.
- Chaque enregistrement est comparé aux données existantes via cette clé afin de détecter les occurrences identiques.
- En cas de correspondance, l’enregistrement existant est mis à jour (upsert) plutôt que dupliqué.

3.7.1.1.5 Validation métier

- Vérification de la cohérence avec les données Pégase (existence de l’étudiant via INE).
- Contrôle des règles métier, notamment le nombre maximum de vœux par étudiant.

3.7.1.1.6 Traçabilité et gestion des erreurs

- Stockage des données rejetées avec leur motif dans le rapport d’import (audit et reprise).
- Attribution d’un identifiant de traitement pour chaque batch.

3.7.1.2 Architecture du pipeline de nettoyage

Le pipeline de nettoyage est conçu comme un module autonome intégré dans le flux global. Il fonctionne par batch afin d'assurer l'isolation des traitements et la robustesse du système. La figure 3.6 le détaille sous forme de diagramme d'activité : les points de décision présentés ci-dessus (champ obligatoire manquant, correspondance avec un enregistrement existant, règles métier) y sont représentés explicitement, jusqu'à la persistance effective en base.

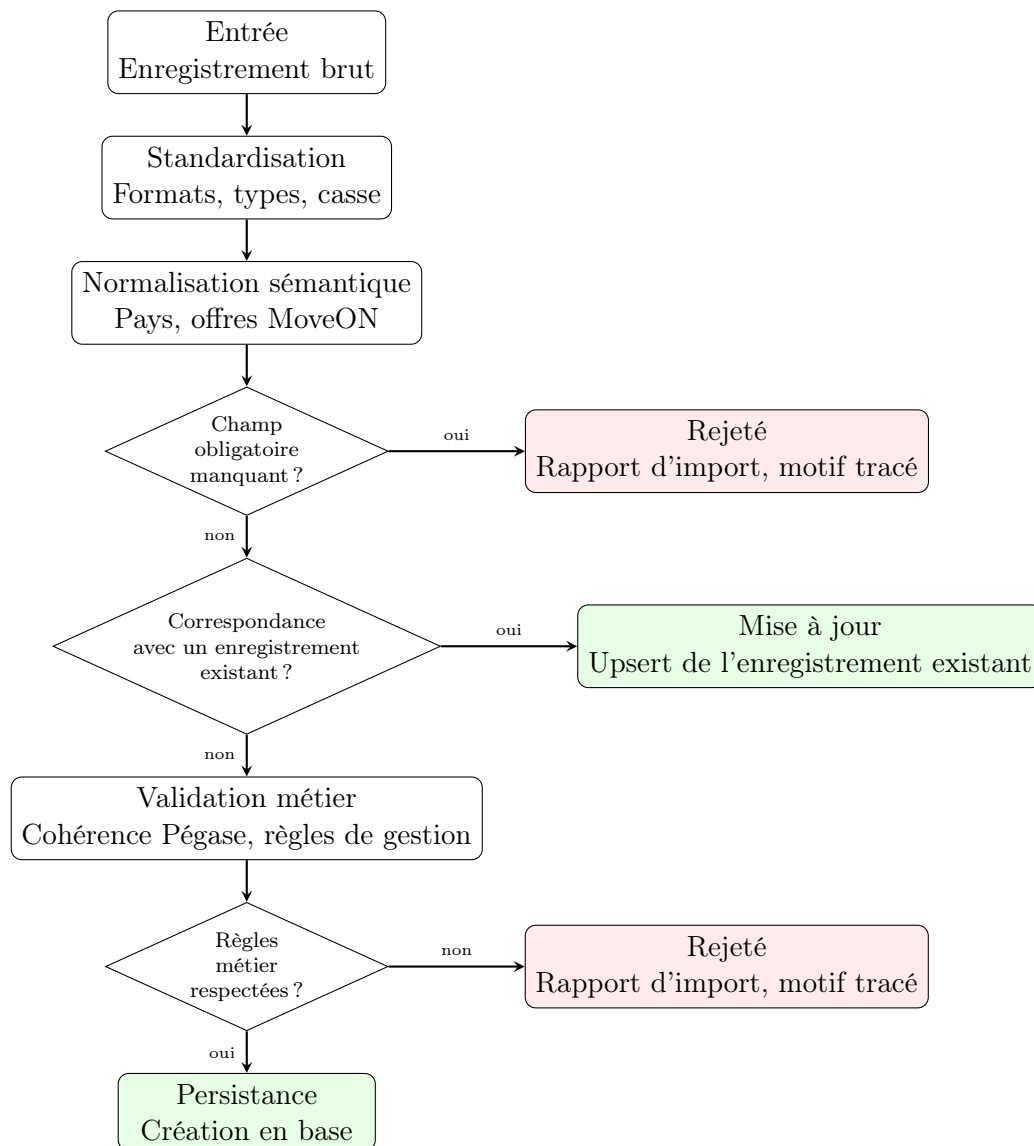


FIGURE 3.6 – Pipeline détaillé de l'étape de nettoyage des données, avec chemins de rejet

Cette architecture garantit :

- une **traçabilité complète** grâce au stockage JSONB,
- une **robustesse** via le traitement isolé de chaque enregistrement,
- une **reprise sur erreur** par batch,
- une **maintenabilité** grâce à la modularité du pipeline.

3.7.1.3 Technologies et outils utilisés

- **Python** : traitement et transformation des données.
- **openpyxl** : lecture et écriture des fichiers Excel (imports et exports CTI).
- **Regex** : validation et transformation des formats.
- **PostgreSQL (JSONB)** : stockage temporaire des données brutes.

Ce pipeline garantit qu’aucune donnée non validée n’atteint la couche métier. Chaque rejet est documenté dans le rapport d’import avec le motif exact, ce qui permet à l’administrateur de corriger la source ou de saisir manuellement les données manquantes.

3.8 Gestion des risques

Concevoir un système qui s’interface avec plusieurs sources institutionnelles implique d’accepter dès le départ que certaines choses ne se passeront pas comme prévu. Le tableau ci-dessous liste les risques les plus significatifs identifiés en phase de conception et les mesures retenues pour en limiter l’impact.

TABLE 3.9 – Matrice des risques

Risque	Prob.	Impact	Mitigation
INE absent dans une source externe (Eudonet, MoveON)	Haute	Élevé	Réconciliation par score de similarité pondéré : 50 % nom, 30 % prénom, 20 % correspondance exacte de l’e-mail (seuil 0,60). Cas ambigus (confiance faible ou moyenne) remontés pour validation manuelle par le service RI.
API externe indisponible pendant une campagne	Moyenne	Élevé	Import Excel de secours documenté pour chaque source. Cache local des données de la dernière synchronisation réussie.

Risque	Prob.	Impact	Mitigation
Dérive du périmètre en cours de PFE	Haute	Moyen	Priorisation stricte MVP / V2. Les fonctionnalités V2 (chatbot, recommandation) restent hors engagement initial et ne sont activées que dans une réserve conditionnelle, après validation du MVP et selon la capacité disponible.
Résistance au changement du service RI	Moyenne	Moyen	Implication du service RI dans les tests d'acceptation (UAT). Interface proche des pratiques Excel actuelles.
Fuite de données personnelles	Faible	Très élevé	Chiffrement des données sensibles (pgcrypto), cloisonnement strict par rôle (RBAC), audit log de toutes les actions.
Démarrage à froid du moteur de recommandation	Haute	Faible	Les premières campagnes utiliseront des heuristiques simples (GPA + département). Le modèle s'affine progressivement à mesure que l'historique s'accumule (2-3 ans de données suffisent).
Performance de l'algorithme d'affectation sous contraintes multiples	Faible	Moyen	Gale-Shapley est en $O(n^2)$ dans le cas général, mais $O(n)$ dans notre cas puisque chaque étudiant ne formule que trois vœux : parfaitement adapté à quelques centaines d'étudiants. Un test de charge vérifie les temps de réponse avant chaque mise en production.

Conclusion

Ce chapitre a défini la conception du système : backlog, architecture et modèle de données. Le chapitre suivant justifie les choix algorithmiques et les machines à états, avant de présenter la réalisation du système.

Choix algorithmiques et machines à états

Plan

Introduction	87
4.1 Choix de l'algorithme d'affectation	87
4.1.1 Besoins spécifiques	87
4.1.2 Alternatives évaluées	87
4.1.2.1 Algorithme glouton	87
4.1.2.2 Gale-Shapley	88
4.1.3 Synthèse comparative	88
4.1.4 Décision	89
4.2 Choix de la méthode de répartition des quotas par département	89
4.2.1 Besoins spécifiques	90
4.2.2 Alternatives évaluées	90
4.2.2.1 Méthode Hamilton (plus grands restes)	90
4.2.2.2 Méthode Jefferson (plus grandes moyennes)	91
4.2.2.3 Méthode Webster	91
4.2.3 Synthèse comparative	91
4.2.4 Décision	92
4.3 Choix du moteur de recommandation	92
4.3.1 Besoins spécifiques	92
4.3.2 Alternatives évaluées	92
4.3.2.1 Approche basée sur des règles (rule-based)	92
4.3.2.2 Approche par similarité (collaborative filtering)	93
4.3.2.3 Approche par apprentissage supervisé	93
4.3.2.3.1 Régression logistique	93
4.3.2.3.2 XGBoost	93
4.3.3 Synthèse comparative	94
4.3.4 Décision	94
4.4 Choix de l'orchestration RAG	95
4.4.1 Besoins spécifiques	95
4.4.2 Modèle de langage	96
4.4.3 Alternatives évaluées	96
4.4.3.1 LangChain	96
4.4.3.2 LlamaIndex	96

4.4.3.3	Haystack	97
4.4.3.4	Approche directe (sans framework)	97
4.4.4	Synthèse comparative	97
4.4.5	Décision	98
4.5	Synthèse des choix algorithmiques	98
4.6	Machines à états finis (FSM)	99
4.6.1	Principe et implémentation	99
4.6.2	FSM de l'année universitaire	99
4.6.3	FSM des entités métier	102
	Conclusion	102

Introduction

Le chapitre précédent a présenté la conception globale du système. Celui-ci justifie les choix algorithmiques qui structurent sa logique métier, puis détaille les machines à états qui sécurisent le cycle de vie de ses entités.

4.1 Choix de l’algorithme d’affectation

Cette section présente les contraintes auxquelles l’algorithme doit répondre, évalue les approches disponibles, puis justifie le choix retenu.

4.1.1 Besoins spécifiques

L’algorithme doit attribuer à chaque étudiant une destination parmi ses vœux, en respectant un ensemble de contraintes institutionnelles. Les résultats ont un impact réel sur les parcours : le service RI doit pouvoir justifier chaque affectation. L’équité entre candidats et la reproductibilité des résultats sont des exigences essentielles.

Le classement des étudiants repose principalement sur le GPA, complété par des règles institutionnelles. Les affectations doivent respecter les quotas par accord de mobilité et par département, les contraintes de financement, ainsi que la situation des étudiants étrangers dans le classement global. Chaque étudiant exprime jusqu’à trois vœux ordonnés ; l’algorithme traite les vœux dans l’ordre et recherche une affectation alternative en cas d’indisponibilité.

4.1.2 Alternatives évaluées

Deux approches algorithmiques ont été évaluées pour répondre à ces besoins.

4.1.2.1 Algorithme glouton

L’algorithme glouton parcourt les candidats dans l’ordre de leur GPA et affecte chacun à son meilleur vœu encore disponible, sans jamais revenir sur une décision déjà prise.

Avantages :

- Simple à implémenter et à expliquer
- Résultat prévisible, complexité $O(n^2)$

Limites :

- Le résultat dépend de l’ordre de traitement : deux étudiants avec des GPA très proches peuvent se retrouver dans des situations très différentes selon leur position dans la liste
- Une légère variation de l’ordre d’entrée peut produire des résultats différents, ce qui est incompatible avec l’exigence de reproductibilité

4.1.2.2 Gale-Shapley

Gale-Shapley fonctionne par rondes : à chaque tour, un étudiant non encore affecté propose sa destination préférée parmi celles qu’il n’a pas encore essayées, et une destination peut rejeter une proposition si elle en reçoit ensuite une meilleure selon sa priorité. Le processus s’arrête dès que plus aucune proposition n’est rejetée. Le résultat garantit qu’aucun étudiant et aucune destination ne préféreraient se retrouver ensemble plutôt que de garder l’affectation obtenue [27].

Dans le problème d’origine (le mariage stable), les deux côtés choisissent librement leurs préférences l’un envers l’autre. Ce n’est pas le cas ici : les destinations n’ont pas de préférence propre, elles suivent simplement un ordre de priorité fixé par l’ENSEEIH (nationalité, puis GPA). Ce cas se rapproche donc davantage des mécanismes de *school choice* utilisés pour affecter des élèves à des écoles [28] que du problème original du mariage stable. Les mêmes garanties, stabilité et résultat optimal du point de vue des étudiants, restent valables dans ce cas.

Avantages :

- Résultat stable : il est mathématiquement impossible qu’un étudiant et une destination forment une meilleure paire que celle retenue, quelles que soient les données
- Équitable : le résultat ne dépend pas de l’ordre de traitement, deux exécutions avec les mêmes données donnent toujours le même résultat
- Optimal pour les étudiants dans la variante Student-Proposing : chacun obtient la meilleure destination possible compte tenu de son classement et de ses vœux
- Complexité $O(n^2)$ dans le cas général, mais $O(n)$ dans notre cas puisque chaque étudiant ne formule que trois vœux ordonnés, ce qui borne le nombre total de propositions à $3n$; tout à fait adapté à quelques centaines d’étudiants

Limites :

- Légèrement plus complexe à implémenter que l’algorithme glouton

4.1.3 Synthèse comparative

Le tableau 4.1 récapitule les critères évalués pour chacune des deux approches, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 4.1 – Comparaison des algorithmes d'affectation

Critère	Poids	Algo. glouton	Gale-Shapley
Stabilité des affectations	Bloquant	✗	✓
Reproductibilité	Bloquant	✗	✓
Indépendance à l'ordre de traitement	Bloquant	✗	✓
Optimalité pour les étudiants	Secondaire	Résultat non garanti selon l'ordre de traitement	✓
Explicabilité des résultats	Bloquant	Difficile à justifier si l'ordre a influencé le résultat	✓
Simplicité d'implémentation	Secondaire	✓	Plus complexe que l'algo glouton, bien documenté

4.1.4 Décision

Gale-Shapley est retenu, dans sa variante **Student-Proposing** : ce sont les étudiants qui font des propositions aux destinations, et non l'inverse. Cette variante garantit que chaque étudiant obtient la meilleure destination possible compte tenu de son classement et de ses vœux, ce qui constitue la propriété dite d'*optimalité côté proposant*.

Les quotas fonctionnent à deux niveaux : une limite totale qui ne doit jamais être dépassée, et un nombre de places réservées par département, mais de façon plus souple : si un département n'utilise pas toutes ses places, les autres peuvent s'en servir. Quand un étudiant fait une proposition, il prend d'abord la place réservée à son département si elle est encore libre ; sinon, il peut quand même être accepté tant que la limite totale n'est pas atteinte. Une fois toutes les places prises, on compare les étudiants par leur score (nationalité puis GPA) : le moins bien classé peut être remplacé par le nouveau candidat.

L'algorithme glouton est écarté car il est sensible à l'ordre de traitement et ne garantit pas la reproductibilité. Gale-Shapley assure un résultat stable, équitable et reproductible dans ce cadre multi-contraint.

4.2 Choix de la méthode de répartition des quotas par département

Cette section présente les besoins liés à la répartition des quotas entre départements, évalue les méthodes disponibles, puis justifie le choix retenu.

4.2.1 Besoins spécifiques

Chaque accord bilatéral fixe un nombre total de places disponibles. Ce nombre doit ensuite être distribué entre les départements pédagogiques, ce qui soulève une question qui n’a pas de réponse évidente : sur quelle base décider combien de places revient à chaque département ?

Une répartition basée uniquement sur les demandes de l’année en cours serait insuffisante : elle ne tient pas compte du fait qu’un département peut avoir été systématiquement défavorisé les années précédentes, ni du fait que certains accords voient leur nombre de places évoluer d’une campagne à l’autre. Sans intégrer cet historique, la répartition risque d’être perçue comme arbitraire et de creuser des déséquilibres entre départements au fil des années.

Le besoin est donc de disposer d’une méthode qui distribue les places de façon proportionnelle à la demande réelle, tout en restant simple à comprendre, transparente et vérifiable par le service RI. La méthode retenue doit également garantir que la somme des places attribuées aux départements est exactement égale au nombre de places de l’accord, sans qu’aucune place ne soit perdue lors des arrondis.

4.2.2 Alternatives évaluées

Trois méthodes de répartition proportionnelle ont été comparées. Toutes partent du même principe : chaque département reçoit une quote-part proportionnelle à l’historique des vœux et des affectations le concernant sur les dernières années universitaires clôturées. La différence porte sur la manière de distribuer les places entières lorsque la division produit des résultats décimaux.

4.2.2.1 Méthode Hamilton (plus grands restes)

La méthode Hamilton attribue d’abord à chaque département la partie entière de sa quote-part. Les places restantes sont ensuite données, une par une, aux départements ayant les fractions les plus élevées, jusqu’à épuiser le total disponible.

Avantages :

- Simple à comprendre et à expliquer à tous les acteurs
- Résultat facile à vérifier manuellement
- Respecte les proportions au plus près du total disponible

Limites :

- Un département dont la quote-part est inférieure à 1 ne reçoit aucune place garantie, même si sa fraction est élevée

4.2.2.2 Méthode Jefferson (plus grandes moyennes)

La méthode Jefferson procède de façon itérative : à chaque étape, la prochaine place est attribuée au département dont le rapport entre sa taille et le nombre de places déjà reçues est le plus élevé.

Avantages :

- Aucun arrondi explicite nécessaire
- Résultats stables et cohérents d’une campagne à l’autre

Limites :

- Avantage les grands départements et pénalise les petits
- Calcul itératif moins intuitif à vérifier manuellement

4.2.2.3 Méthode Webster

La méthode Webster est proche de Jefferson, mais elle utilise un arrondi standard (seuil à 0,5) pour décider si un département reçoit une place supplémentaire ou non.

Avantages :

- Plus équilibrée que Jefferson pour les petits départements
- Bien adaptée aux cas où peu de places sont à distribuer

Limites :

- Calcul moins intuitif qu’Hamilton, plus difficile à vérifier sans outil
- Résultats moins prévisibles dans les cas limites

4.2.3 Synthèse comparative

Le tableau 4.2 récapitule les critères évalués pour chacune des trois méthodes, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 4.2 – Comparaison des méthodes de répartition des quotas

Critère	Poids	Hamilton	Jefferson	Webster
Simplicité de calcul	Secondaire	✓	✗	✗
Vérification manuelle possible	Bloquant	✓	Calcul itératif difficile à suivre	Moins prévisible dans les cas limites
Équité entre départements	Bloquant	✓	Favorise les grands départements	✓
Respect du total disponible	Bloquant	✓	✓	✓
Adapté à un petit nombre de places	Secondaire	✓	✗	✓
Transparence pour les acteurs	Bloquant	✓	✗	✗

4.2.4 Décision

La **méthode Hamilton** est retenue. Elle offre le meilleur équilibre entre équité, simplicité et transparence dans ce contexte. Les résultats peuvent être vérifiés à la main par le service RI, ce qui est important pour un processus qui touche directement les chances de départ des étudiants.

La méthode Jefferson est écartée car elle avantage systématiquement les grands départements, ce qui irait à l'encontre de l'équité visée. Webster est plus difficile à vérifier sans outil de calcul. Hamilton est donc la méthode la plus adaptée pour une répartition claire et justifiable.

4.3 Choix du moteur de recommandation

Cette section présente les besoins du module de recommandation, évalue les approches disponibles, puis justifie la méthode retenue.

4.3.1 Besoins spécifiques

Le moteur de recommandation a pour objectif d'aider les étudiants à choisir leurs destinations avant la saisie des vœux dans MoveON. Il ne prend pas de décision, mais propose un classement des destinations selon leur pertinence pour le profil de l'étudiant. Les recommandations doivent s'appuyer sur le profil académique et sur les données historiques disponibles, tout en restant interprétables pour l'étudiant.

4.3.2 Alternatives évaluées

Trois grandes familles d'approches ont été évaluées pour répondre à ce besoin.

4.3.2.1 Approche basée sur des règles (rule-based)

Cette approche repose sur des règles métier simples définies manuellement, telles que filtrer les destinations selon le département et classer par taux d'acceptation historique.

Avantages :

- Simple à mettre en place, sans données d'apprentissage
- Facile à comprendre et à expliquer

Limites :

- Peu flexible, ne s'adapte pas aux profils individuels
- Résultats souvent trop génériques

4.3.2.2 Approche par similarité (collaborative filtering)

Cette approche propose des destinations en se basant sur les choix d'étudiants ayant des profils similaires dans les promotions précédentes.

Avantages :

- Prend en compte les expériences passées des promotions
- Permet des recommandations plus personnalisées

Limites :

- Nécessite un volume important de données historiques
- Moins efficace pour les nouvelles destinations
- Mise en œuvre plus complexe

4.3.2.3 Approche par apprentissage supervisé

Cette approche consiste à entraîner un modèle pour prédire une probabilité de compatibilité entre un étudiant et une destination. Deux modèles ont été comparés au sein de cette famille.

4.3.2.3.1 Régression logistique

La régression logistique est un modèle statistique qui estime la probabilité d'un résultat à partir de variables d'entrée numériques [29]. Elle est particulièrement adaptée aux volumes de données modérés.

Avantages :

- Modèle simple et rapide à entraîner
- Résultats faciles à interpréter et à expliquer
- Adapté à des volumes de données modérés

Limites :

- Capture difficilement des relations complexes entre variables

4.3.2.3.2 XGBoost

XGBoost est un algorithme de gradient boosting qui construit un ensemble d'arbres de décision de façon séquentielle pour améliorer la précision prédictive [30].

Avantages :

- Capable de modéliser des relations complexes entre variables
- Très bonnes performances prédictives

Limites :

- Plus complexe à entraîner et à maintenir
- Résultats moins interprétables
- Risque de surapprentissage avec un volume de données limité

4.3.3 Synthèse comparative

Le tableau 4.3 récapitule les critères évalués pour chacune des approches, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 4.3 – Comparaison des approches de recommandation

Critère	Poids	Règles	Collab.	Régression log.	XGBoost
Personnalisation	Secondaire	✗	✓	✓	✓
Volume de données requis	Bloquant	✓	✗	Efficace à partir de 500 observations	Risque de surapprentissage avec peu de données
Interprétabilité	Secondaire	✓	Difficile sans connaître les profils similaires	✓	✗
Risque de surapprentissage	Bloquant	✓	Dépend de la qualité des données historiques	✓	✗
Simplicité de maintenance	Secondaire	✓	✗	✓	✗

4.3.4 Décision

L'apprentissage supervisé par régression logistique est retenu. Pour chaque couple étudiant–destination, le modèle estime une probabilité de compatibilité ; la cible (*target*) sur laquelle il est entraîné est binaire : l'étudiant a-t-il, historiquement, été affecté à cette destination (ou une destination du même accord) ou non. Le tableau 4.4 distingue les variables d'entrée (*features*) numériques du modèle de la donnée de contexte qui sert uniquement à les calculer.

TABLE 4.4 – Variables d’entrée du moteur de recommandation

Rôle	Variable	Description
Feature	GPA de l’étudiant	Note académique normalisée, principal signal de classement
Feature	Département pédagogique	Variable catégorielle à faible cardinalité
Feature	Taux historique d’affectation	Taux de sélection observé pour la destination évaluée sur les trois dernières années
Contexte (non une feature)	Destination	Sert uniquement de clé pour retrouver le taux historique ci-dessus ; n’est pas encodée en tant que variable catégorielle propre (pas de <i>one-hot encoding</i> par université)
Cible (<i>target</i>)	Affecté / non affecté	Variable binaire observée historiquement pour le couple étudiant–destination

Le modèle ne compte donc que trois variables d’entrée effectives ; la destination elle-même n’ajoute aucune dimension au vecteur de features, ce qui limite le nombre de paramètres à estimer et rend le volume de données disponible (environ 900 observations sur trois ans) réaliste pour ce modèle, sans terme d’interaction entre variables. Les destinations sont ensuite classées par ordre de probabilité décroissante afin de proposer une liste de recommandations personnalisées.

La régression logistique est retenue car elle offre un bon compromis entre simplicité, interprétabilité et facilité de maintenance. Le volume de données ne justifie pas XGBoost, dont les performances supplémentaires s’accompagnent d’un risque accru de surapprentissage, d’autant plus avec un espace de features aussi restreint.

Risque de démarrage à froid. Certaines destinations rares ou récentes disposent de peu d’observations historiques. Pour ces cas, le modèle se rabat sur le GPA et le département comme seuls signaux, ce qui reste cohérent avec les règles de sélection. Le module sera activé à partir de la deuxième année de fonctionnement, une fois qu’un historique minimal d’une campagne complète aura été constitué.

4.4 Choix de l’orchestration RAG

Cette section présente les besoins du module RAG et le modèle de langage imposé, évalue les frameworks d’orchestration disponibles, puis justifie le choix retenu.

4.4.1 Besoins spécifiques

Le composant RAG du projet est un chatbot destiné aux étudiants, capable de répondre à des questions sur les procédures de mobilité à partir de la documentation institutionnelle

de l'ENSEEIH : guides, critères d'éligibilité, délais et démarches administratives. Le cas d'usage est relativement simple : une base documentaire stable, des questions en français ou en anglais, et des réponses ancrées dans les documents officiels.

4.4.2 Modèle de langage

Le modèle de langage retenu est **Mistral** [31] (`mistral-small-latest`), conformément aux exigences de l'ENSEEIH. Il est accessible via l'**API Mistral ILAS**, une infrastructure nationale dédiée aux établissements de l'enseignement supérieur français, hébergée sur des serveurs localisés en France.

Ce mode d'accès institutionnel répond directement aux exigences RGPD : les données traitées restent dans l'infrastructure nationale et ne sont pas transmises à des tiers commerciaux. Il est néanmoins impératif que les données personnelles des étudiants ne soient jamais incluses dans les requêtes envoyées au modèle : les questions au chatbot sont transmises telles que saisies, sans enrichissement avec des données de dossier.

4.4.3 Alternatives évaluées

Quatre approches ont été évaluées pour orchestrer le pipeline RAG autour du modèle Mistral.

4.4.3.1 LangChain

LangChain est un framework d'orchestration IA généraliste, compatible avec un large éventail de modèles et de bases vectorielles [32].

Avantages :

- Grande communauté et nombreuses ressources disponibles
- Intégration simple avec Mistral et ChromaDB
- Adapté aux pipelines IA complexes et multi-étapes

Limites :

- API qui évolue fréquemment, induisant des risques de rupture
- Nombreuses dépendances, surcoût pour un chatbot documentaire simple

4.4.3.2 LlamaIndex

LlamaIndex est un framework spécialisé dans la recherche et la génération à partir de documents structurés [33].

Avantages :

- Conçu spécifiquement pour les applications RAG documentaires
- Bonne intégration avec Mistral et les bases vectorielles

- API plus stable, prise en main plus directe

Limites :

- Moins adapté aux architectures IA multi-agents très complexes

4.4.3.3 Haystack

Haystack est un framework orienté production pour les systèmes de recherche et de génération augmentée [34].

Avantages :

- Architecture modulaire adaptée aux déploiements en production
- Compatible avec Mistral

Limites :

- Documentation moins accessible pour des cas d’usage simples
- Intégration avec Mistral moins directe que LlamaIndex

4.4.3.4 Approche directe (sans framework)

L’approche directe consiste à implémenter manuellement le pipeline RAG via des appels à l’API Mistral, sans couche d’abstraction intermédiaire.

Avantages :

- Contrôle total sur chaque étape du pipeline
- Peu de dépendances externes

Limites :

- Nécessite de développer manuellement toute la logique RAG (chunking, indexation, retrieval, construction du prompt)
- Maintenance plus complexe à long terme

4.4.4 Synthèse comparative

Le tableau 4.5 récapitule les critères évalués pour chacune des quatre approches, afin de faciliter la comparaison et de justifier le choix retenu.

TABLE 4.5 – Comparaison des frameworks d’orchestration RAG

Critère	Poids	LangChain	LlamaIndex	Haystack	Direct
Spécialisé RAG	Secondaire	Généraliste, non optimisé pour le RAG	✓	Orienté pipelines complexes	✗
Intégration Mistral	Bloquant	✓	✓	Possible mais moins directe	✓
Stabilité de l’API	Bloquant	✗	✓	✓	✓
Adapté à un cas simple	Bloquant	Surcoût de dépendances pour un cas simple	✓	Trop complexe pour un chatbot documentaire	Logique RAG à développer entièrement
Maintenance à long terme	Bloquant	API instable, risque de rupture à chaque mise à jour	✓	✓	✗

4.4.5 Décision

LlamaIndex est retenu comme framework d’orchestration RAG, avec **ChromaDB** comme base vectorielle et **Mistral** comme modèle de langage. Cette combinaison correspond directement au besoin du projet : construire un chatbot documentaire simple, basé sur une base de connaissances stable. LlamaIndex fournit les fonctionnalités nécessaires sans surcoût de complexité, avec une API stable et une intégration directe avec Mistral et ChromaDB.

4.5 Synthèse des choix algorithmiques

TABLE 4.6 – Synthèse des choix algorithmiques

Composant	Méthode retenue	Justification principale
Algorithme d’affectation	Gale-Shapley	Stabilité, équité et reproductibilité des affectations
Répartition des quotas	Hamilton	Méthode proportionnelle simple, transparente et vérifiable manuellement
Moteur de recommandation	Régression logistique	Modèle simple, interprétable et adapté à un volume de données limité

Composant	Méthode retenue	Justification principale
Orchestration RAG	LlamaIndex + ChromaDB	Solution adaptée à un chatbot documentaire simple et stable
Modèle de langage	Mistral	Compatible avec le besoin multilingue du chatbot étudiant

4.6 Machines à états finis (FSM)

Cette section détaille les machines à états qui sécurisent le cycle de vie des principales entités métier du système.

4.6.1 Principe et implémentation

Dans un système comme celui-ci, certaines entités ne peuvent pas changer d'état librement. Une campagne clôturée ne doit pas pouvoir être rouverte, une affectation validée ne doit pas être modifiable sans trace, et l'algorithme ne doit pas s'exécuter sur des données incomplètes. Ces contraintes sont encodées dans des machines à états finis, implémentées via `django-fsm`. Le champ de statut est déclaré avec l'option `protected=True` : toute tentative de modification directe du statut, sans passer par une méthode de transition nommée, lève immédiatement une exception. Il n'est donc pas possible de contourner ces règles, même par une opération en base de données effectuée manuellement.

4.6.2 FSM de l'année universitaire

C'est la machine à états centrale du système. Elle gouverne l'accès à toutes les fonctionnalités de la campagne et compte neuf états. L'état `initialization` est celui dans lequel une `AcademicYear` entre à sa création : aucune transition n'y mène, c'est l'état par défaut du modèle. Cette création est soit manuelle, via une action de l'administrateur, soit automatique, une tâche planifiée créant l'année suivante dès que la précédente atteint l'état `closed`. La figure 4.1 présente cette machine à états.

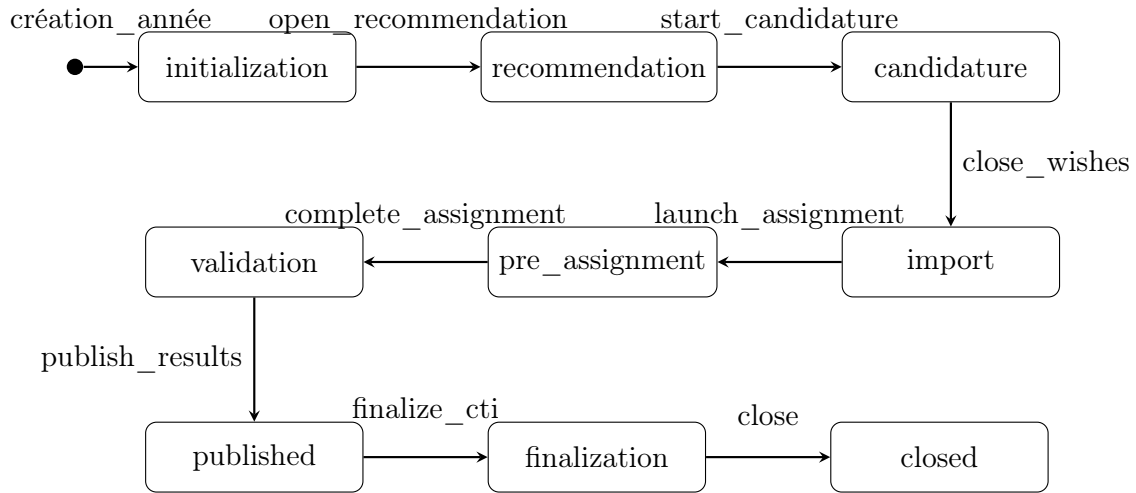


FIGURE 4.1 – Machine à états : AcademicYear

Chaque transition marque un cap dans le déroulement de la campagne.

- **création_année** : une **AcademicYear** est créée, soit manuellement par l’administrateur RI, soit automatiquement dès que l’année précédente atteint **closed** la veille.
- **initialization** → **recommendation** (**open_recommendation**) : les recommandations et le catalogue de destinations s’ouvrent aux étudiants ; les quotas par département sont validés (et redistribués selon la méthode Hamilton).
- **recommendation** → **candidature** (**start_candidature**) : la période de saisie des vœux ordonnés dans MoveON démarre. Cette transition est vérifiée par une tâche planifiée, qui la déclenche automatiquement dès que la date prévue est atteinte ; elle reste également accessible manuellement à l’administrateur RI.
- **candidature** → **import** (**close_wishes**) : la fenêtre de saisie se ferme ; l’administrateur importe et consolide les données finales (GPA Pégase, vœux MoveON) avant affectation. Comme la transition précédente, elle est déclenchée automatiquement à la date prévue par la même vérification quotidienne, ou manuellement si nécessaire.
- **import** → **pre_assignment** (**launch_assignment**) : l’algorithme Gale-Shapley est soumis à Django-Q2 pour exécution asynchrone.
- **pre_assignment** → **validation** (**complete_assignment**) : déclenchée automatiquement à la fin du calcul asynchrone (un point de reprise manuel existe pour les incidents) ; les propositions sont soumises à l’examen du service RI.
- **validation** → **published** (**publish_results**) : les résultats sont publiés aux étudiants ; l’**Assignment** associé est lui-même transitionné vers **published**.
- **published** → **finalization** (**finalize_cti**) : le calcul des durées de mobilité pour le rapport CTI (**MobilityDuration**) est soumis à Django-Q2 pour exécution asynchrone.
- **finalization** → **closed** (**close**) : la campagne est définitivement clôturée (**closed_at** horodaté) et l’ensemble de ses données bascule en lecture seule. Cette transition est elle

aussi vérifiée quotidiennement par la tâche planifiée et se déclenche automatiquement une fois la date de fin d’année atteinte, sans exclure un déclenchement manuel.

Le tableau 4.7 liste les préconditions effectivement vérifiées par le système avant chaque transition, ainsi que le comportement en cas d’échec.

TABLE 4.7 – Conditions préalables (guard conditions) des transitions FSM

Transition	Préconditions (guard conditions)	Comportement en cas d’échec
création_année	Dates cohérentes (début $\leq$ fin, ouverture/clôture des vœux comprise dans la période); label unique; automatique uniquement si une année en <code>closed</code> a terminé la veille	HTTP 400 avec message explicite (création manuelle); aucune création si aucune année ne vient de se clôturer (création automatique)
open_recommendation	Dates d’ouverture/clôture des vœux renseignées (condition FSM), au moins un GPA importé pour l’année, au moins un quota de département actif configuré	HTTP 400 avec message explicite, transition non exécutée
start_candidature	Aucune précondition applicative au-delà de l’état source; déclenchée automatiquement à la date prévue, ou manuellement	<code>TransitionNotAllowed</code> si l’année n’est pas en <code>recommendation</code>
close_wishes	Aucune précondition applicative au-delà de l’état source; déclenchée automatiquement à la date prévue, ou manuellement	<code>TransitionNotAllowed</code> si l’année n’est pas en <code>candidature</code>
launch_assignment	Aucune précondition bloquante; si aucun vœu n’est enregistré, une alerte système est créée mais l’algorithme est tout de même exécuté (tous les étudiants ressortent non affectés)	<code>TransitionNotAllowed</code> , tâche Django-Q2 non déclenchée
complete_assignment	Déclenchée automatiquement en fin de tâche Gale-Shapley; le point de reprise manuel est réservé aux années en <code>pre_assignment</code> en cas d’incident	Transition refusée hors de l’état <code>pre_assignment</code>
publish_results	Aucune précondition applicative au-delà de l’état source	<code>TransitionNotAllowed</code> si l’année n’est pas en <code>validation</code>
finalize_cti	Aucune précondition applicative au-delà de l’état source	<code>TransitionNotAllowed</code> si l’année n’est pas en <code>published</code> , tâche Django-Q2 non déclenchée

Transition	Préconditions (guard conditions)	Comportement en cas d'échec
close	Aucune précondition applicative au-delà de l'état source ; déclenchée automatiquement à la date de fin d'année, ou manuellement	<code>TransitionNotAllowed</code> si l'année n'est pas en <code>finalization</code>

4.6.3 FSM des entités métier

La FSM de l'année universitaire n'est pas la seule, mais elle reste de loin la plus riche : deux autres entités disposent de leur propre machine à états, protégée de la même façon par `django-fsm`. Le tableau 4.8 en donne la liste exhaustive.

TABLE 4.8 – FSM des entités métier

Entité	États et transitions
Assignment	<code>proposed</code> → <code>validated</code> → <code>published</code>
ComplementaryMobility	<code>pending</code> → <code>validated</code> <code>rejected</code>

Pour `Assignment`, c'est le service RI qui déclenche la transition `validate` (`proposed` → `validated`) depuis l'interface de supervision, une fois les propositions du calcul d'affectation examinées. Cette validation porte sur l'ensemble des propositions produites par un même calcul Gale-Shapley, pas sur un étudiant à la fois. Tant que ces propositions restent à l'état `proposed`, il reste possible de corriger une affectation individuelle, en changeant sa destination ou en la retirant, à condition d'indiquer un motif, conservé dans le journal d'audit. Vient ensuite la transition `publish` (`validated` → `published`) : elle publie les affectations aux étudiants.

Pour `ComplementaryMobility`, l'état `pending` correspond à la déclaration initiale : l'étudiant la dépose lui-même, avec un justificatif. Le service RI l'examine ensuite. S'il la valide (`validated`), le justificatif est conservé pendant une durée fixée par la politique RGPD. S'il la rejette (`rejected`), il doit indiquer un motif, conservé dans le journal d'audit, et le justificatif déposé est supprimé.

Conclusion

Ce chapitre a justifié les choix algorithmiques de la plateforme et détaillé les machines à états qui sécurisent son fonctionnement. Le chapitre suivant présente la réalisation du système, sprint par sprint.

Réalisation

Plan

Introduction	105
5.1 Sprint 1 : Socle technique et référentiels	105
5.1.1 Objectif du sprint	105
5.1.2 Sprint Backlog : Sprint 1	105
5.1.3 Conception détaillée	107
5.1.3.1 Diagramme de classes	108
5.1.4 Implémentation	110
5.1.5 Tests et validation du sprint	111
5.2 Sprint 2 : Données et traçabilité	112
5.2.1 Objectif du sprint	112
5.2.2 Sprint Backlog : Sprint 2	113
5.2.3 Conception détaillée	115
5.2.3.1 Diagramme de classes	115
5.2.3.2 Diagramme de séquence	116
5.2.4 Implémentation	117
5.2.5 Tests et validation du sprint	118
5.3 Sprint 3 : Algorithme d'affectation	119
5.3.1 Objectif du sprint	119
5.3.2 Sprint Backlog : Sprint 3	120
5.3.3 Conception détaillée	122
5.3.3.1 Diagramme de classes	122
5.3.3.2 Diagrammes de séquence	123
5.3.4 Implémentation	125
5.3.5 Tests et validation du sprint	128
5.4 Sprint 4 : Sources complémentaires et mobilités	128
5.4.1 Objectif du sprint	128
5.4.2 Sprint Backlog : Sprint 4	129
5.4.3 Conception détaillée	133
5.4.3.1 Diagramme de classes	133
5.4.3.2 Diagrammes de séquence	135
5.4.4 Implémentation	136
5.4.5 Tests et validation du sprint	139
5.5 Sprint 5 : Conformité CTI et authentification	140

5.5.1	Objectif du sprint	140
5.5.2	Sprint Backlog : Sprint 5	140
5.5.3	Conception détaillée	143
5.5.3.1	Diagrammes de séquence	144
5.5.4	Implémentation	144
5.5.5	Tests et validation du sprint	147
5.6	Sprint 6 : Fonctionnalités intelligentes (V2)	149
5.6.1	Objectif du sprint	149
5.6.2	Sprint Backlog : Sprint 6	149
5.6.3	Conception détaillée	151
5.6.3.1	Diagramme de séquence	152
5.6.4	Implémentation	152
5.6.5	Tests et validation du sprint	153
	Conclusion	154

Introduction

Le chapitre précédent a justifié les choix algorithmiques et les machines à états du système. Ce chapitre présente sa réalisation, sprint par sprint : backlog, conception, implémentation et tests de chaque itération.

5.1 Sprint 1 : Socle technique et référentiels

Ce sprint met en place le socle technique du système : référentiels, année universitaire, accords et quotas. Son objectif, son backlog, sa conception, son implémentation et ses tests sont présentés dans les sections qui suivent.

5.1.1 Objectif du sprint

Le Sprint 1 a pour objectif de mettre en place les principales bases sur lesquelles repose le reste du système. Il concerne les données de référence, telles que les pays, les départements et les universités partenaires, ainsi que la gestion de l'année universitaire, des accords de mobilité et des quotas associés.

Une attention particulière est portée au cycle de vie de l'année universitaire, qui permet d'organiser les différentes étapes de la campagne de mobilité et de contrôler leur enchaînement. À la fin du sprint, les principales données nécessaires aux fonctionnalités suivantes sont donc disponibles et peuvent être consultées et gérées depuis l'application.

Définition of Done. À l'issue de ce sprint, le système doit être exécutable via `docker compose up`, toutes les migrations Django doivent s'appliquer sans erreur, les tests unitaires doivent passer avec une couverture supérieure à 80 %, et le pipeline CI/CD doit être opérationnel sur GitHub Actions.

5.1.2 Sprint Backlog : Sprint 1

Le premier sprint concerne la mise en place des fondations nécessaires au fonctionnement du système. Il couvre la gestion des données de référence, le cycle de vie de l'année universitaire ainsi que la gestion des accords et des quotas. Ces éléments constituent la base sur laquelle s'appuient les fonctionnalités développées dans les sprints suivants.

Le tableau 5.1 présente les éléments du Product Backlog sélectionnés pour ce sprint ainsi que les principales tâches associées, leur priorité et leur estimation en story points.

TABLE 5.1 – Sprint Backlog du Sprint 1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-01 : Référentiels de base : <i>En tant qu'administrateur RI, je veux disposer d'un référentiel stable des pays, départements et universités partenaires afin que toutes les entités du système s'appuient sur des données de référence cohérentes.</i>					
01	Gestion des pays	Mettre en place les données permettant d'identifier et de gérer les différents pays utilisés dans le système.	Les pays peuvent être enregistrés et utilisés de manière cohérente dans les différentes fonctionnalités.	Haute	1
02	Gestion des départements	Mettre en place les données nécessaires à la gestion des départements de l'établissement.	Les départements peuvent être enregistrés et associés aux informations qui les concernent.	Haute	1
03	Gestion des universités partenaires	Mettre en place les informations permettant de référencer les universités partenaires et de les identifier de manière fiable.	Les universités partenaires peuvent être enregistrées, consultées et utilisées dans la gestion des accords et des mobilités.	Haute	1
04	Consultation des référentiels	Mettre à disposition une interface permettant au service des relations internationales de consulter et de rechercher les données de référence.	Les différentes données de référence sont accessibles et peuvent être facilement recherchées.	Haute	2
US-02 : Année universitaire et cycle de vie : <i>En tant qu'administrateur RI, je veux que le système gère le cycle de vie de l'année universitaire via une machine à états afin de contrôler chaque phase de la campagne de manière sécurisée.</i>					
05	Gestion de l'année universitaire	Mettre en place la gestion d'une année universitaire et des différentes phases qui composent son cycle de vie.	L'année universitaire peut être créée et son état évolue en respectant l'ordre prévu des différentes phases.	Haute	2
06	Contrôle des périodes de la campagne	Définir les principales périodes de la campagne et vérifier la cohérence des dates associées.	Les périodes enregistrées respectent un ordre chronologique cohérent et les dates incohérentes sont refusées.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
07	Interface de suivi de l'année universitaire	Créer une interface permettant à l'administrateur de consulter l'état de la campagne, ses principales dates et les actions disponibles.	L'état actuel de la campagne est clairement affiché et seules les actions correspondant à la phase en cours sont proposées.	Haute	2
US-03 : Accords et quotas, modèle de données : <i>En tant qu'administrateur RI, je veux disposer d'un modèle de données complet pour les accords et leurs quotas afin d'avoir une base fiable pour l'algorithme d'affectation.</i>					
08	Gestion des accords	Mettre en place les informations nécessaires pour gérer les accords conclus avec les universités partenaires.	Chaque accord est associé à une université partenaire et sa période de validité peut être prise en compte dans la gestion des mobilités.	Haute	2
09	Gestion annuelle des quotas	Mettre en place la gestion des places disponibles pour chaque accord et leur adaptation selon l'année universitaire.	Les quotas peuvent être définis pour une année universitaire et leur valeur effective est correctement prise en compte.	Haute	2
10	Répartition des quotas par département	Permettre de répartir les places disponibles entre les différents départements concernés.	Les quotas peuvent être associés aux départements et ajustés lorsque cela est nécessaire.	Haute	1
11	Préparation du stockage des données importées	Mettre en place un mécanisme permettant de conserver les données provenant des différentes sources avant leur traitement.	Les données importées sont associées à leur source et à l'année universitaire concernée, afin d'assurer leur suivi.	Haute	1
12	Consultation des accords et des quotas	Créer une interface permettant de consulter les accords, leur validité et les places disponibles pour chaque année universitaire.	Les accords et les quotas associés sont clairement présentés et les informations importantes peuvent être consultées par le service RI.	Haute	2

5.1.3 Conception détaillée

La conception UML de ce sprint est détaillée dans la section qui suit.

5.1.3.1 Diagramme de classes

Le diagramme de classes 5.1 présente les principaux éléments introduits dans ce sprint et les relations qui les unissent. Il permet de visualiser la place des référentiels, de l'année universitaire, des accords et des quotas dans l'organisation générale du système.

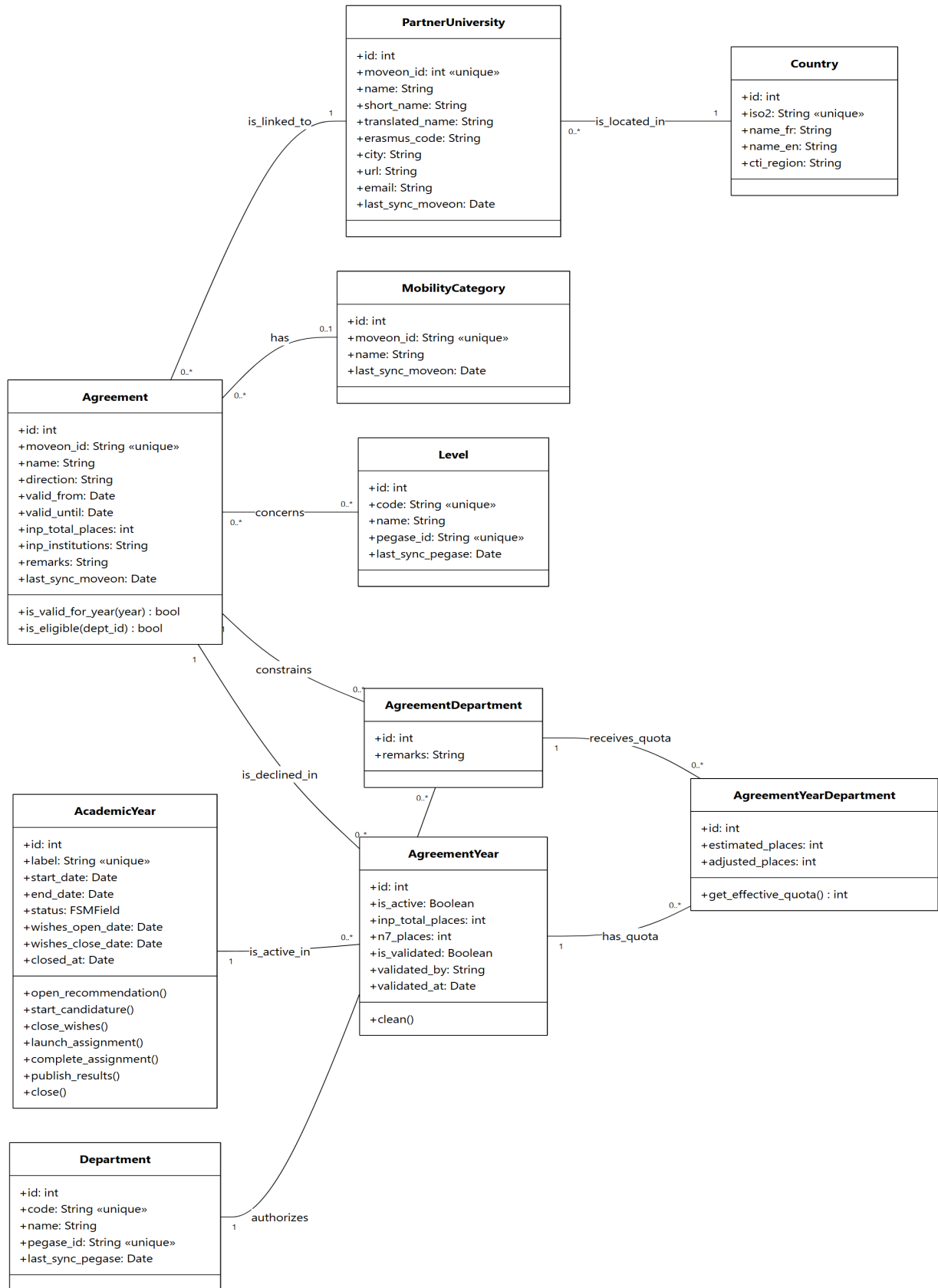


FIGURE 5.1 – Diagramme de classes : Sprint 1

5.1.4 Implémentation

Les fonctionnalités prévues dans ce sprint ont permis de disposer d'un premier socle fonctionnel. L'administrateur peut consulter les données de référence utilisées par le système et suivre les informations relatives aux universités partenaires, aux accords et aux quotas. La gestion de l'année universitaire permet également de connaître l'étape en cours de la campagne et de n'afficher que les actions compatibles avec son état.

Cette première réalisation constitue une base commune pour les sprints suivants. Les données et règles introduites ici sont ensuite réutilisées pour les imports, l'affectation des étudiants et les autres fonctionnalités de gestion de la mobilité.

La figure 5.2 illustre l'une des interfaces réalisées dans ce sprint, dédiée à la consultation des données de référence. Elle permet au service des relations internationales de retrouver et de rechercher les informations nécessaires à la gestion des mobilités.

N7 MOBILITE

- Tableau de bord
- Étudiants
- Mobilités sortantes
- Mobilités entrantes
- Stages internationaux
- Mobilités complémentaires
- Accords et quotas
- Statistiques CTI
- Référentiels
- Années universitaires
- Rapports d'import
- Journal d'audit

Admin RI
admin@n7.fr

Departements
3
Departements pédagogiques N7

Parcours
8
Parcours pédagogiques

Niveaux
8
Niveaux d'étude

Universités partenaires
54
Établissements partenaires

Pays
246
Référentiel international

Departements 3 Parcours 8 Niveaux 8 Universités 54 Pays 246

Rechercher dans les refs

Modifications des référentiels disponibles uniquement en **phase Initialisation**

Departements
Departements pédagogiques

+ Ajouter Sync Pégase

CODE	INTITULE	PEGASE ID	DERNIER SYNC	ACTIONS
3EA	Electronique, Energie Electrique et Automatique	-	—	
MF2E	Mécanique des Fluides, Énergétique & Environnement	-	—	
SN	Sciences du Numérique	-	—	

FIGURE 5.2 – Interface d'administration des référentiels

Cette page confirme, côté utilisateur, que les données synchronisées depuis Pégase et MoveON sont directement exploitables par le service RI, sans ressaisie ni retraitement manuel.

La figure 5.3 présente l'interface de suivi de l'année universitaire, qui affiche la phase en cours de la campagne et les actions disponibles à ce stade.

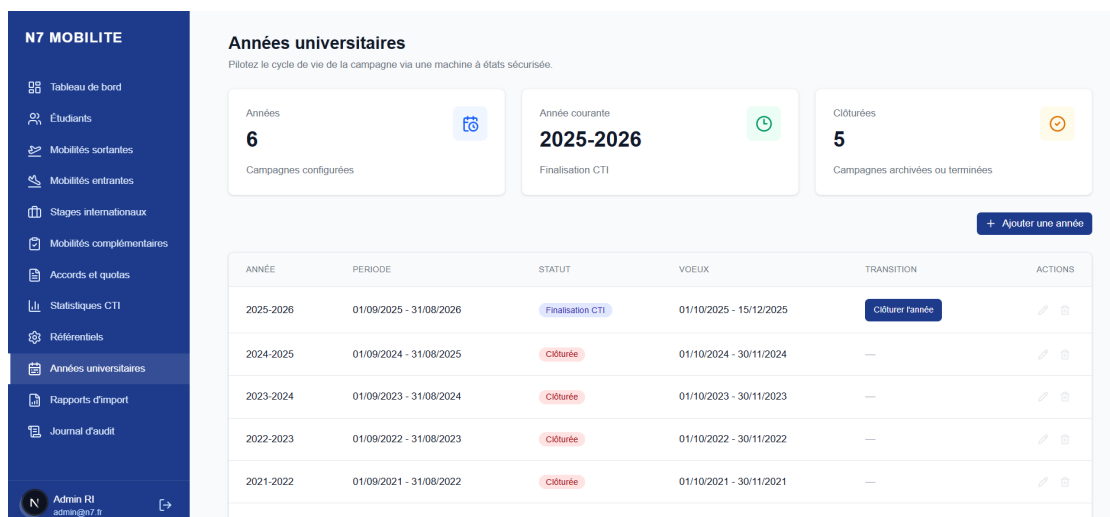


FIGURE 5.3 – Suivi du cycle de vie de l'année universitaire

Seules les actions compatibles avec la phase actuelle sont proposées à l'administrateur, ce qui évite de déclencher une transition hors séquence depuis l'interface.

La figure 5.4 présente, pour sa part, l'interface de consultation des accords et de leurs quotas.

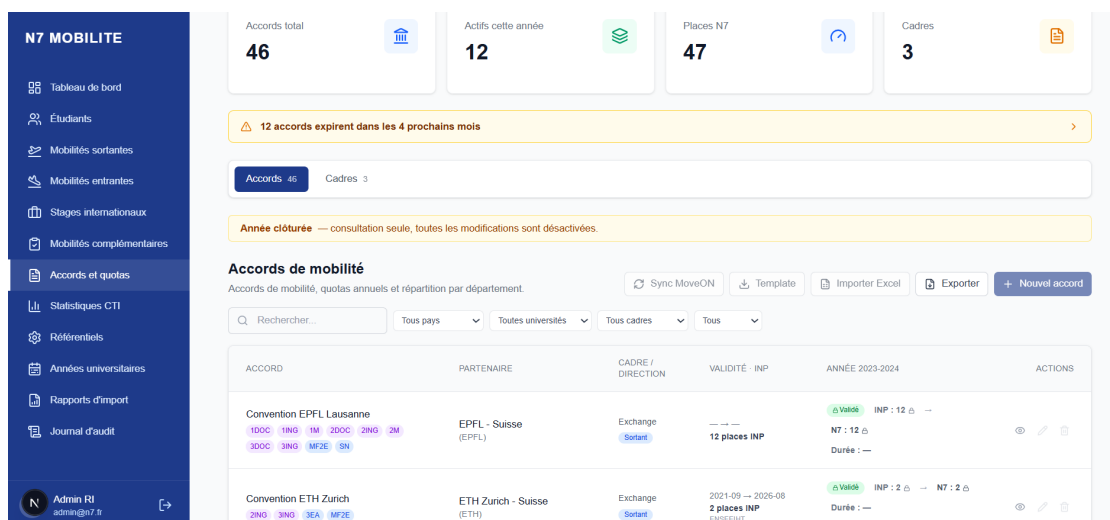


FIGURE 5.4 – Consultation des accords et des quotas

Le service RI y retrouve chaque accord, sa validité et les places disponibles par département.

5.1.5 Tests et validation du sprint

Ce sprint pose les fondations du système en introduisant le cycle de vie de l'année universitaire, les référentiels de base et la gestion des quotas. Le tableau 5.2 porte sur les principales règles métier associées à ces fondations, dont dépend l'ensemble des sprints suivants, validées au moyen de tests unitaires et de tests d'intégration.

TABLE 5.2 – Validation du Sprint 1

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Cycle de vie de l'année universitaire	Vérifier les transitions autorisées et le rejet des transitions hors séquence	Unitaire	pytest	Conforme	Validé
Cohérence des dates de campagne	Vérifier le rejet d'une incohérence entre date de début et date de fin	Unitaire	pytest	Conforme	Validé
Référentiels de base (pays, départements, universités partenaires)	Vérifier la création et l'unicité des données de référence	Intégration	pytest	Conforme	Validé
Calcul des quotas (accords et années)	Vérifier le calcul du quota effectif selon la configuration retenue (quota estimé ou ajusté)	Unitaire	pytest	Conforme	Validé
Validité et éligibilité d'un accord	Vérifier qu'un accord expiré est écarté pour l'année considérée et que l'éligibilité d'un département est correctement évaluée	Unitaire	pytest	Conforme	Validé

L'incrément du Sprint 1 est ainsi validé au regard des règles métier couvertes par les tests disponibles.

5.2 Sprint 2 : Données et traçabilité

Ce sprint porte sur la gestion des étudiants, de leurs inscriptions annuelles et de leurs vœux de mobilité. Objectif, backlog, conception, implémentation et tests en sont détaillés ci-dessous.

5.2.1 Objectif du sprint

Le Sprint 2 a pour objectif de permettre au système de gérer les données relatives aux étudiants et à leurs vœux de mobilité. Il introduit également les mécanismes nécessaires pour conserver une trace des principales actions réalisées sur ces données.

À l'issue du sprint, les informations provenant des sources utilisées par le service des relations internationales peuvent être intégrées au système, mises à jour sans créer de doublons et associées aux étudiants concernés. Les opérations importantes, notamment les imports, peuvent également être retracées afin de faciliter le suivi des données.

Définition of Done. À l'issue de ce sprint, les imports Pégase et MoveON s'exécutent sans erreur, les données sont persistées en base et rattachées à l'année universitaire courante, un rapport d'import est généré pour chaque synchronisation, et toute action d'un administrateur RI est tracée dans l'audit log avec l'auteur et l'horodatage.

5.2.2 Sprint Backlog : Sprint 2

Après la mise en place des fondations du système lors du premier sprint, le Sprint 2 est consacré à l'intégration et à la gestion des données nécessaires au traitement des mobilités. Il concerne notamment la récupération des informations relatives aux étudiants et à leurs vœux, ainsi que la mise en place des mécanismes permettant d'assurer la traçabilité des actions et le suivi des opérations d'import.

Le tableau 5.3 présente les éléments du Product Backlog retenus pour ce sprint. Pour chaque User Story, les principales tâches à réaliser, les critères d'acceptation, la priorité et l'estimation en story points sont indiqués.

TABLE 5.3 – Sprint Backlog du Sprint 2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-04 : Audit log et traçabilité : <i>En tant qu'administrateur RI, je veux que chaque action importante (import, transition FSM, modification, validation) soit enregistrée avec l'auteur et l'horodatage, afin de garantir la traçabilité complète des décisions.</i>					
01	Enregistrer les actions importantes	Mettre en place un mécanisme permettant de conserver une trace des principales actions réalisées dans le système, notamment les imports, les modifications et les validations.	Chaque action importante génère une entrée de suivi permettant d'identifier l'action réalisée, son auteur et sa date.	Haute	2
02	Consulter l'historique des actions	Permettre à l'administrateur de consulter les actions enregistrées et de retrouver facilement les informations liées à une opération donnée.	L'historique est accessible et les actions peuvent être consultées et filtrées selon différents critères.	Haute	1
US-05 : Données étudiants : <i>En tant qu'administrateur RI, je veux importer et synchroniser les données académiques des étudiants (INE, GPA, département, niveau) depuis Pégase, afin d'alimenter l'algorithme d'affectation avec des données étudiants fiables et à jour.</i>					
03	Récupérer les données académiques	Mettre en place la récupération des informations académiques des étudiants depuis Pégase afin de disposer des données nécessaires à la gestion des mobilités.	Les données des étudiants peuvent être récupérées et prises en compte par le système.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
04	Gérer les dossiers étudiants	Enregistrer les informations relatives aux étudiants et à leur inscription universitaire afin de conserver leur parcours académique d'une année à l'autre.	Chaque étudiant est identifié de manière unique et peut disposer de plusieurs inscriptions correspondant à différentes années universitaires.	Haute	2
05	Importer et mettre à jour les données étudiants	Traiter les données récupérées afin de créer les nouveaux dossiers et de mettre à jour les informations des étudiants déjà présents dans le système.	Les nouveaux étudiants sont ajoutés au système, les informations existantes sont mises à jour et les doublons sont évités.	Haute	3
06	Consulter les données étudiants	Mettre à disposition une interface permettant à l'administrateur de consulter les étudiants enregistrés et de rechercher un étudiant selon ses informations académiques.	La liste des étudiants est accessible et les informations peuvent être filtrées selon les critères prévus.	Haute	1
US-06 : Vœux étudiants : <i>En tant qu'administrateur RI, je veux importer les vœux ordonnés des étudiants depuis MoveON, et les consulter par étudiant, afin de les intégrer dans le système d'affectation.</i>					
07	Gérer les vœux de mobilité	Définir les informations nécessaires pour représenter les différents choix formulés par un étudiant dans le cadre de sa demande de mobilité.	Chaque vœu est associé à un étudiant, à l'accord de destination visé et à un rang permettant de respecter l'ordre des choix.	Haute	2
08	Importer les vœux depuis MoveON	Récupérer les vœux des étudiants et les intégrer au système en les associant aux dossiers étudiants correspondants.	Les vœux correspondant à un étudiant identifié sont enregistrés et les données ne pouvant pas être associées correctement sont signalées.	Haute	2
09	Consulter les vœux des étudiants	Mettre à disposition une interface permettant de consulter les choix de mobilité associés à chaque étudiant.	Les vœux d'un étudiant peuvent être consultés avec leur ordre de préférence et l'établissement ou l'accord concerné.	Haute	1
US-07 : Rapport d'import : <i>En tant qu'administrateur RI, je veux consulter un rapport détaillé après chaque import (succès, erreurs, doublons) avec la raison précise de chaque rejet, afin de corriger les anomalies rapidement.</i>					

#	Tâche	Description	Critères d'acceptation	Prior.	SP
10	Produire un bilan après chaque import	Générer automatiquement un résumé permettant d'identifier le nombre de données correctement traitées, les éventuelles erreurs et les anomalies détectées.	Chaque opération d'import produit un bilan indiquant clairement les données traitées avec succès et les éventuelles anomalies.	Haute	1
11	Consulter les rapports d'import	Mettre à disposition une interface permettant de consulter l'historique des opérations d'import et le détail des anomalies rencontrées.	Les rapports sont accessibles, classés par opération et permettent d'identifier la source ainsi que les erreurs détectées.	Haute	2

5.2.3 Conception détaillée

Cette partie détaille la conception UML retenue pour ce sprint.

5.2.3.1 Diagramme de classes

Le diagramme de classes 5.5 met en évidence les principaux éléments nécessaires à la gestion des étudiants, de leurs vœux et de la traçabilité des opérations. Il permet de comprendre les relations essentielles sans détailler le fonctionnement interne du code.

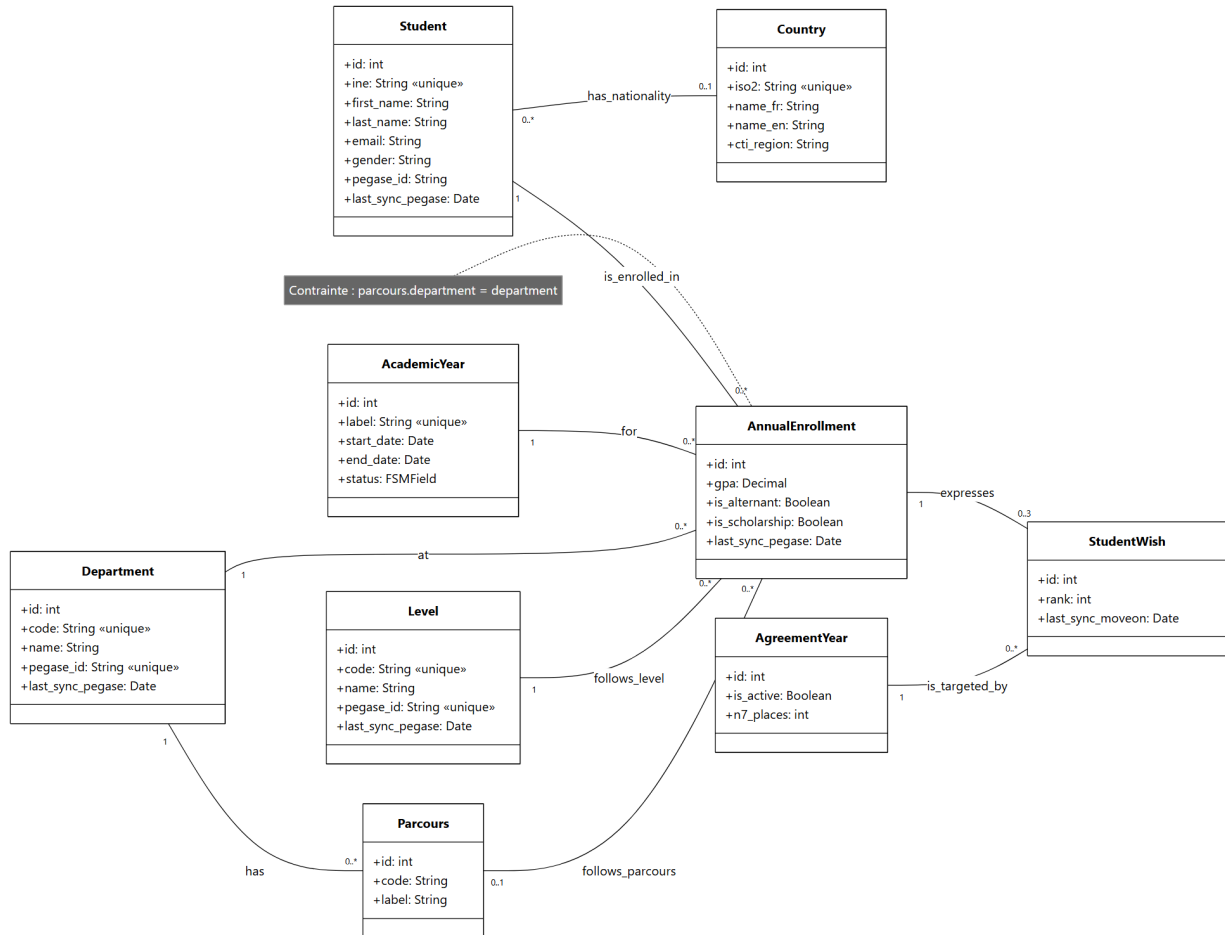


FIGURE 5.5 – Diagramme de classes : Sprint 2

5.2.3.2 Diagramme de séquence

Le scénario de la figure 5.6 détaille le déroulement type d'un import, de son déclenchement par l'administrateur jusqu'au rapport final.

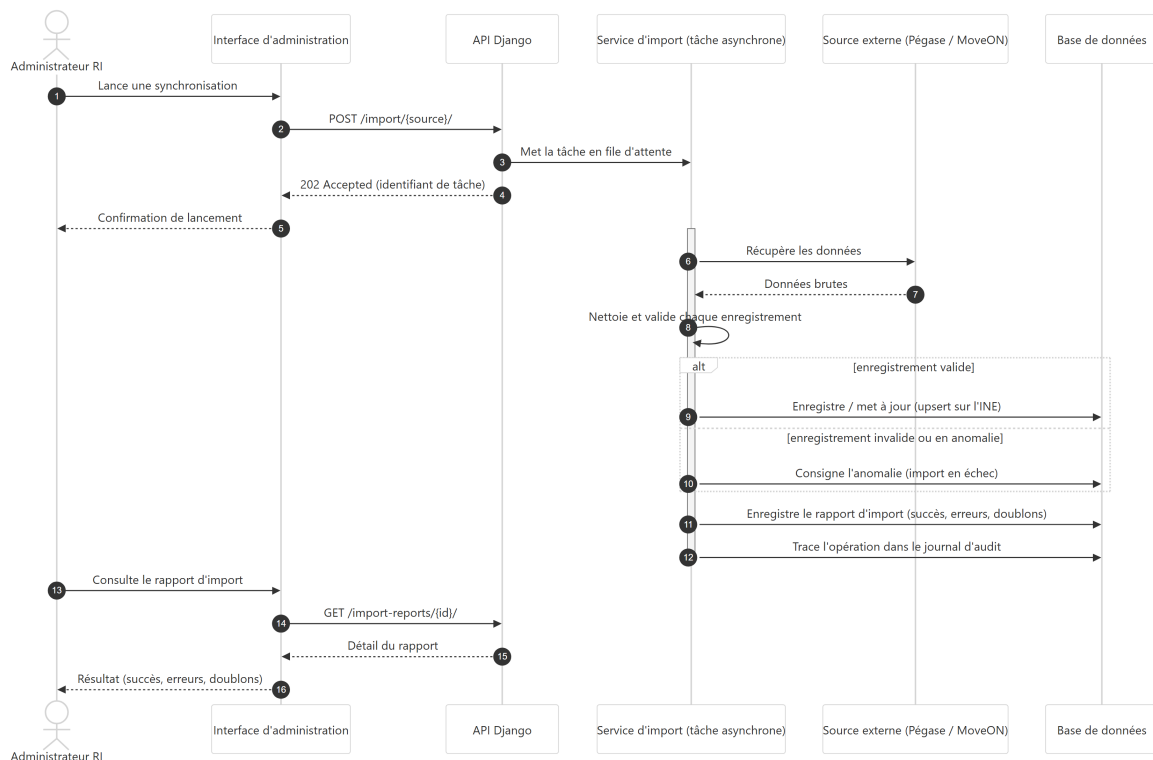


FIGURE 5.6 – Déroulement général de l'import et de la synchronisation des données

Le déclenchement reste manuel (c'est l'administrateur RI qui lance la synchronisation), mais le traitement s'exécute ensuite en tâche de fond : l'administrateur n'a pas à attendre la fin de l'import pour continuer à travailler. Chaque enregistrement est traité individuellement : une donnée valide est enregistrée ou mise à jour, une donnée invalide est consignée comme anomalie plutôt que de faire échouer tout l'import. Le rapport produit à la fin permet à l'administrateur de vérifier le résultat et de traiter les éventuelles anomalies.

5.2.4 Implémentation

Ce sprint a permis de rendre opérationnelle la gestion des données étudiantes et de leurs vœux. Lorsqu'une nouvelle importation est effectuée, les informations déjà présentes peuvent être mises à jour sans créer de doublons inutiles. Les données qui ne respectent pas les règles attendues sont identifiées afin de faciliter leur traitement.

Un mécanisme de traçabilité complète également cette partie du système. Il permet au service des relations internationales de retrouver les principales opérations réalisées sur les données et d'en identifier l'auteur et la date.

La figure 5.7 présente le journal utilisé pour consulter ces informations. Cette interface facilite notamment la recherche d'une action ou le suivi des opérations réalisées lors des imports.

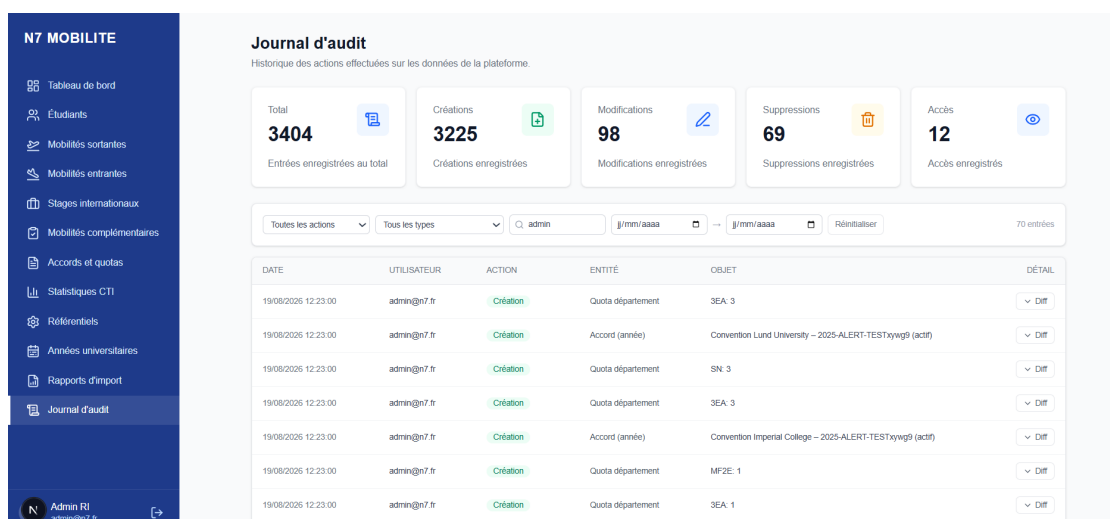


FIGURE 5.7 – Journal d’audit

Chaque ligne associe une action à son auteur et à son horodatage, ce qui permet au service RI de retracer l’origine d’une modification en cas de besoin.

La figure 5.8 présente l’interface de consultation des étudiants importés depuis Pégase.

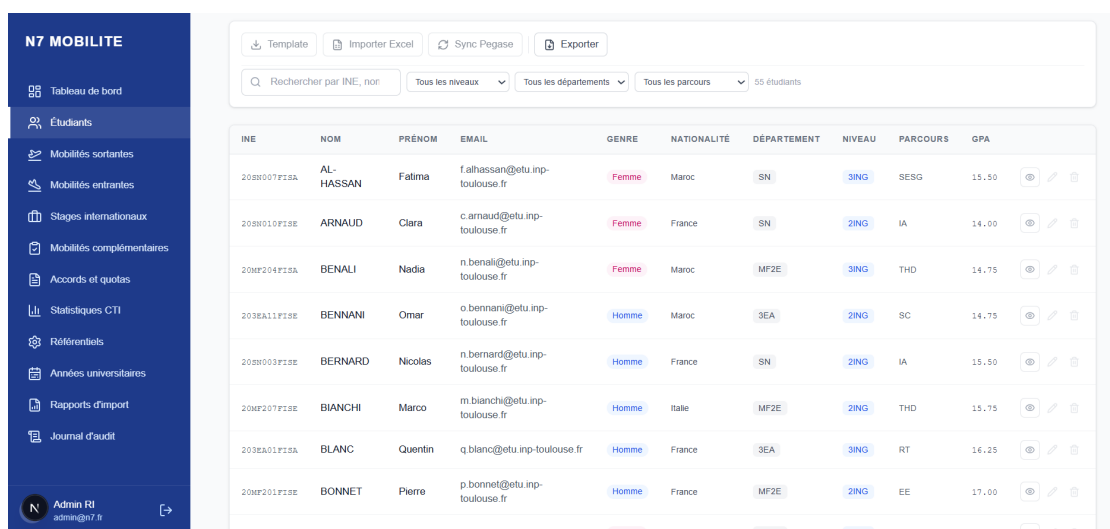


FIGURE 5.8 – Consultation des données étudiants

Le service RI y retrouve chaque étudiant avec son GPA, son département et son année d’inscription, avec un filtre pour retrouver rapidement un dossier particulier.

5.2.5 Tests et validation du sprint

Ce sprint rend le système opérationnel pour la collecte des données étudiants et de leurs vœux de mobilité, ainsi que pour leur traçabilité. Le tableau 5.4 porte sur la fiabilité du rattachement des données importées et sur le respect des règles de gestion associées, validées au moyen de tests d’intégration et de tests unitaires (composant) côté frontend.

TABLE 5.4 – Validation du Sprint 2

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Import et mise à jour des données étudiants	Vérifier l'identification unique par identifiant national, sans création de doublon lors des imports successifs	Intégration	pytest	Conforme	Validé
Vœux de mobilité (rattachement, unicité, rejet)	Vérifier le rattachement à un étudiant existant, l'unicité par rang et le rejet des vœux sans correspondance valide	Intégration	pytest	Conforme	Validé
Traçabilité des actions (audit)	Vérifier la création d'une entrée d'audit associant l'auteur et l'horodatage à chaque import	Intégration	pytest	Conforme	Validé
Journal d'audit : affichage	Vérifier le rendu et le filtrage des entrées d'audit côté interface	Composant	Vitest	Conforme	Validé
Rapport d'import	Vérifier que chaque import génère un rapport détaillant le nombre de succès, d'erreurs et de doublons	Unitaire	pytest	Conforme	Validé

L'incrément du Sprint 2 est ainsi validé au regard des règles de gestion des données étudiants et de leur traçabilité.

5.3 Sprint 3 : Algorithme d'affectation

Ce sprint couvre le cœur algorithmique du système : l'algorithme d'affectation et la publication de ses résultats, du backlog jusqu'aux tests réalisés.

5.3.1 Objectif du sprint

Le Sprint 3 correspond à la mise en place du processus central d'affectation des étudiants. Son objectif est de produire des propositions d'affectation à partir des vœux des étudiants et des places disponibles, puis de permettre au service des relations internationales de les examiner avant leur publication.

Le sprint couvre donc le calcul des affectations, leur suivi, les éventuelles corrections avant validation ainsi que la consultation des résultats par les étudiants. L'objectif est de disposer, à la fin du sprint, d'un processus complet allant du lancement du calcul jusqu'à la publication des résultats.

Définition of Done. À l'issue de ce sprint, l'algorithme Gale-Shapley doit s'exécuter sans erreur et produire des affectations stables respectant les quotas par accord et par département, les propositions doivent être consultables et modifiables par le service RI avant

publication, et chaque étudiant doit pouvoir consulter son affectation après publication des résultats.

5.3.2 Sprint Backlog : Sprint 3

Après la mise en place des données relatives aux étudiants et à leurs vœux au cours du sprint précédent, ce sprint concerne la fonctionnalité centrale du système : l'affectation des étudiants aux établissements partenaires. Il comprend le calcul des propositions d'affectation, leur consultation et leur supervision par le service des relations internationales, ainsi que la publication des résultats aux étudiants.

Le tableau 5.5 présente les éléments du Product Backlog retenus pour ce sprint, ainsi que les tâches prévues pour leur réalisation.

TABLE 5.5 – Sprint Backlog du Sprint 3

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-08 : Algorithme Gale-Shapley : <i>En tant qu'administrateur RI, je veux lancer l'algorithme d'affectation (Deferred Acceptance) qui affecte chaque étudiant à son meilleur vœu disponible en respectant les quotas par département, afin de produire une proposition équitable et stable.</i>					
01	Mettre en place la gestion des campagnes d'affectation	Créer les éléments nécessaires pour représenter une campagne d'affectation et conserver les résultats obtenus pour chaque étudiant. La campagne doit également permettre de suivre son avancement jusqu'à la publication des résultats.	Une campagne d'affectation peut être créée et les résultats individuels sont correctement associés aux étudiants concernés. Les différentes étapes de la campagne sont respectées.	Haute	2
02	Mettre en œuvre l'algorithme d'affectation	Développer le mécanisme permettant de proposer une affectation à chaque étudiant à partir de ses vœux, tout en tenant compte des places disponibles et des quotas définis pour les accords et les départements.	Les affectations respectent les quotas disponibles. Un étudiant est affecté au meilleur choix possible selon ses vœux. Les étudiants ne pouvant pas être affectés sont identifiés.	Haute	5
03	Lancer et suivre le calcul des affectations	Permettre au service RI de déclencher le calcul des affectations et de suivre son exécution avant de consulter les résultats produits.	Le lancement du calcul est possible depuis le système. Les résultats deviennent disponibles une fois le traitement terminé, sans nécessiter une nouvelle exécution.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
04	Permettre la consultation des résultats d'affectation	Mettre à disposition les informations nécessaires pour consulter les affectations réalisées, notamment l'étudiant concerné, la destination retenue et le résultat obtenu.	Les résultats peuvent être consultés et filtrés. Les étudiants non affectés sont clairement identifiés. Les informations affichées correspondent aux résultats du calcul.	Haute	2
05	Créer le tableau de bord des affectations	Mettre en place un tableau de bord synthétique affichant, pour la campagne en cours, le taux de remplissage des places par accord ainsi que le nombre d'étudiants non affectés.	Le taux de remplissage par accord s'affiche correctement. Les étudiants non affectés apparaissent avec leur motif. Le statut de la campagne (en cours, terminée, publiée) est visible.	Haute	2
US-09 : Supervision des affectations : <i>En tant qu'administrateur RI, je veux consulter les propositions d'affectation, les ajuster manuellement pour les cas particuliers et tracer chaque modification avec un motif, avant de valider et publier les résultats.</i>					
06	Mettre en place l'espace de supervision des affectations	Créer une liste détaillée des propositions individuelles, filtrable par département, statut et nom d'étudiant, permettant au service RI d'examiner chaque cas avant de le valider ou de le corriger.	Les propositions peuvent être filtrées par département, statut et étudiant. Chaque ligne affiche le vœu retenu et la destination proposée, ce qui permet de vérifier chaque cas individuellement.	Haute	3
07	Permettre la modification d'une affectation	Autoriser le service RI à modifier une proposition d'affectation lorsqu'un cas particulier le nécessite, tout en demandant la justification de cette modification et en assurant sa traçabilité.	Une modification doit être accompagnée d'un motif. Les changements effectués sont enregistrés. Les résultats publiés ne peuvent plus être modifiés.	Haute	2
08	Valider les propositions d'affectation	Permettre au service RI de confirmer les résultats après leur vérification afin de finaliser la campagne d'affectation avant leur communication aux étudiants.	La validation n'est possible qu'une fois les résultats disponibles. L'état de la campagne est mis à jour après validation.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
09	Publier les résultats d'affectation	Mettre en place la publication des résultats validés afin de les rendre accessibles aux étudiants concernés.	Seuls les résultats validés peuvent être publiés. Une fois publiés, les résultats deviennent accessibles aux étudiants et ne peuvent plus être modifiés.	Haute	1
US-10 : Consultation des résultats : <i>En tant qu'étudiant, je veux consulter mon affectation une fois les résultats publiés afin de connaître ma destination et les démarches à suivre.</i>					
10	Permettre à l'étudiant de consulter son résultat	Mettre à disposition de chaque étudiant son résultat d'affectation une fois la campagne publiée, en veillant à ce qu'il puisse uniquement accéder à ses propres informations.	Un étudiant peut consulter uniquement son propre résultat. Les résultats ne sont pas accessibles avant leur publication.	Haute	1
11	Créer l'interface de consultation des résultats	Mettre en place une page permettant à l'étudiant de consulter sa destination et les principales informations relatives à son affectation. Lorsqu'aucune affectation n'a été obtenue, une information adaptée est affichée.	La page est accessible après la publication des résultats. Les informations affichées correspondent à l'affectation de l'étudiant. Une information est prévue pour les étudiants non affectés.	Haute	2

5.3.3 Conception détaillée

La conception UML de ce sprint est présentée ci-dessous.

5.3.3.1 Diagramme de classes

Le diagramme de classes 5.9 présente les principaux éléments utilisés pour représenter les affectations et leur suivi. Il permet de situer les données produites par le processus d'affectation et les informations nécessaires à leur supervision.

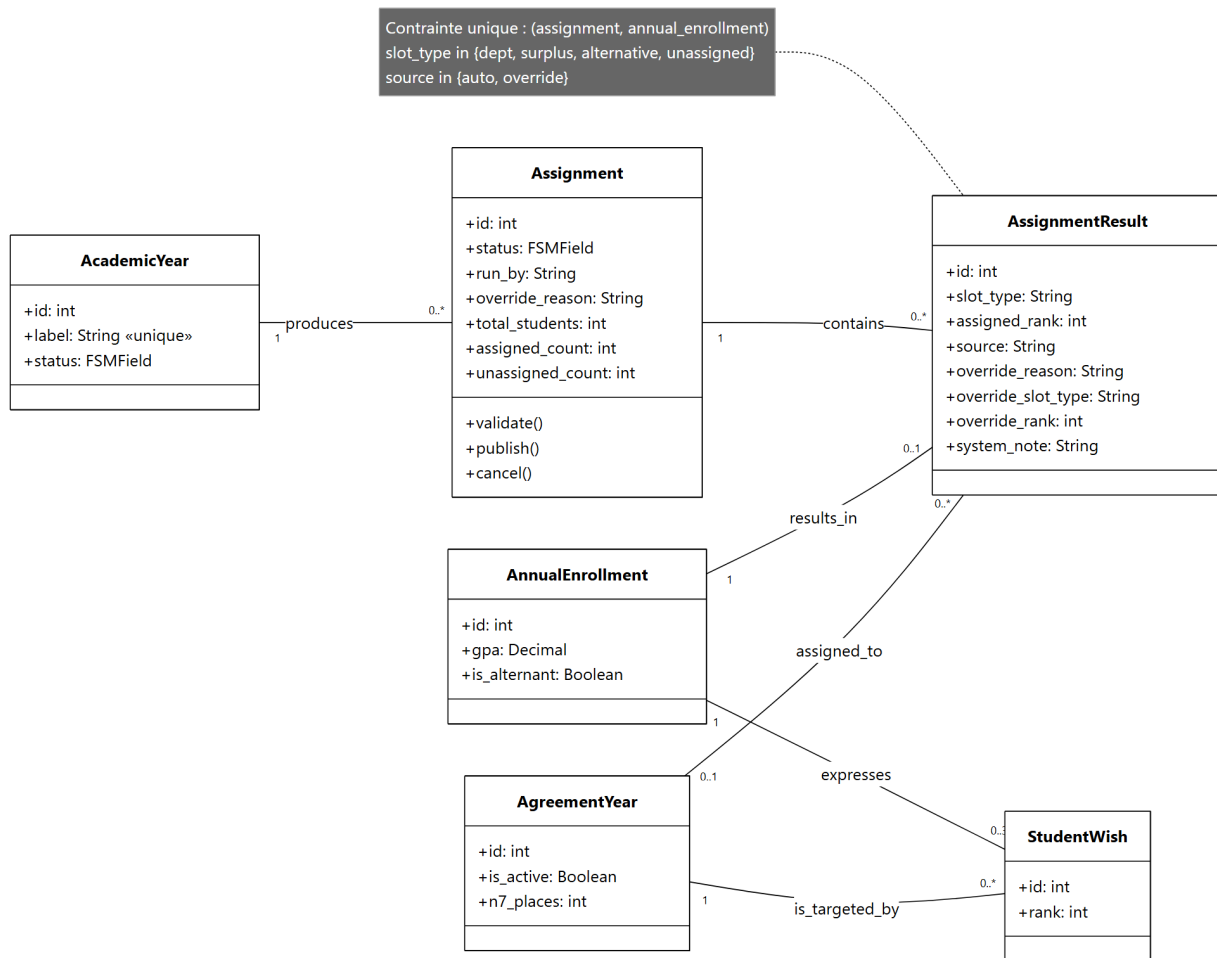


FIGURE 5.9 – Diagramme de classes : Sprint 3

5.3.3.2 Diagrammes de séquence

Deux scénarios illustrent le fonctionnement de ce sprint : le lancement du calcul d’affectation, présenté en figure 5.10, puis sa supervision jusqu’à la publication des résultats.

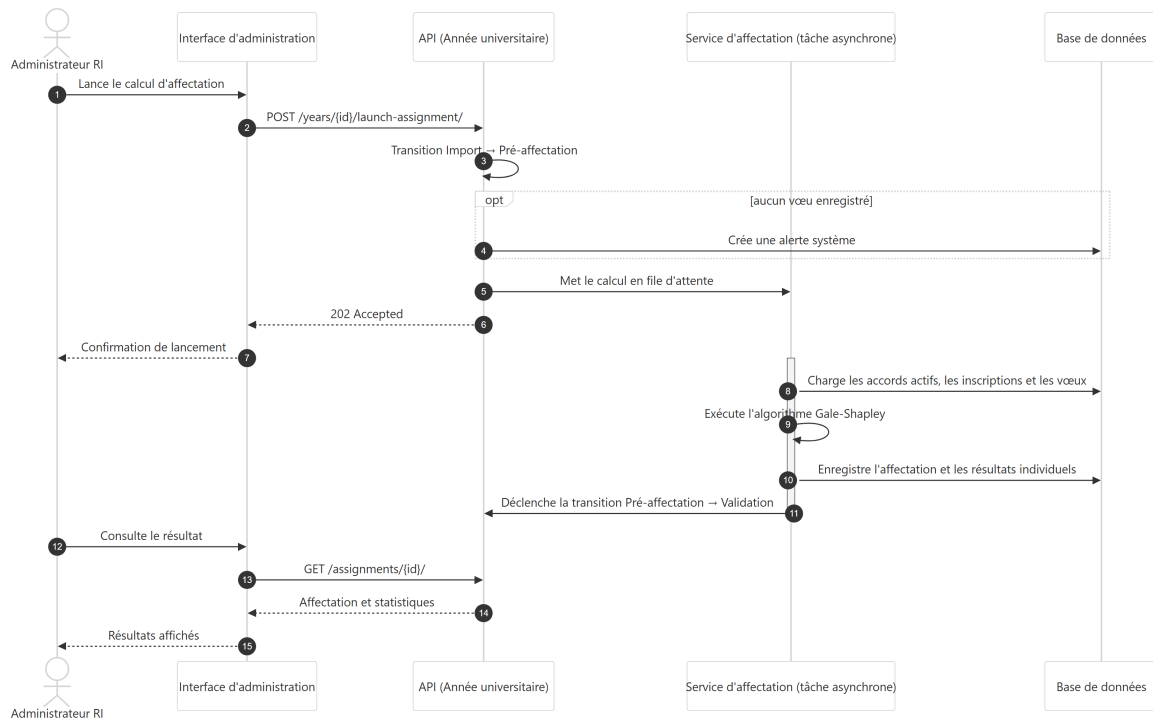


FIGURE 5.10 – Lancement et génération des affectations

Le calcul s'exécute en tâche de fond : sur une promotion complète, il peut prendre plusieurs minutes, et l'administrateur RI n'a pas à rester bloqué sur cet écran en attendant. Si aucun vœu n'a été enregistré pour l'année, une alerte prévient l'administrateur avant que le calcul ne se lance quand même, plutôt que de le bloquer silencieusement. Une fois le calcul terminé, l'année universitaire avance automatiquement à la phase suivante de son cycle de vie.

La supervision du résultat par le service RI, jusqu'à sa publication aux étudiants, suit le scénario de la figure 5.11.

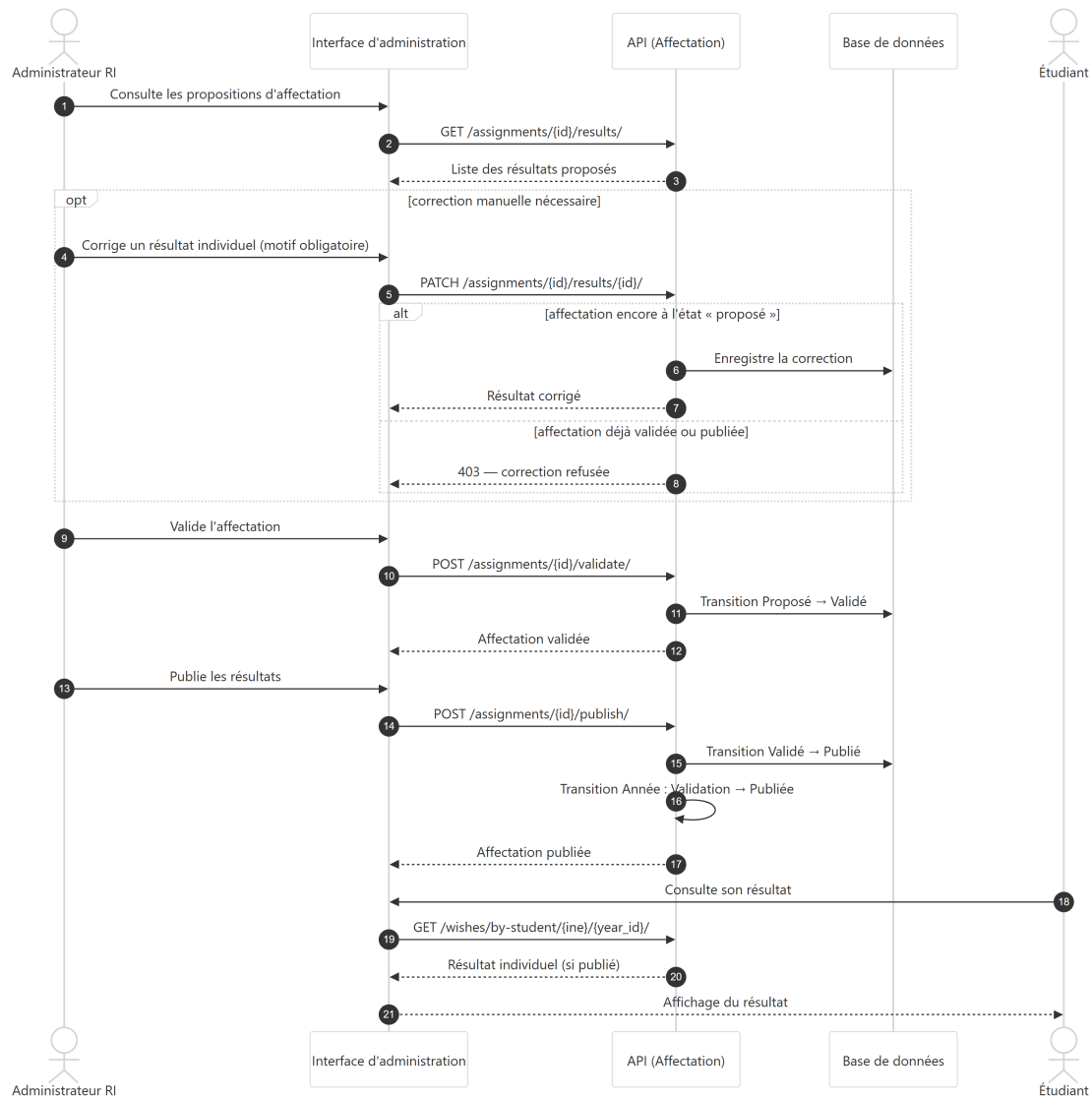


FIGURE 5.11 – Validation, correction et publication des affectations

Une correction manuelle individuelle reste possible tant que l’affectation n’a pas encore été validée ; elle exige toujours un motif, conservé dans le journal d’audit. Une fois l’affectation validée puis publiée, plus aucune modification n’est acceptée, et l’étudiant peut consulter son propre résultat, jamais celui d’un autre.

5.3.4 Implémentation

La réalisation de ce sprint permet au service des relations internationales de lancer le calcul des affectations puis d’en suivre le résultat depuis l’application. Une fois les propositions obtenues, elles peuvent être examinées avant leur validation et leur publication.

L’interface de supervision regroupe les informations nécessaires pour suivre la répartition obtenue et intervenir lorsqu’une correction est nécessaire. Certaines actions deviennent volontairement indisponibles après validation afin d’éviter la modification d’un résultat déjà confirmé.

La figure 5.12 présente le tableau de bord consulté par le service RI après le lancement d'une campagne d'affectation.

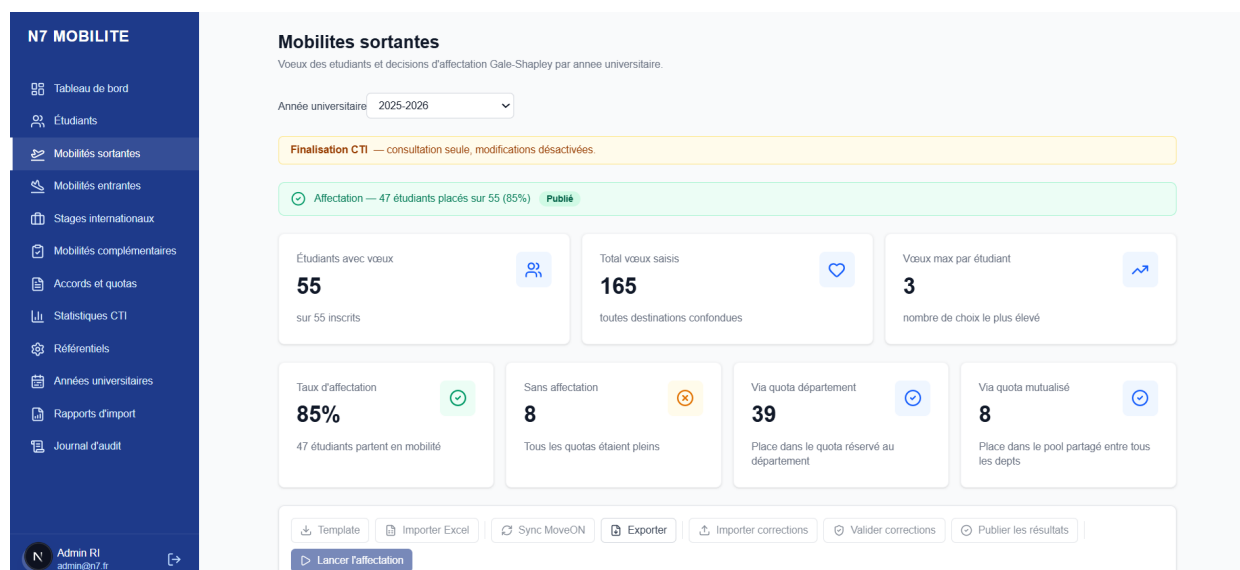


FIGURE 5.12 – Tableau de bord des affectations

Le taux d'affectation global et sa répartition par type de créneau (quota département, quota mutualisé) sont visibles en un coup d'œil, avant même d'entrer dans le détail des dossiers individuels ; c'est depuis cet écran que le service RI déclenche le calcul, importe des corrections ou publie les résultats.

La figure 5.13 présente ensuite l'espace de supervision, qui liste chaque proposition individuelle avec les vœux de l'étudiant et la décision retenue.

ÉTUDIANT	FILIÈRE	TYPE	BOURSE	GPA	VŒU 1	VŒU 2	VŒU 3	DÉCISION D'AFFECTATION
AL-HASSAN Fatima Maroc	SN SESG 3ING	FISA	—	15.50	Technische Universität Berlin (Technische Universität Berlin) — Allemagne Erasmus+ TU Berlin	McGill University (McGill) — Canada Convention McGill	Technische Universität München (TUM) — Allemagne Erasmus+ TU München	Non affecté
ARNAUD Clara France	SN IA 2ING	FISE	—	14.00	Lund University (Lund University) — Suède Convention Lund University	EPFL (EPFL) — Suisse Convention EPFL Lausanne Vœu retenu	McGill University (McGill) — Canada Convention McGill	EPFL (EPFL) — Suisse Convention EPFL Lausanne Quota mutualisé
BENALI Nadia Maroc	MF2E THD 3ING	FISA	Boursier	14.75	Delft University of Technology (TU Delft) — Pays-Bas Erasmus+ TU Delft	ETH Zurich (ETH) — Suisse Convention ETH Zurich	Politecnico di Torino (Politecnico di Torino) — Italie Erasmus+ Politecnico Torino Vœu retenu	Politecnico di Torino (Politecnico di Torino) — Italie Erasmus+ Politecnico Torino Quota département
BENNANI Omar Maroc	3EA SC 2ING	FISE	—	14.75	Imperial College London (Imperial College London) — Royaume-Uni Convention Imperial College	Politecnico di Torino (Politecnico di Torino) — Italie Erasmus+ Politecnico Torino	McGill University (McGill) — Canada Convention McGill	Non affecté
BERNARD Nicolas France	SN IA 2ING	FISE	—	15.50	Universidad Politécnica de Madrid (UPM) — Espagne Erasmus+ UPM Madrid Vœu retenu	Technische Universität München (TUM) — Allemagne Erasmus+ TU München	KTH Royal Institute of Technology (KTH) — Suède Erasmus+ KTH Stockholm	Universidad Politécnica de Madrid (UPM) — Espagne Erasmus+ UPM Madrid Quota département
BIANCHI Marco Italie	MF2E THD 2ING	FISE	—	15.75	ETH Zurich (ETH) — Suisse Convention ETH Zurich	Universidad Politécnica de Madrid (UPM) — Espagne Erasmus+ UPM Madrid Vœu retenu	Lund University (Lund University) — Suède Convention Lund University	Universidad Politécnica de Madrid (UPM) — Espagne Erasmus+ UPM Madrid Quota département

FIGURE 5.13 – Supervision individuelle des propositions d'affectation

Cette vue permet au service RI de contrôler, étudiant par étudiant, le vœu retenu et le type de quota utilisé, avant de valider puis de publier les résultats.

La figure 5.14 détaille enfin les statistiques d’affectation par accord, par département et par pays de destination.

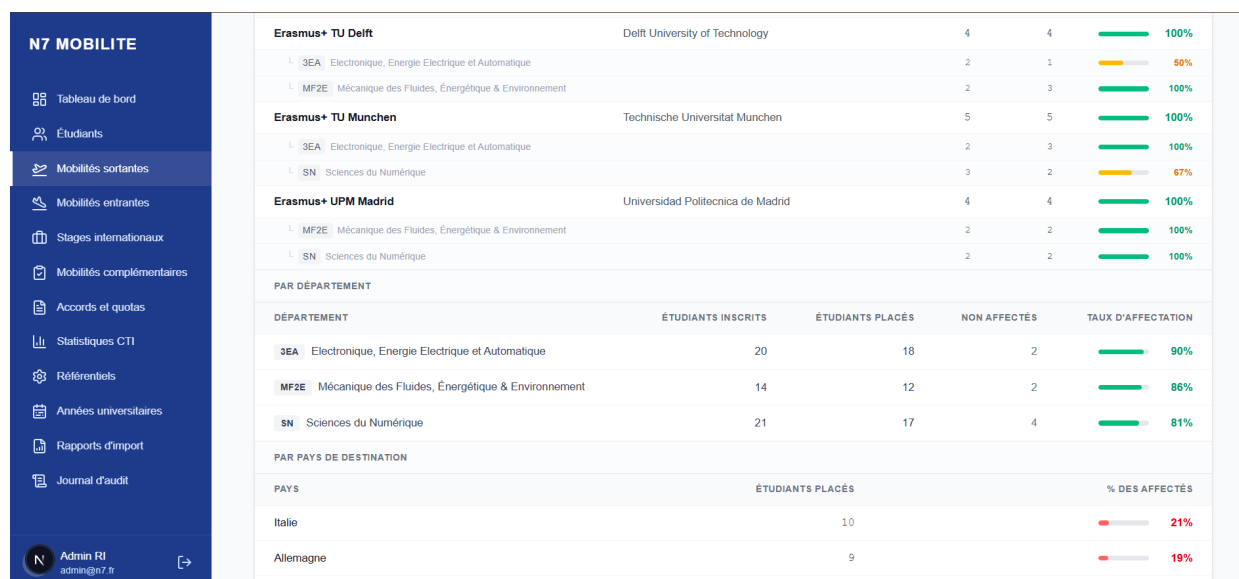


FIGURE 5.14 – Statistiques détaillées de l’affectation

Cette répartition fine, jusqu’au niveau de chaque accord et de chaque département, permet au service RI de repérer les destinations sous-remplies ou sursollicitées d’une campagne à l’autre.

Côté étudiant, la figure 5.15 présente la page « Ma mobilité », qui rassemble les vœux formulés et, une fois les résultats publiés, l’affectation obtenue.

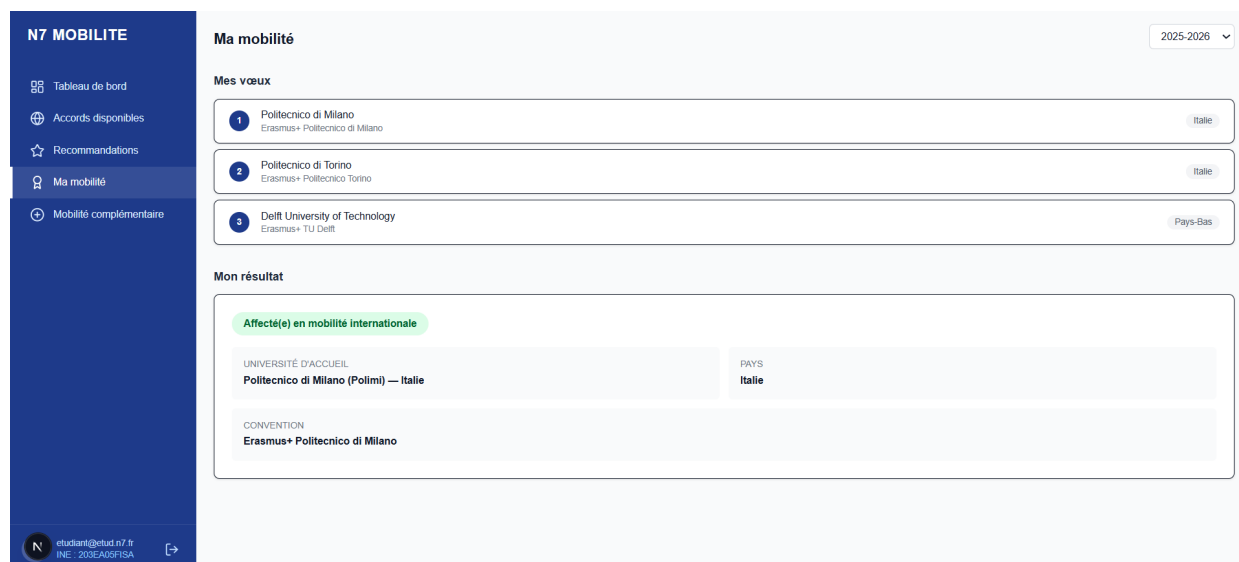


FIGURE 5.15 – Consultation des vœux et du résultat d’affectation (espace étudiant)

L’étudiant y retrouve ses vœux dans l’ordre choisi et, dès la publication, l’université et le pays de destination retenus, sans accès à l’affectation d’un autre étudiant.

5.3.5 Tests et validation du sprint

Ce sprint constitue le cœur fonctionnel du système : il produit et publie les propositions d’affectation issues des vœux et des quotas définis. Le tableau 5.6 porte sur la fiabilité de l’algorithme et sur le respect du cycle de supervision avant publication, validés au moyen de tests unitaires sur l’algorithme et de tests d’intégration sur les règles de gestion.

TABLE 5.6 – Validation du Sprint 3

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Algorithme d’affectation (Gale-Shapley)	Vérifier la stabilité des affectations, le respect des quotas et le comportement sur un volume représentatif	Unitaire	pytest	Conforme	Validé
Cycle de vie d’une affectation (transitions)	Vérifier que seules les transitions autorisées entre les statuts (proposé, validé, publié) sont acceptées	Intégration	pytest	Conforme	Validé
Correction manuelle des affectations	Vérifier qu’une modification manuelle n’est acceptée qu’avec un motif, et qu’elle est tracée dans le journal d’audit	Intégration	pytest	Conforme	Validé
Statistiques et consultation étudiant	Vérifier la cohérence des statistiques agrégées exposées au service RI et l’accès restreint de l’étudiant à son propre résultat	Intégration	pytest	Conforme	Validé

L’incrément du Sprint 3 est ainsi validé au regard des règles de gestion de l’affectation et de sa supervision.

5.4 Sprint 4 : Sources complémentaires et mobilités

L’intégration de nouvelles sources de données (Eudonet, Excel) et la déclaration des mobilités hors cursus font l’objet de ce sprint, dont le backlog, la conception, l’implémentation et les tests sont détaillés ci-après.

5.4.1 Objectif du sprint

Le Sprint 4 élargit le périmètre du système en intégrant de nouvelles sources de données et en prenant en compte d’autres formes de mobilité internationale. Il concerne notamment les stages internationaux, les mobilités entrantes et les expériences complémentaires déclarées par les étudiants.

Le sprint vise également à améliorer le traitement des dossiers qui ne peuvent pas être rapprochés automatiquement d’un étudiant et à automatiser la répartition des places dispo-

nibles entre les départements. À son terme, ces nouvelles informations peuvent être intégrées et exploitées par le système.

Définition of Done. À l’issue de ce sprint, les imports Eudonet et Excel s’exécutent sans erreur et produisent un rapport détaillé, les quotas par département sont recalculés automatiquement à l’ouverture de la phase de recommandation, les enregistrements sans INE valide sont proposés à la réconciliation manuelle plutôt que rejetés silencieusement, et un étudiant peut déclarer une expérience complémentaire avec justificatif depuis son espace personnel.

5.4.2 Sprint Backlog : Sprint 4

Après la mise en place de la gestion des données étudiants et des affectations, ce sprint vise à élargir le périmètre du système à de nouvelles sources de données et à de nouveaux types de mobilité. Il concerne notamment la répartition des quotas entre les départements, l’intégration des stages internationaux et des mobilités entrantes, ainsi que la gestion des expériences internationales complémentaires déclarées par les étudiants. Le tableau 5.7 présente les tâches retenues pour ce sprint, ainsi que leur priorité et leur estimation.

TABLE 5.7 – Sprint Backlog du Sprint 4

#	Tâche	Description	Critères d’acceptation	Prior.	SP
US-11 : Répartition des quotas par département : <i>En tant qu’administrateur RI, je veux que le système calcule automatiquement la répartition des places de chaque accord entre les départements pédagogiques selon la méthode Hamilton (plus grands restes), afin que l’algorithme d’affectation dispose de quotas par département précis, équitables et vérifiables.</i>					
01	Mettre en place le calcul de répartition des quotas	Développer le mécanisme permettant de répartir automatiquement les places disponibles d’un accord entre les différents départements, en tenant compte des informations disponibles sur les mobilités précédentes.	La répartition est calculée automatiquement et le total des quotas attribués aux départements correspond au nombre total de places disponibles.	Haute	2
02	Permettre le recalcul des quotas	Ajouter une fonctionnalité permettant de recalculer la répartition des quotas lorsqu’une modification des données ou de la configuration le nécessite.	Le recalcul met à jour les quotas des départements et les nouvelles valeurs peuvent être consultées dans le système.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
03	Intégrer le calcul au déroulement de la campagne	Déclencher automatiquement la répartition des quotas lors du passage à la phase de recommandation afin de garantir que les affectations utilisent des données à jour.	Les quotas sont calculés avant le lancement de la phase d'affectation.	Haute	1
04	Afficher la répartition des quotas	Présenter, dans l'interface de gestion des accords, le nombre de places attribuées à chaque département.	Les quotas de chaque département sont visibles et leur total correspond au quota global de l'accord.	Haute	1
US-12 : Stages internationaux (Eudonet) : En tant qu'administrateur RI, je veux importer et synchroniser les conventions de stages internationaux depuis Eudonet afin de les comptabiliser dans le calcul CTI de chaque étudiant.					
05	Mettre en place la récupération des données de stage	Développer le mécanisme permettant de récupérer les informations relatives aux stages internationaux depuis la source Eudonet.	Les données de stage peuvent être récupérées et les éventuelles erreurs de communication sont prises en compte.	Haute	2
06	Créer le modèle de gestion des stages	Ajouter la structure nécessaire pour enregistrer les informations principales relatives aux stages internationaux des étudiants.	Les informations essentielles d'un stage peuvent être enregistrées et rattachées à l'étudiant concerné.	Haute	2
07	Importer et mettre à jour les stages	Mettre en place le processus permettant d'intégrer les données récupérées dans le système et de mettre à jour les informations existantes si nécessaire.	Les stages valides sont enregistrés ou mis à jour, tandis que les anomalies rencontrées sont signalées dans un rapport d'import.	Haute	2
08	Consulter les stages importés	Créer une interface permettant au service des relations internationales de consulter et rechercher les stages internationaux enregistrés.	Les stages peuvent être affichés et filtrés selon différents critères utiles à leur suivi.	Haute	2
US-13 : Mobilités entrantes (Excel) : En tant qu'administrateur RI, je veux importer les mobilités entrantes via un template Excel standardisé afin de centraliser le suivi des étudiants internationaux accueillis à l'ENSEEIH.					

#	Tâche	Description	Critères d'acceptation	Prior.	SP
09	Préparer un modèle de fichier d'import	Mettre à disposition un fichier Excel standardisé indiquant les informations nécessaires à l'import des mobilités entrantes.	Le fichier peut être téléchargé et contient les informations attendues pour réaliser l'import.	Haute	1
10	Lire et vérifier les données du fichier	Mettre en place le traitement du fichier importé afin de contrôler les informations fournies avant leur intégration dans le système.	Les données valides sont retenues et les anomalies sont identifiées avec une indication de leur origine.	Haute	2
11	Créer le modèle des mobilités entrantes	Ajouter la structure permettant d'enregistrer les informations relatives aux étudiants internationaux accueillis à l'ENSEEIH.	Une mobilité entrante peut être enregistrée avec les principales informations concernant l'étudiant, son établissement d'origine et sa période de mobilité.	Haute	2
12	Permettre l'import des mobilités entrantes	Ajouter une fonctionnalité permettant à l'administrateur de déposer un fichier Excel afin d'intégrer plusieurs mobilités entrantes en une seule opération.	L'import est effectué à partir du fichier fourni et un rapport permet d'identifier les données correctement traitées et les éventuelles erreurs.	Haute	2
13	Consulter les mobilités entrantes	Créer une interface permettant de consulter les étudiants internationaux importés et d'effectuer une recherche ou un filtrage selon différents critères.	Les mobilités entrantes sont affichées sous forme de liste et peuvent être consultées facilement.	Haute	1
US-14 : Réconciliation sans INE : <i>En tant qu'administrateur RI, je veux que le système me propose des correspondances pour les enregistrements sans INE valide (via nom et prénom) afin de résoudre manuellement les cas ambigus.</i>					
14	Identifier les données non rapprochées automatiquement	Repérer les enregistrements pour lesquels l'identifiant national ne permet pas d'effectuer automatiquement le rapprochement avec un étudiant existant.	Les enregistrements concernés sont conservés et peuvent être consultés afin d'être traités ultérieurement.	Haute	1

#	Tâche	Description	Critères d'acceptation	Prior.	SP
15	Proposer des correspondances possibles	Mettre en place un mécanisme permettant de rechercher des étudiants susceptibles de correspondre à un enregistrement incomplet à partir des informations disponibles, notamment le nom et le prénom.	Les correspondances proposées sont classées selon leur degré de similarité.	Haute	2
16	Afficher les propositions de rapprochement	Permettre à l'administrateur de consulter les correspondances proposées pour chaque enregistrement qui n'a pas pu être rapproché automatiquement.	Les informations de l'enregistrement et les différentes propositions peuvent être consultées avant la prise de décision.	Haute	1
17	Permettre la résolution manuelle	Ajouter une interface permettant à l'administrateur de choisir la correspondance appropriée ou de traiter autrement un enregistrement ambigu.	La décision prise est enregistrée et l'action réalisée reste traçable.	Moyenne	1
US-15 : Déclaration mobilité complémentaire : <i>En tant qu'étudiant, je veux déclarer une expérience internationale hors cursus (summer school, séjour linguistique, programme court) avec dépôt de justificatif, afin qu'elle soit prise en compte dans mon calcul de durée CTI.</i>					
18	Créer le modèle de mobilité complémentaire	Ajouter la structure permettant d'enregistrer les expériences internationales déclarées par les étudiants et de suivre leur état de traitement.	Une mobilité complémentaire peut être enregistrée et suit les différentes étapes prévues jusqu'à sa validation ou son rejet.	Haute	2
19	Permettre le dépôt d'un justificatif	Ajouter la possibilité de joindre un document permettant de justifier l'expérience internationale déclarée par l'étudiant.	Le document est enregistré lorsque son format et sa taille sont conformes aux règles définies.	Haute	2
20	Enregistrer une déclaration de mobilité	Mettre à disposition les fonctionnalités nécessaires pour permettre à un étudiant de transmettre une nouvelle déclaration et de consulter ses déclarations existantes.	Une nouvelle déclaration est enregistrée avec le statut initial prévu et le service RI est informé de son arrivée.	Haute	2
21	Créer l'interface de déclaration	Développer un formulaire permettant à l'étudiant de renseigner les informations relatives à son expérience internationale et de suivre l'état de ses demandes.	L'étudiant peut soumettre une déclaration et consulter l'état des demandes déjà envoyées.	Haute	2

#	Tâche	Description	Critères d'acceptation	Prior.	SP
US-16 : Validation des mobilités complémentaires : <i>En tant qu'administrateur RI, je veux consulter les justificatifs soumis, valider ou rejeter chaque demande avec un motif, afin de maintenir la qualité des données comptabilisées dans le calcul CTI.</i>					
22	Consulter les demandes reçues	Mettre à disposition une fonctionnalité permettant au service RI de consulter les mobilités complémentaires soumises par les étudiants.	Les demandes peuvent être affichées et filtrées selon leur état de traitement.	Haute	1
23	Valider ou rejeter une demande	Permettre à l'administrateur de prendre une décision concernant une mobilité complémentaire après consultation des informations et du justificatif fourni.	Une demande peut être validée ou rejetée, et le motif du rejet est conservé lorsqu'il est nécessaire.	Haute	2
24	Créer l'interface de validation	Mettre en place un espace permettant au service RI de consulter les demandes en attente, d'accéder aux justificatifs et de prendre une décision.	Les demandes en attente sont facilement identifiées et les actions de validation ou de rejet peuvent être réalisées depuis l'interface.	Haute	2

5.4.3 Conception détaillée

Voici la conception UML des entités introduites par ce sprint.

5.4.3.1 Diagramme de classes

Le diagramme de classes 5.16 regroupe les principaux éléments ajoutés pour gérer les nouvelles sources de données et les mobilités complémentaires. Il permet de visualiser leur lien avec les éléments déjà introduits dans les sprints précédents.

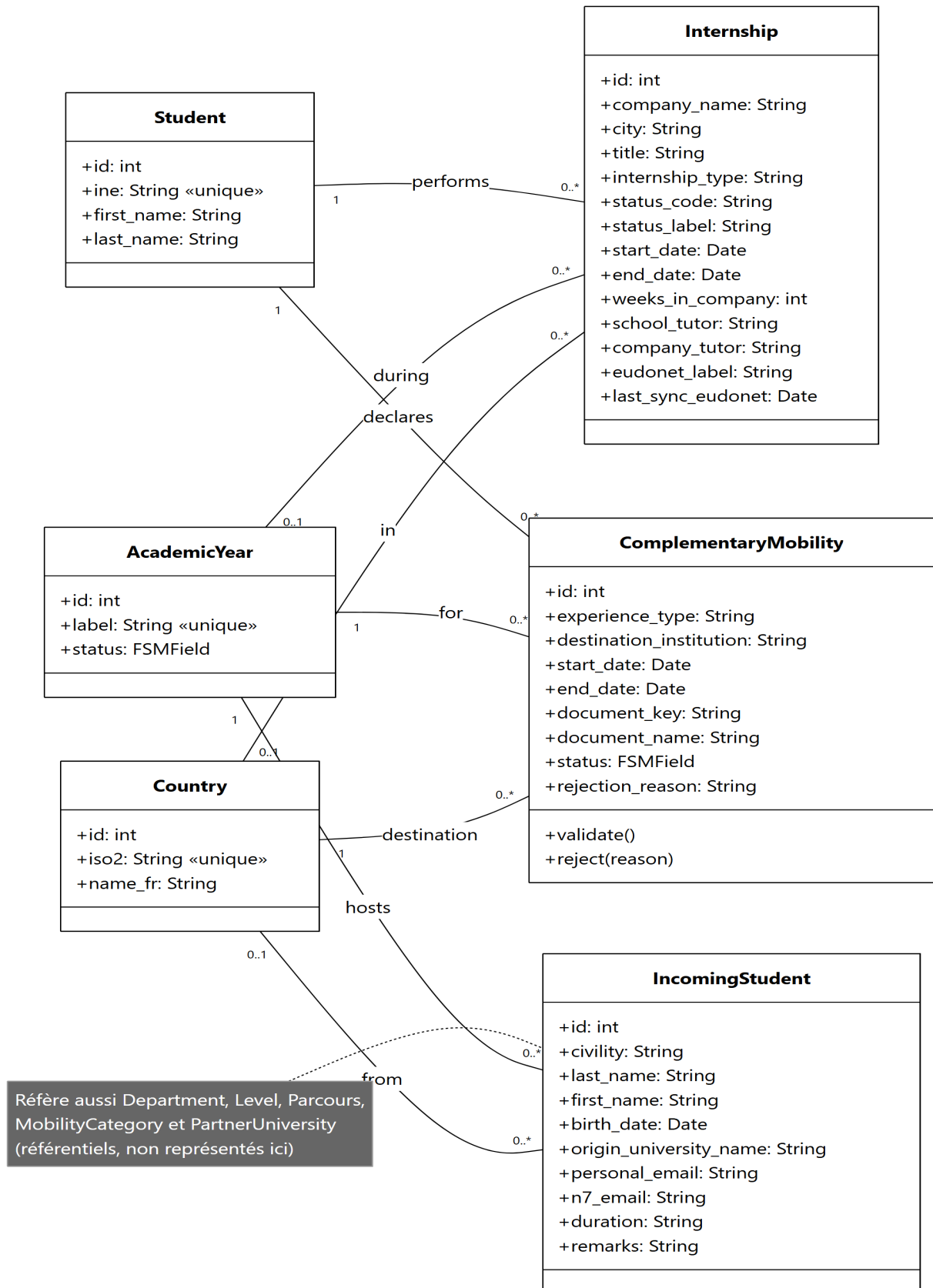


FIGURE 5.16 – Diagramme de classes : Sprint 4

5.4.3.2 Diagrammes de séquence

Deux scénarios propres à ce sprint sont détaillés : la réconciliation d'un dossier sans identifiant, présentée en figure 5.17, puis la déclaration d'une mobilité complémentaire.

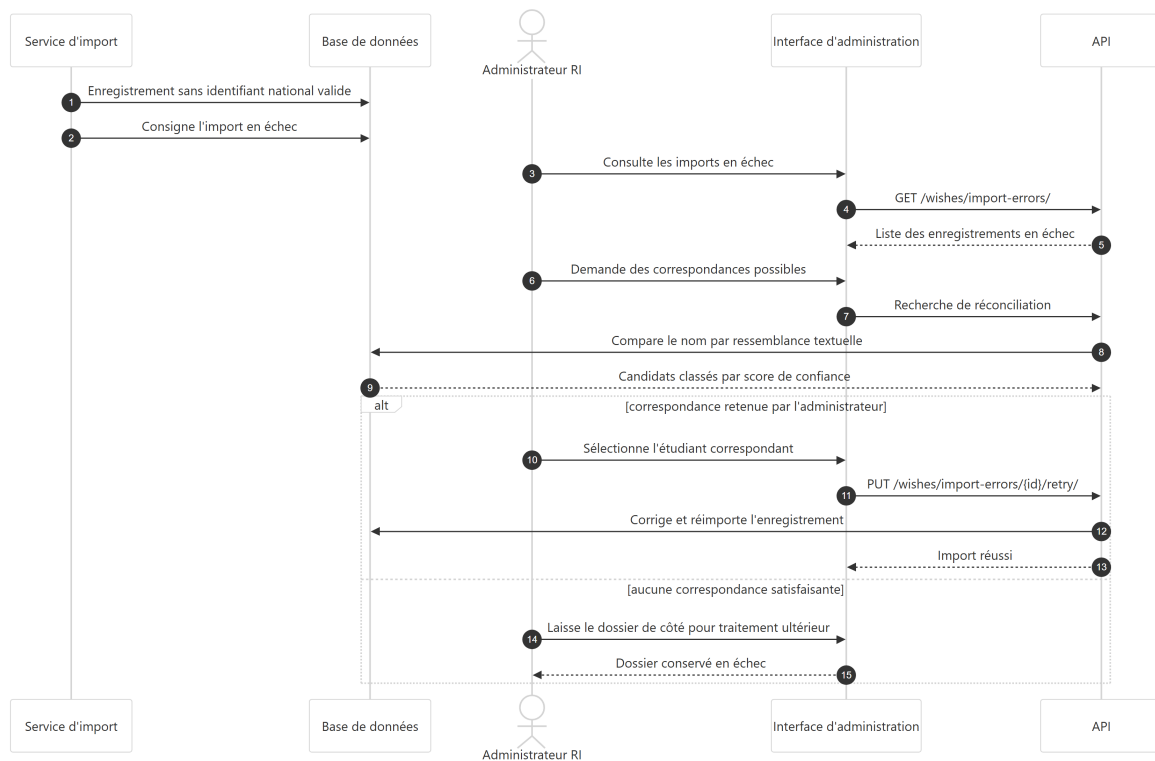


FIGURE 5.17 – Réconciliation d'un dossier sans identifiant valide

Le système propose des correspondances possibles en comparant le nom du dossier en échec à ceux des étudiants déjà enregistrés, classées par degré de confiance ; c'est ensuite l'administrateur qui choisit de retenir une correspondance ou de laisser le dossier de côté pour un traitement ultérieur.

La déclaration d'une mobilité complémentaire par un étudiant, jusqu'à son examen par le service RI, suit le scénario de la figure 5.18.

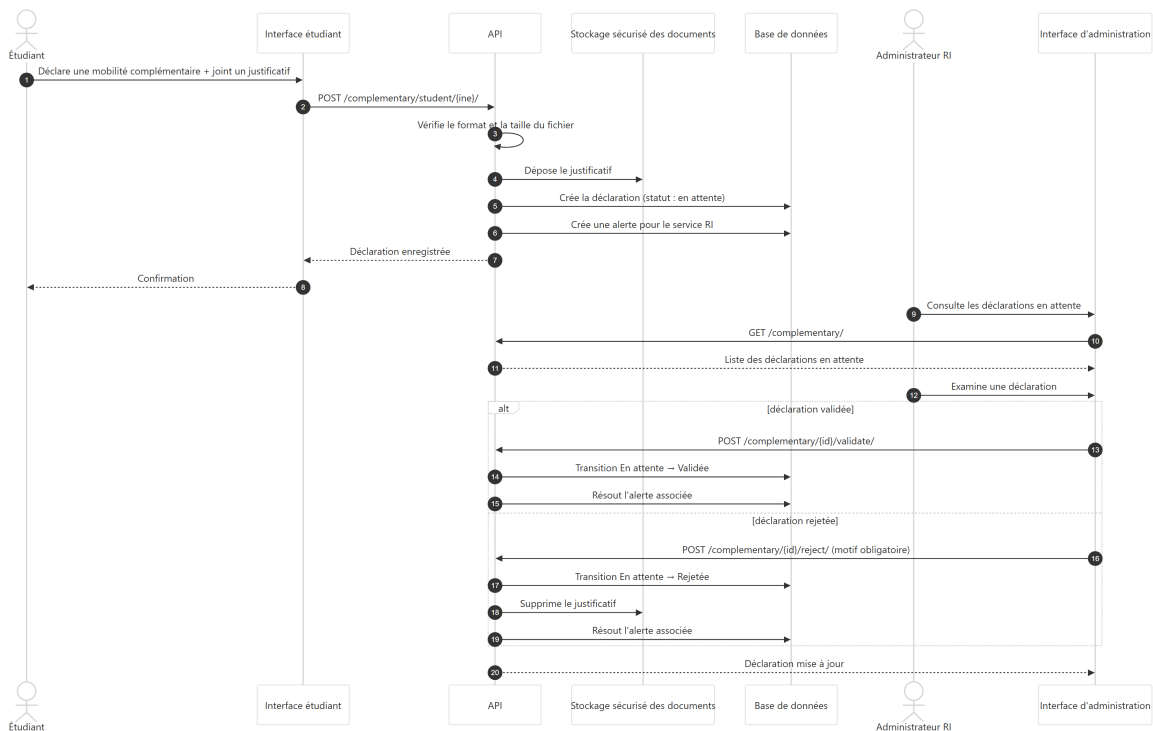


FIGURE 5.18 – Déclaration et traitement d'une mobilité complémentaire

Le dépôt d'un justificatif est obligatoire et déclenche automatiquement une alerte à destination du service RI, qui peut alors valider ou rejeter la déclaration ; un rejet exige un motif et entraîne la suppression du justificatif déposé, devenu inutile.

5.4.4 Implémentation

Les fonctionnalités réalisées dans ce sprint permettent d'intégrer de nouvelles informations liées à la mobilité internationale. Les stages et les mobilités entrantes peuvent être importés dans le système, tandis que les dossiers qui ne peuvent pas être associés automatiquement à un étudiant peuvent être examinés avec l'aide de propositions de correspondance.

La figure 5.19 présente l'interface de consultation des stages internationaux importés depuis Eudonet.

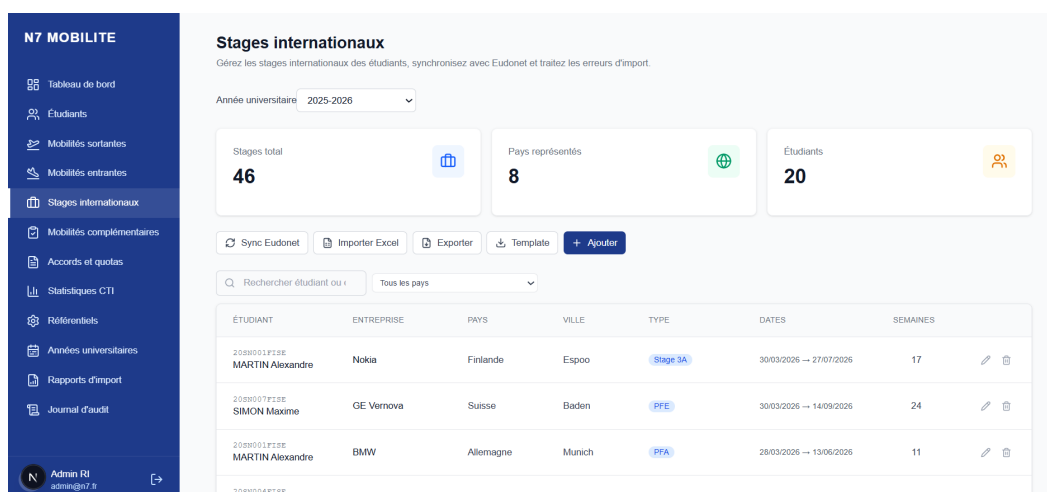


FIGURE 5.19 – Consultation des stages internationaux

Le service RI y retrouve, pour chaque stage synchronisé depuis Eudonet, l'étudiant concerné, l'entreprise d'accueil et la période, sans avoir à consulter Eudonet directement.

La figure 5.20 présente, de la même façon, l'interface de consultation des mobilités entrantes importées via le template Excel.

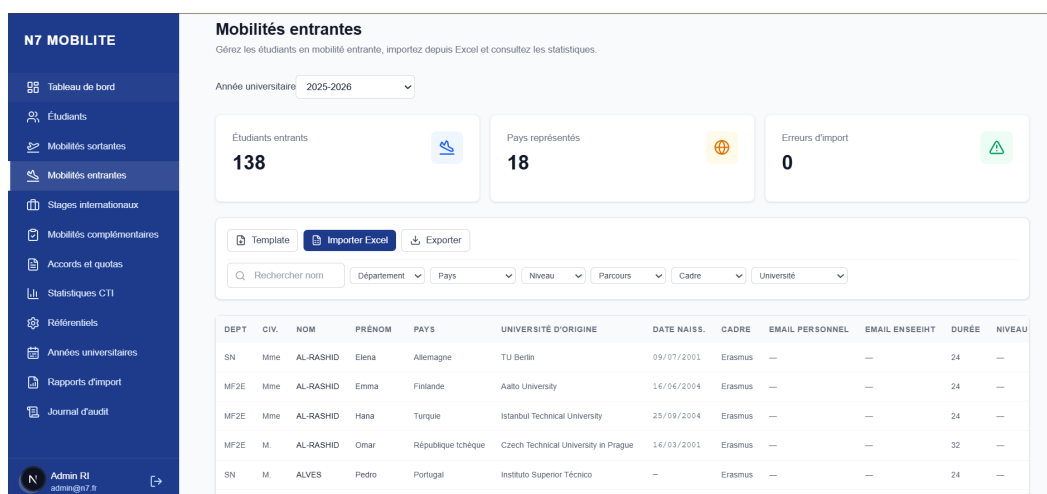


FIGURE 5.20 – Consultation des mobilités entrantes

Cette page centralise le suivi des étudiants internationaux accueillis à l'ENSEEIH, jusque-là dispersé dans des fichiers Excel gérés séparément par le service RI.

La figure 5.21 illustre l'écran de réconciliation, où l'administrateur retrouve un dossier resté sans identifiant national valide et les correspondances proposées par le système.

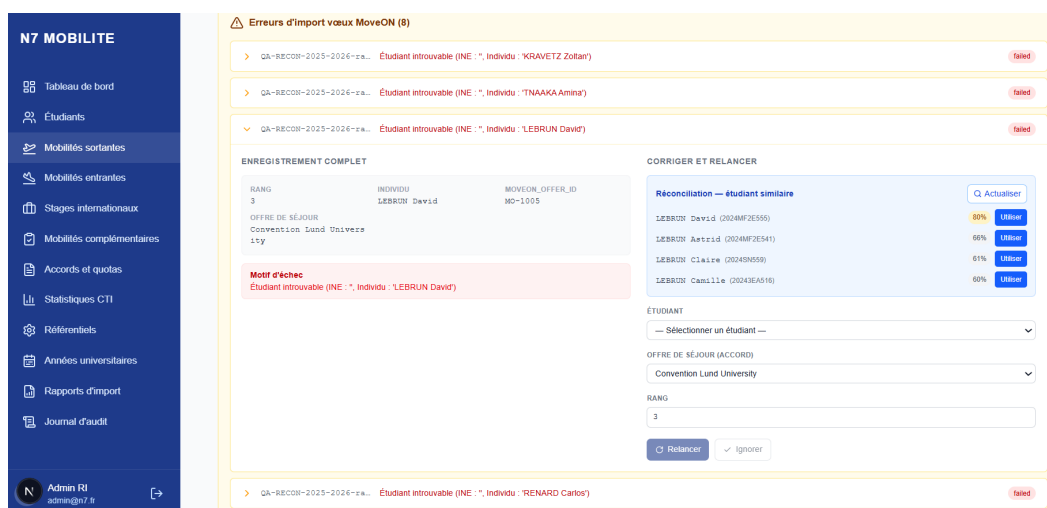


FIGURE 5.21 – Réconciliation d'un dossier sans identifiant valide

Les candidats proposés sont classés par degré de confiance, ce qui permet à l'administrateur de trancher rapidement sans devoir rechercher lui-même l'étudiant correspondant.

Le sprint introduit également la déclaration d'expériences complémentaires par les étudiants. La figure 5.22 présente le formulaire utilisé par l'étudiant pour déclarer une expérience et joindre son justificatif.

FIGURE 5.22 – Déclaration d'une mobilité complémentaire (espace étudiant)

L'étudiant y suit également le statut de ses déclarations précédentes, sans avoir besoin de solliciter le service RI pour savoir où en est son dossier.

Une fois la déclaration déposée, elle est transmise au service des relations internationales pour examen. Celui-ci peut alors consulter les informations fournies et prendre une décision.

La figure 5.23 illustre cette étape de validation. Elle met en évidence les informations mises à disposition de l'administrateur pour examiner une déclaration et choisir l'action appropriée.

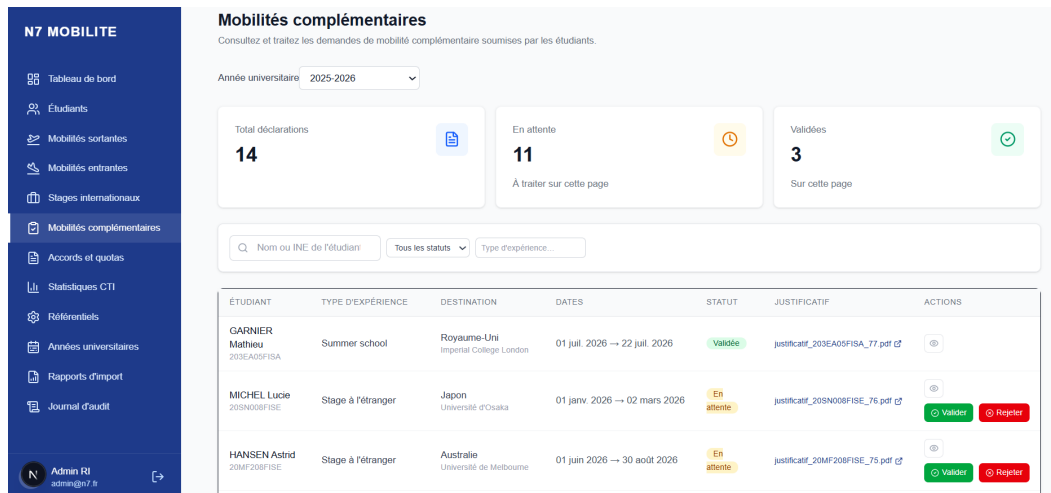


FIGURE 5.23 – Validation d'une mobilité complémentaire

Le service RI y dispose de toutes les informations nécessaires (type d'expérience, dates, justificatif) pour statuer sur la déclaration sans avoir à solliciter l'étudiant.

5.4.5 Tests et validation du sprint

Ce sprint élargit le système à des sources de données complémentaires ainsi qu'à la déclaration d'expériences hors cursus. Le tableau 5.8 porte sur la fiabilité des nouveaux imports, sur le calcul de répartition des quotas par département et sur le cycle de vie des déclarations complémentaires, validés au moyen de tests unitaires et de tests d'intégration.

TABLE 5.8 – Validation du Sprint 4

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Répartition des quotas par département (méthode Hamilton)	Vérifier que la somme des quotas répartis correspond au quota global, y compris en l'absence d'historique	Unitaire	pytest	Conforme	Validé
Nettoyage et normalisation des données importées	Vérifier la détection de doublons et la normalisation des données issues des différentes sources	Unitaire	pytest	Conforme	Validé
Import des stages internationaux et des mobilités entrantes	Vérifier la validation des données et la consignation des anomalies détectées	Unitaire	pytest	Conforme	Validé
Mobilités complémentaires (déclaration, justificatif, validation/rejet)	Vérifier le cycle de vie de la déclaration et le contrôle du justificatif déposé	Intégration	pytest	Conforme	Validé

L'incrément du Sprint 4 est ainsi validé au regard de la répartition des quotas par département, de l'intégration des nouvelles sources de données (stages internationaux, mobilités

entrantes), de la réconciliation des dossiers sans identifiant et du cycle de vie des mobilités complémentaires.

5.5 Sprint 5 : Conformité CTI et authentification

Ce sprint clôt cette première version du système avec le suivi de la conformité CTI, les tableaux de bord analytiques et l'authentification institutionnelle ; son backlog, sa conception, son implémentation et ses tests sont présentés dans les sections suivantes.

5.5.1 Objectif du sprint

Le Sprint 5 complète le système avec les fonctionnalités liées au suivi de la mobilité internationale dans le cadre de la conformité CTI, à la production de statistiques et à la sécurisation des accès.

L'objectif est de consolider les différentes expériences de mobilité d'un étudiant afin de calculer leur durée totale, de produire les informations nécessaires au suivi de la promotion et de fournir au service des relations internationales des indicateurs synthétiques. Ce sprint introduit également l'authentification institutionnelle et le contrôle des accès selon le rôle de l'utilisateur.

Définition of Done. À l'issue de ce sprint, la durée CTI de chaque étudiant est calculée automatiquement et consultable par l'étudiant, un rapport CTI est exportable pour l'ensemble de la promotion, les tableaux de bord analytiques sont alimentés par les données réelles des sprints précédents, et l'authentification institutionnelle protège l'ensemble des routes API et des pages Next.js selon le rôle de l'utilisateur.

5.5.2 Sprint Backlog : Sprint 5

Ce sprint reprend les besoins du Product Backlog relatifs au suivi de la conformité CTI, à l'exploitation des données de mobilité et à l'authentification des utilisateurs. Il vise notamment à automatiser le calcul des durées de mobilité, à permettre la consultation et l'export des résultats, à fournir des indicateurs de pilotage et à sécuriser l'accès aux différents espaces du système. Le tableau 5.9 présente les éléments retenus pour ce sprint.

TABLE 5.9 – Sprint Backlog du Sprint 5

#	Tâche	Description	Critères d'acceptation	Prior.	SP
	US-17 : Calcul automatique des durées CTI : <i>En tant qu'administrateur RI, je veux que le système calcule automatiquement la durée totale de mobilité de chaque étudiant (séjours académiques, stages internationaux, mobilités complémentaires validées) afin de vérifier la conformité CTI sans intervention manuelle.</i>				

#	Tâche	Description	Critères d'acceptation	Prior.	SP
01	Calculer la durée totale de mobilité	Mettre en place le calcul de la durée cumulée de mobilité pour chaque étudiant en tenant compte des séjours académiques, des stages internationaux et des mobilités complémentaires validées. Le résultat est conservé afin de permettre son suivi et sa consultation.	La durée totale correspond à la somme des différentes mobilités prises en compte pour l'étudiant. Les informations calculées peuvent être consultées avec le détail de chaque source.	Haute	3
02	Recalculer les durées des étudiants de dernière année	Permettre de recalculer et d'enregistrer, en une seule fois pour l'ensemble des étudiants de dernière année, la durée de mobilité de chaque étudiant. Le recalcul est déclenché par la finalisation de l'année universitaire et s'exécute en arrière-plan afin de ne pas bloquer les autres opérations du système.	Le recalcul pour les étudiants de dernière année s'exécute au moment de la finalisation CTI. Les durées enregistrées reflètent les mobilités validées à cet instant.	Haute	3
03	Consulter les informations de mobilité CTI	Permettre au service RI d'accéder, pour chaque étudiant, à la durée totale de mobilité ainsi qu'à la répartition des durées selon les différents types de mobilité.	La durée totale et le détail par type de mobilité sont accessibles pour chaque étudiant. Les informations affichées correspondent aux données calculées par le système.	Haute	2
US-18 : Affichage des durées de mobilité dans le tableau de bord étudiant : <i>En tant qu'étudiant, je veux visualiser ma durée cumulée de mobilité afin de suivre mon avancement.</i>					
04	Consulter sa durée cumulée de mobilité	Ajouter au tableau de bord étudiant un espace permettant de visualiser la durée totale de mobilité cumulée, exprimée en semaines.	L'étudiant peut consulter sa durée cumulée depuis son espace personnel. La valeur affichée correspond aux données calculées par le système.	Haute	2
05	Visualiser le détail des mobilités	Afficher la contribution des différents types de mobilité, notamment les séjours académiques, les stages internationaux et les mobilités complémentaires validées.	Chaque type de mobilité est identifiable et sa contribution au total est clairement affichée.	Haute	1
US-19 : Export rapport CTI : <i>En tant qu'administrateur RI, je veux exporter un rapport CTI pour l'ensemble des étudiants de la promotion afin de le soumettre lors des audits.</i>					

#	Tâche	Description	Critères d'acceptation	Prior.	SP
06	Générer le rapport de conformité CTI	Permettre la génération d'un rapport regroupant les informations nécessaires au suivi de la conformité CTI des étudiants d'une promotion ou d'une année universitaire.	Le rapport reconstitue le formulaire statistique officiel attendu par le service RI pour ses audits, avec les répartitions par genre, par régime de formation, par durée et par région.	Moyenne	3
07	Exporter le rapport par année universitaire	Permettre à l'administrateur RI de sélectionner l'année universitaire concernée avant de générer le rapport correspondant.	Le fichier exporté contient uniquement les étudiants associés à l'année universitaire sélectionnée.	Moyenne	2
US-20 : Tableaux de bord analytiques : <i>En tant qu'administrateur RI, je veux disposer de tableaux de bord analytiques (taux d'affectation par département, évolution des flux entrants/sortants, destinations les plus demandées, indicateurs CTI) afin d'alimenter le pilotage opérationnel et la réflexion stratégique sur les partenariats.</i>					
08	Consulter les principaux indicateurs de mobilité	Mettre à disposition des indicateurs permettant de suivre les affectations, les flux de mobilité entrants et sortants ainsi que les destinations les plus demandées.	Les indicateurs affichés correspondent aux données enregistrées dans le système et peuvent être consultés depuis un espace dédié.	Moyenne	3
09	Visualiser les données analytiques	Présenter les principales statistiques sous une forme graphique afin de faciliter l'analyse de l'évolution des mobilités et des tendances observées.	Les graphiques sont correctement alimentés par les données du système et permettent de visualiser les différents indicateurs.	Moyenne	3
10	Suivre la conformité CTI globale	Afficher un indicateur synthétique présentant le nombre et la proportion d'étudiants respectant les exigences de mobilité définies par la CTI.	L'indicateur de conformité est calculé à partir des données réelles de mobilité et reflète la situation de la population sélectionnée.	Moyenne	1
11	Exporter les données analytiques	Permettre l'export des données utilisées pour les indicateurs et les tableaux de bord afin de faciliter leur exploitation en dehors de la plateforme.	Le fichier exporté contient les informations correspondant aux données affichées dans le tableau de bord.	Moyenne	1
US-21 : Authentification SSO : <i>En tant qu'utilisateur, je veux m'authentifier via le SSO institutionnel (CAS v2) afin d'accéder à mon espace personnalisé (étudiant ou administrateur) sans créer de compte séparé.</i>					

#	Tâche	Description	Critères d'acceptation	Prior.	SP
12	S'authentifier via le SSO institutionnel	Intégrer le mécanisme d'authentification institutionnelle afin de permettre aux utilisateurs d'accéder à la plateforme avec leurs identifiants habituels, sans création de compte spécifique.	Un utilisateur peut se connecter avec le SSO institutionnel et une session valide est établie après authentification.	Haute	3
13	Identifier le profil de l'utilisateur	Déterminer automatiquement le profil de l'utilisateur connecté afin de distinguer les étudiants des administrateurs RI et d'adapter les accès à leurs besoins.	Le profil détecté correspond aux informations associées à l'utilisateur et permet d'identifier correctement son type d'accès.	Haute	1
14	Accéder à son espace personnalisé	Rediriger l'utilisateur vers l'espace correspondant à son profil après son authentification.	Après la connexion, chaque utilisateur est automatiquement dirigé vers l'espace correspondant à son rôle.	Haute	1
15	Sécuriser l'accès aux fonctionnalités	Mettre en place un contrôle des accès afin que chaque utilisateur puisse uniquement consulter et utiliser les fonctionnalités autorisées pour son profil.	Les utilisateurs non authentifiés ne peuvent pas accéder aux ressources protégées. Les utilisateurs authentifiés ne peuvent accéder qu'aux fonctionnalités correspondant à leur rôle.	Haute	2
16	Vérifier le parcours d'authentification	Tester le parcours complet de connexion, l'identification du profil et la protection des espaces afin de s'assurer du bon fonctionnement de l'authentification institutionnelle.	Le parcours de connexion fonctionne correctement et les accès non autorisés sont refusés.	Haute	1

5.5.3 Conception détaillée

Le Sprint 5 n'introduit pas de nouvelles entités métier majeures. Les fonctionnalités liées au suivi de la conformité CTI et aux tableaux de bord analytiques reposent principalement sur l'exploitation et l'agrégation des données déjà modélisées lors des sprints précédents, notamment `AssignmentResult`, `Internship`, `ComplementaryMobility` et `IncomingStudent`. De même, l'authentification institutionnelle s'appuie sur le modèle utilisateur standard de Django et ne nécessite pas l'ajout d'un nouveau modèle métier spécifique.

Une seule nouvelle entité, `MobilityDuration`, est introduite dans le module `cti`. Elle

permet de conserver, pour chaque étudiant, les durées cumulées associées aux différentes formes de mobilité : séjours académiques, stages internationaux et mobilités complémentaires. La durée totale est également enregistrée afin de faciliter son exploitation dans les différentes fonctionnalités du système.

Ces informations sont mises à jour lors des opérations susceptibles de modifier la durée de mobilité d'un étudiant. Cette approche permet d'éviter de recalculer systématiquement les durées à chaque consultation, notamment lors de la génération des rapports CTI ou du calcul des indicateurs statistiques.

5.5.3.1 Diagrammes de séquence

Le scénario de finalisation CTI, présenté en figure 5.24, complète les diagrammes déjà introduits aux sprints précédents.

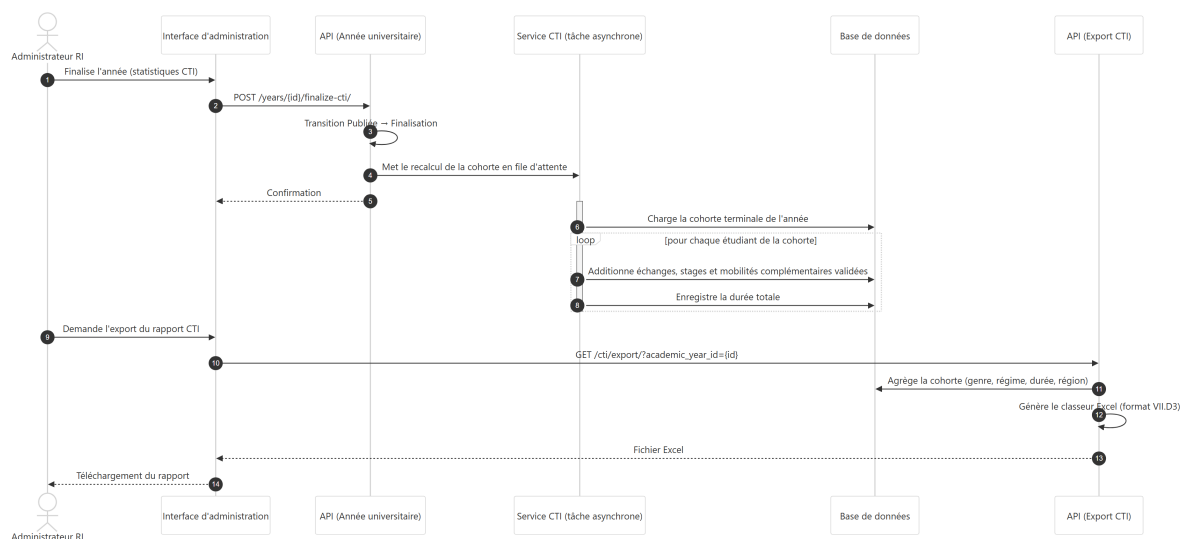


FIGURE 5.24 – Finalisation CTI et génération du rapport

Le recalcul des durées de mobilité des étudiants de dernière année s'exécute en tâche de fond, déclenché par le seul passage de l'année en phase de finalisation. L'export du rapport, demandé séparément par l'administrateur, n'est pas enchaîné automatiquement après ce recalcul : il agrège directement les données de ces étudiants au moment de la demande, pour produire le formulaire statistique attendu par le service RI.

5.5.4 Implémentation

Ce sprint regroupe plusieurs fonctionnalités destinées à exploiter les données de mobilité déjà collectées par le système. Les différentes expériences prises en compte peuvent être regroupées afin de calculer la durée totale de mobilité d'un étudiant. Ces informations servent ensuite à produire les données nécessaires au suivi de la conformité CTI.

Côté étudiant, la figure 5.25 présente le tableau de bord où cette durée cumulée est consultée, avec le détail par type de mobilité.

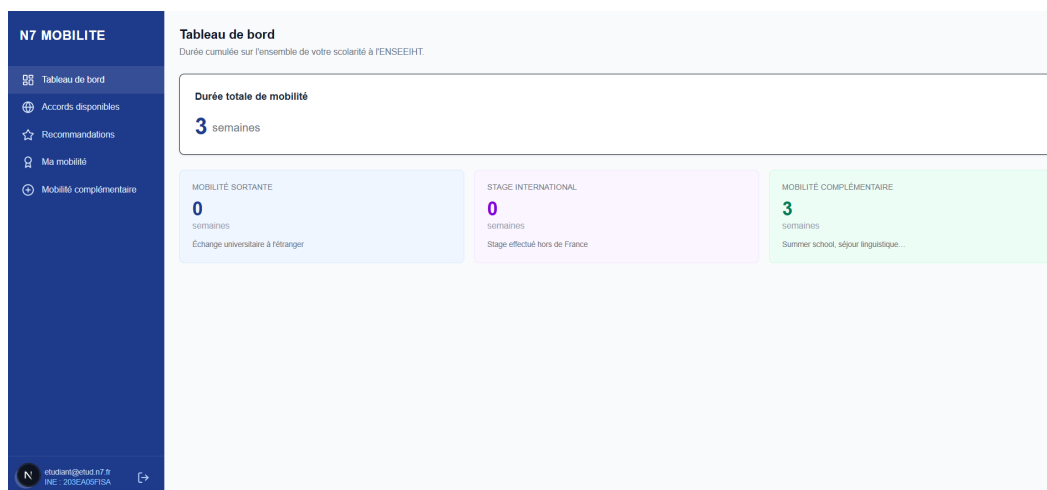


FIGURE 5.25 – Tableau de bord étudiant : durée de mobilité cumulée

L'étudiant y retrouve, en un coup d'œil, la contribution de chaque type de mobilité (échanges académiques, stages internationaux, mobilités complémentaires) au total affiché.

Le service des relations internationales dispose également d'indicateurs permettant d'observer les principales évolutions et répartitions des mobilités. Les résultats peuvent être consultés sous forme de tableaux ou de graphiques et certaines données peuvent être exportées.

Enfin, l'accès au système est désormais lié à l'authentification institutionnelle. Les utilisateurs n'ont accès qu'aux fonctionnalités et aux données correspondant à leur rôle.

La figure 5.26 présente le rapport de conformité CTI consulté par le service RI, avec les effectifs de la promotion terminale et le détail des mobilités par sexe et par régime de formation.



FIGURE 5.26 – Rapport de conformité CTI

Le formulaire reprend exactement la structure attendue par la CTI, ce qui évite au service RI toute retranscription manuelle au moment des audits d'accréditation.

Le tableau de bord analytique se décline ensuite en plusieurs vues. La figure 5.27 en

présente la vue d'ensemble, avec l'évolution des flux sortants, entrants, des stages et des mobilités complémentaires sur les cinq dernières années.



FIGURE 5.27 – Tableau de bord analytique : évolution des flux

La même évolution peut ensuite être détaillée par département, comme le montre la figure 5.28.

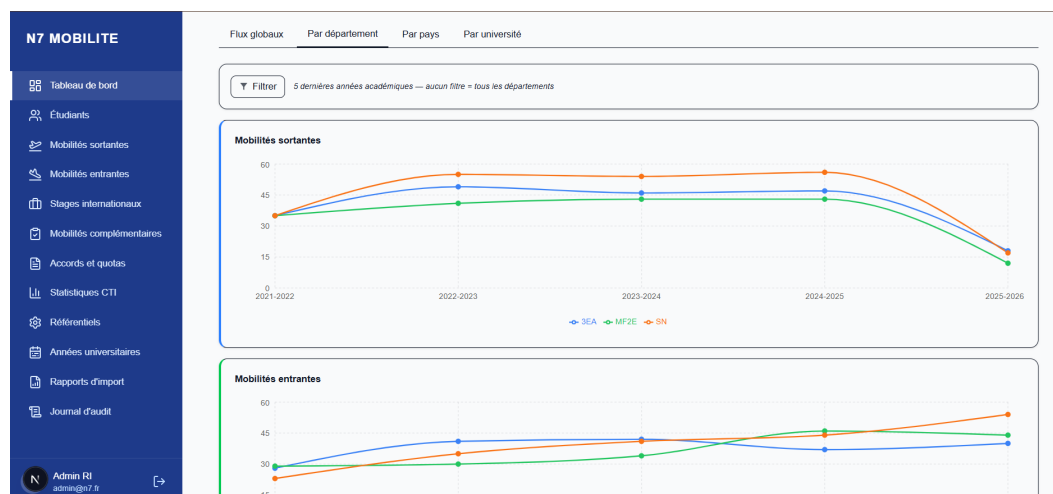


FIGURE 5.28 – Tableau de bord analytique : répartition par département

Ou par université partenaire, pour repérer les établissements les plus sollicités (figure 5.29).

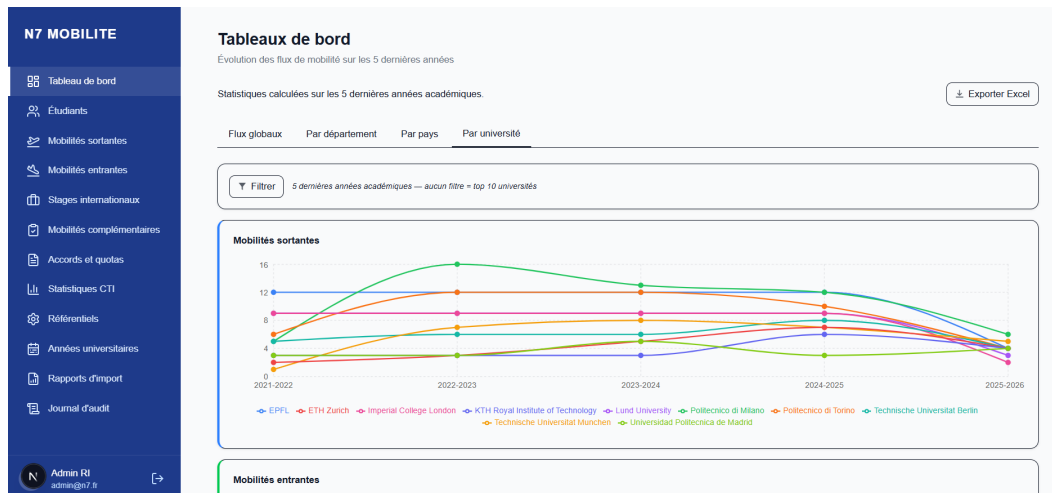


FIGURE 5.29 – Tableau de bord analytique : répartition par université partenaire

Et enfin par pays de destination (figure 5.30).



FIGURE 5.30 – Tableau de bord analytique : répartition par pays

Ces différentes vues, toutes exportables en Excel, donnent au service RI une vision synthétique de l'évolution des flux, utile pour préparer les échanges avec les partenaires et anticiper les besoins des campagnes suivantes.

5.5.5 Tests et validation du sprint

Ce sprint introduit le calcul de conformité CTI, les tableaux de bord analytiques et l'authentification institutionnelle du système. Le tableau 5.10 confirme la validation du calcul et de l'export CTI, des statistiques analytiques, ainsi que de l'authentification et du contrôle d'accès par rôle, au moyen de tests unitaires et de tests d'intégration.

TABLE 5.10 – Validation du Sprint 5

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Calcul de la durée CTI (échanges, stages, mobilités complémentaires)	Vérifier l'agrégation par source, l'exclusion des stages en France et des mobilités non validées, et l'arrondi de la durée	Unitaire	pytest	Conforme	Validé
API de consultation et de recalcul de la durée CTI	Vérifier la réponse pour un étudiant sans mobilité, la persistance à l'issue du recalcul et son idempotence	Intégration	pytest	Conforme	Validé
Tâche asynchrone de recalcul CTI (Django-Q2)	Vérifier la mise en file d'attente et le traitement de chaque étudiant de dernière année	Unitaire	pytest	Conforme	Validé
Export du rapport CTI et statistiques associées	Vérifier la génération d'un fichier Excel valide, son en-tête de téléchargement et la cohérence des répartitions par genre et par région	Intégration	pytest	Conforme	Validé
Statistiques analytiques (tendances, répartitions par département, pays, université)	Vérifier la structure et la cohérence des séries retournées par les endpoints d'agrégation	Intégration	pytest	Conforme	Validé
Authentification (jeton absent, expiré ou falsifié)	Vérifier le refus d'accès pour toute requête sans jeton valide, avec un jeton expiré ou avec un jeton altéré	Intégration	pytest	Conforme	Validé
Autorisation : étudiant vers ressource administrateur	Vérifier qu'un étudiant authentifié ne peut pas accéder à un endpoint réservé à l'administrateur	Intégration	pytest	Conforme	Validé
Autorisation : étudiant vers dossier d'un autre étudiant	Vérifier qu'un étudiant authentifié ne peut accéder qu'à son propre dossier, et non à celui d'un autre étudiant	Intégration	pytest	Conforme	Validé
Authentification par cookie et endpoints publics	Vérifier le comportement de l'authentification par cookie ainsi que l'accès sans restriction aux endpoints publics	Intégration	pytest	Conforme	Validé

Le calcul et l'export CTI, les statistiques analytiques et les contrôles d'accès testés respectent les règles de gestion définies ; l'incrément du Sprint 5 est ainsi validé, sur ce périmètre backend, au regard des tests disponibles. La couverture frontend des nouvelles pages (tableau de bord étudiant, tableaux de bord analytiques, connexion) reste, elle, un point à compléter.

5.6 Sprint 6 : Fonctionnalités intelligentes (V2)

5.6.1 Objectif du sprint

Le Sprint 6 est consacré à l'intégration de fonctionnalités intelligentes destinées à accompagner les étudiants dans leurs démarches de mobilité internationale. Il comprend, d'une part, un moteur de recommandation permettant de proposer des destinations adaptées au profil de chaque étudiant avant la formulation de ses vœux dans MoveON (US-22). D'autre part, il intègre un assistant virtuel capable de répondre aux questions relatives aux procédures, aux critères d'éligibilité et aux différentes démarches administratives (US-23).

Ces deux fonctionnalités s'appuient sur les approches présentées et justifiées au chapitre 4, notamment l'utilisation d'un modèle de régression logistique pour la recommandation et d'une architecture RAG basée sur LlamaIndex, ChromaDB et un modèle de langage pour l'assistant virtuel.

Définition of Done. À l'issue de ce sprint, l'étudiant doit pouvoir consulter des destinations recommandées en fonction de son profil et des données historiques disponibles. Il doit également pouvoir interroger un assistant virtuel afin d'obtenir des informations à partir de la documentation institutionnelle, sans que ses données personnelles ne soient transmises au modèle de langage.

5.6.2 Sprint Backlog : Sprint 6

Ce sprint regroupe les fonctionnalités intelligentes prévues dans le Product Backlog et destinées à améliorer l'accompagnement des étudiants. Le tableau 5.11 présente les différentes tâches planifiées pour la mise en œuvre du moteur de recommandation et de l'assistant virtuel.

TABLE 5.11 – Sprint Backlog du Sprint 6

#	Tâche	Description	Critères d'acceptation	Prior.	SP
	US-22 : Moteur de recommandation : <i>En tant qu'étudiant, je veux recevoir des suggestions de destinations adaptées à mon profil (GPA, département, historique des affectations similaires) afin de cibler mes vœux de manière réaliste avant la saisie dans MoveON.</i>				

#	Tâche	Description	Critères d'acceptation	Prior.	SP
01	Préparer les données historiques des destinations	Exploiter les résultats des campagnes précédentes afin de déterminer, pour chaque destination, les informations utiles à l'évaluation des possibilités d'affectation des étudiants.	Les données historiques sont disponibles pour les destinations disposant d'un historique d'affectation.	Moyenne	2
02	Mettre en place le modèle de recommandation	Utiliser les informations relatives au profil académique de l'étudiant et aux résultats des campagnes précédentes afin de calculer un score pour les destinations disponibles.	Le modèle produit un score de recommandation pour les destinations éligibles lorsqu'un historique suffisant est disponible.	Moyenne	3
03	Proposer des destinations adaptées à l'étudiant	Mettre à disposition les résultats du moteur de recommandation afin de fournir à chaque étudiant une liste de destinations classées selon leur niveau de pertinence.	Les destinations proposées sont classées selon leur score de recommandation et seules les destinations éligibles sont affichées.	Moyenne	3
04	Afficher les recommandations dans l'espace étudiant	Créer une interface permettant à l'étudiant de consulter les destinations qui lui sont recommandées avant la saisie de ses vœux dans MoveON.	Le classement des destinations est affiché correctement et correspond aux résultats fournis par le moteur de recommandation.	Moyenne	3
05	Vérifier le fonctionnement du moteur de recommandation	Tester les différentes étapes du fonctionnement du moteur, notamment la préparation des données historiques, le calcul des scores et la gestion des situations où peu ou aucune donnée historique n'est disponible.	Les tests couvrent les cas avec historique ainsi que les situations de démarrage à froid.	Moyenne	2
US-23 : Assistant virtuel (chatbot RAG) : En tant qu'étudiant, je veux poser des questions sur les procédures, critères d'éligibilité et démarches à un assistant virtuel afin d'obtenir des réponses immédiates sans solliciter le service RI.					
06	Préparer la documentation destinée à l'assistant	Collecter et organiser les documents institutionnels contenant les informations relatives aux procédures de mobilité, aux critères d'éligibilité et aux différentes démarches administratives afin de les rendre exploitables par l'assistant virtuel.	Les documents retenus sont correctement intégrés à la base de connaissances de l'assistant.	Basse	5

#	Tâche	Description	Critères d'acceptation	Prior.	SP
07	Mettre en place le mécanisme de recherche et de génération des réponses	Permettre à l'assistant d'identifier les informations pertinentes dans la documentation disponible, puis de les utiliser pour construire une réponse adaptée à la question posée par l'étudiant.	Les réponses fournies s'appuient sur les informations issues de la documentation institutionnelle.	Basse	5
08	Permettre l'envoi des questions à l'assistant	Mettre en place le service permettant de transmettre une question à l'assistant et de récupérer la réponse générée, sans inclure les données personnelles de l'étudiant dans la requête adressée au modèle de langage.	Une question peut être envoyée et une réponse est retournée. Aucune donnée personnelle de l'étudiant n'est transmise au modèle de langage.	Basse	3
09	Créer l'interface de conversation	Intégrer dans l'espace étudiant une interface permettant de poser des questions à l'assistant virtuel et de consulter les réponses au sein d'un même espace de conversation.	Les questions et les réponses s'affichent correctement dans le fil de conversation.	Basse	5
10	Vérifier la qualité des réponses de l'assistant	Tester le fonctionnement de l'assistant à partir d'un ensemble de questions représentatives afin de vérifier que les réponses reposent sur les informations disponibles dans la documentation institutionnelle.	Les réponses obtenues restent cohérentes avec les documents utilisés comme sources de connaissances.	Basse	3

5.6.3 Conception détaillée

Le moteur de recommandation ne nécessite pas l'ajout de nouvelles entités persistées dans la base de données. Il est implémenté sous la forme d'un service dédié, au sein de l'application `recommandation`, et exploite les données déjà disponibles dans le système. Ces données concernent notamment les vœux des étudiants (`StudentWish`), les résultats des affectations précédentes (`AssignmentResult`), les accords de mobilité (`Agreement` et `AgreementYear`) ainsi que les informations relatives à l'inscription annuelle des étudiants (`AnnualEnrollment`).

Afin de prendre en compte les résultats des campagnes précédentes, un taux historique d'affectation est calculé pour chaque accord. Ce choix permet d'exploiter les données disponibles sur plusieurs années universitaires et d'obtenir une information plus représentative de l'historique des affectations.

À partir de ces données, le moteur utilise un modèle de régression logistique pour estimer la pertinence des différentes destinations pour un étudiant. Le modèle prend notamment en compte le GPA de l'étudiant, son département et les résultats observés lors des campagnes précédentes. Les destinations sont ensuite classées selon le score obtenu afin de proposer à l'étudiant une liste de recommandations adaptée à son profil.

5.6.3.1 Diagramme de séquence

La figure 5.31 détaille le déroulement d'une requête de recommandation, depuis la consultation par l'étudiant jusqu'au classement retourné.

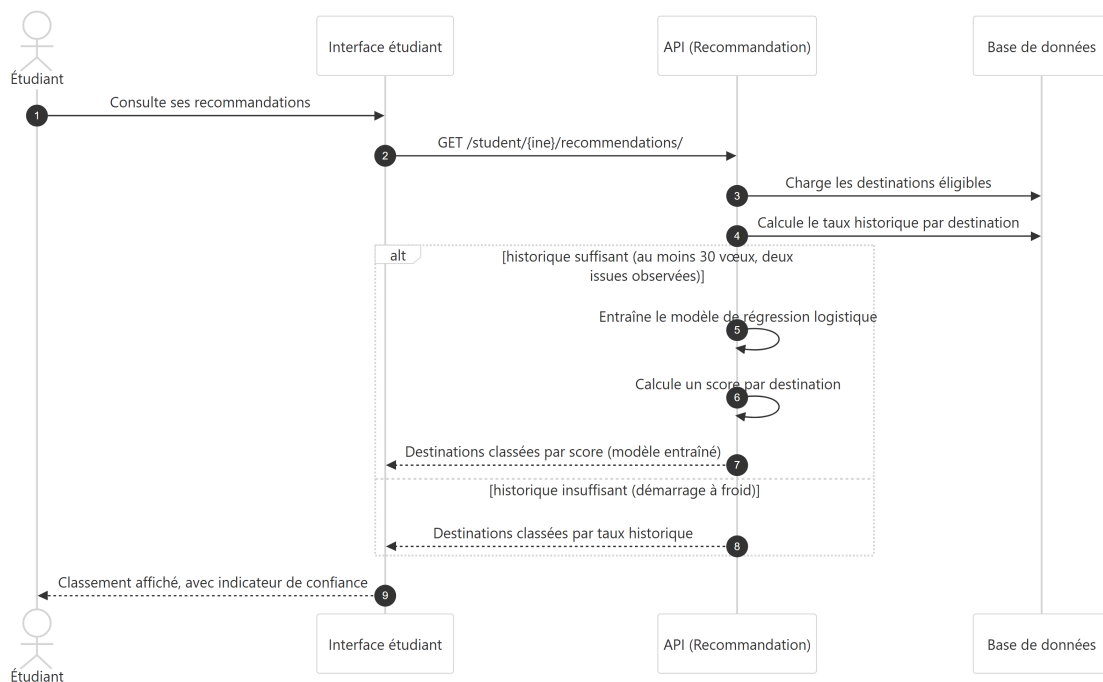


FIGURE 5.31 – Calcul et consultation d'une recommandation de destination

Le calcul du taux historique précède systématiquement la tentative d'entraînement, puisqu'il sert lui-même de repli lorsque le modèle ne peut pas être entraîné. L'étudiant obtient ainsi toujours un classement, que l'historique disponible soit suffisant pour entraîner un modèle ou non.

5.6.4 Implémentation

Pour chaque accord, le système compare le nombre d'étudiants qui l'ont obtenu au nombre d'étudiants qui l'avaient demandé, sur les campagnes déjà terminées. Ce taux, associé au GPA et au département de l'étudiant, sert à entraîner un modèle qui apprend, à partir des vœux passés, quelles destinations correspondent le mieux à quel profil.

Ce modèle n'est entraîné que si assez de vœux passés sont disponibles, avec à la fois des cas d'affectation et de non-affectation. Sinon, faute de données suffisantes, le classement se

base simplement sur le taux historique, sans passer par le modèle. L'étudiant obtient ainsi toujours une liste de destinations classées, même en l'absence d'historique.

La figure 5.32 montre l'espace étudiant où ce classement est consulté.

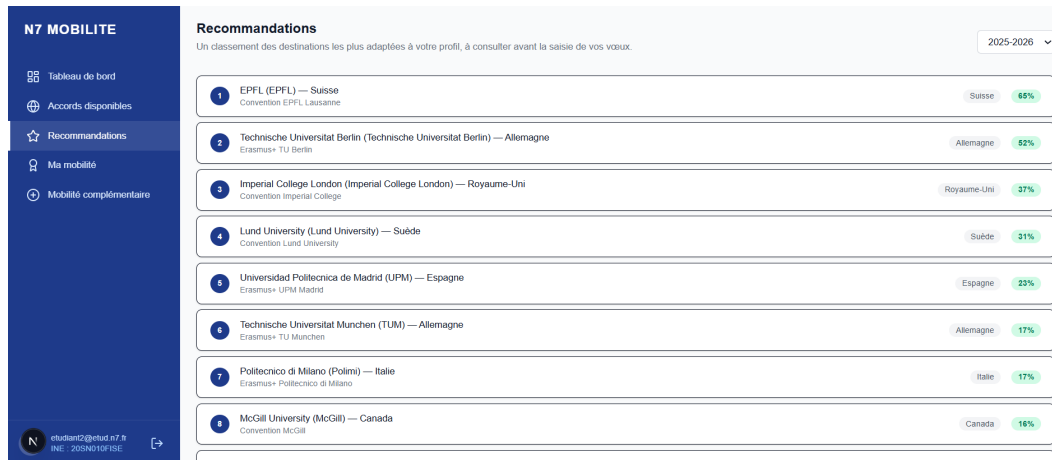


FIGURE 5.32 – Espace étudiant : destinations recommandées

Chaque destination est accompagnée du taux de correspondance calculé par le modèle, et seules les destinations auxquelles l'étudiant peut réellement prétendre sont affichées.

5.6.5 Tests et validation du sprint

Le tableau 5.12 porte sur le calcul du taux historique, la construction du jeu d'entraînement, l'entraînement du modèle et le classement produit par l'endpoint, validés au moyen de tests unitaires et de tests d'intégration.

TABLE 5.12 – Validation de l’US-22 (moteur de recommandation)

Élément / fonctionnalité	Objectif du test	Type	Outil	Résultat	État
Taux historique d’affectation par accord	Vérifier le calcul du taux (affectés / vœux) sur les campagnes closes, y compris sa généralisation entre déclinaisons annuelles d’un même accord	Unitaire	pytest	Conforme	Validé
Construction du jeu d’entraînement	Vérifier que seul le vœu effectivement retenu est marqué positif, y compris pour un étudiant non affecté	Unitaire	pytest	Conforme	Validé
Entraînement et score du modèle	Vérifier l’absence de score en dessous du seuil minimal ou en présence d’une seule classe, et la production de scores valides avec un historique suffisant	Unitaire	pytest	Conforme	Validé
Endpoint de recommandation (démarrage à froid, éligibilité, classement)	Vérifier le classement par taux historique en l’absence de modèle entraîné, l’exclusion des niveaux non éligibles et le classement complet par score	Intégration	pytest	Conforme	Validé

L’incrément du Sprint 6 relatif à la recommandation est ainsi validé au regard du calcul du taux historique, de l’entraînement du modèle et du classement produit, y compris dans les situations de démarrage à froid.

Conclusion

Ce chapitre a présenté la réalisation des sprints du projet. Le chapitre suivant regroupe la validation transversale du système complet.

Tests et validation du système

Plan

Introduction	156
6.1 Stratégie de validation	156
6.2 Synthèse des tests réalisés	156
6.3 Validation End-to-End	157
6.4 Validation des machines à états	158
6.5 Validation des imports, exports et intégrations	159
6.6 Alertes et tâches asynchrones	160
6.7 Statistiques et tableaux de bord	160
6.8 Validation de la sécurité	160
6.9 Performance et charge	161
6.10 Intégration continue et conteneurisation	161
6.11 Analyse de qualité de code	162
6.12 Couverture des besoins non fonctionnels	162
6.13 Matrice de couverture fonctionnelle	163
Conclusion	164

Introduction

Le chapitre précédent a présenté, sprint par sprint, la validation de chaque incrément livré. Celui-ci regroupe la validation transversale du système complet.

6.1 Stratégie de validation

La stratégie de validation repose sur plusieurs niveaux complémentaires.

Le premier niveau regroupe les tests unitaires, qui vérifient une règle de gestion ou un service isolé, et les tests d'intégration, qui vérifient le comportement réel des endpoints REST une fois connectés à la base de données.

Le deuxième niveau concerne les tests de composants frontend. Ceux-ci vérifient le comportement d'un composant ou d'une interface utilisateur pris isolément, sans faire tourner l'application complète.

Enfin, les tests End-to-End permettent de valider certains parcours complets à travers la stack applicative. Ils vérifient ainsi l'interaction entre le frontend, le backend, la base de données et le mécanisme d'authentification utilisé dans l'environnement de test.

Cette stratégie est complétée par des tests spécifiques relatifs à la sécurité, ainsi que par des scripts de performance et de charge implémentés avec `k6`.

6.2 Synthèse des tests réalisés

Le tableau 6.1 présente une synthèse des tests disponibles pour les différentes couches du système.

TABLE 6.1 – Synthèse des tests disponibles par couche

Couche	Outil	Nombre	Constat
Backend	pytest, pytest-django, pytest-cov	856 tests	Couverture mesurée de 84,71 %, supérieure au seuil de 80 % défini dans le pipeline CI.
Frontend	Vitest, Testing Library	131 tests	Couverture globale de 12,84 % des instructions, avec une couverture inégale selon les composants.
End-to-End	Playwright	113 tests	Quatre spécifications principales exécutées contre la stack applicative.
Sécurité applicative	pytest	42 tests	Tests dédiés à l'authentification et au contrôle d'accès.
Performance et charge	k6	2 scripts	Scripts implémentés, mais aucun résultat d'exécution exploitable n'est disponible dans les artefacts analysés.

La figure ?? présente le rapport de couverture backend généré par `pytest-cov`, qui permet d'identifier le niveau de couverture obtenu pour les différents modules du système.

Ce rapport permet d'identifier les modules les moins couverts et de prioriser les tests complémentaires à mettre en place.

6.3 Validation End-to-End

Les tests End-to-End sont réalisés avec Playwright contre la stack applicative complète. Ils s'appuient sur le frontend Next.js, le backend Django et un environnement d'authentification CAS adapté aux tests.

L'authentification est réalisée au préalable pour les deux principaux profils utilisateurs, à savoir l'administrateur et l'étudiant. Les sessions obtenues sont ensuite réutilisées par les différents scénarios afin d'éviter de répéter l'opération de connexion dans chaque test.

Le tableau 6.2 présente les principaux parcours métier et leur niveau de couverture End-to-End.

TABLE 6.2 – Couverture End-to-End des principaux parcours métier

Étape	Test existant	Outil	État
Authentification	Connexion initiale pour les profils administrateur et étudiant	Playwright	✓
Contrôle d'accès par rôle	Vérification qu'un étudiant ne peut pas accéder aux fonctionnalités administratives	Playwright	~
Cycle de vie de l'année universitaire	Scénario parcourant les transitions du cycle de vie	Playwright	✓
Lancement de l'affectation	Vérification de l'alerte en cas d'absence de vœux	Playwright	✓
Déclaration d'une mobilité complémentaire	Scénario étudiant avec validation des champs	Playwright	✓
Validation ou rejet d'une mobilité complémentaire	Aucun scénario End-to-End identifié	—	—
Import Excel réel	Aucun dépôt complet de fichier identifié dans les scénarios E2E	—	—
Export CTI	Le téléchargement réel depuis le navigateur n'est pas couvert	—	—
Tableaux de bord analytiques	Chargement de la page dans le parcours administrateur	Playwright	~
Mobilités entrantes et stages internationaux	Navigation vers les fonctionnalités, sans scénario métier complet	Playwright	~

Les scénarios End-to-End couvrent principalement les parcours centraux liés à l'authentification, au cycle de vie de l'année universitaire, aux affectations et à la déclaration des mobilités complémentaires.

Certaines fonctionnalités sont uniquement couvertes par une navigation vers la page correspondante ou par le chargement de l'interface. Dans ces cas, la validation End-to-End ne permet pas de conclure à une couverture complète du comportement métier.

La figure ?? présente le rapport HTML généré par Playwright à l'issue d'une exécution de la suite End-to-End.

Ce rapport constitue une trace d'exécution permettant de vérifier le statut des différents scénarios et d'identifier les éventuelles régressions après une évolution du système.

La suite End-to-End n'est pas intégrée au pipeline principal d'intégration continue. Elle peut être exécutée séparément à l'aide de l'environnement prévu pour les tests Playwright.

6.4 Validation des machines à états

Les principaux processus métier du système reposent sur des machines à états. Les changements de statut sont effectués par l'intermédiaire de transitions contrôlées, ce qui permet de limiter les modifications incohérentes de l'état des entités.

Le tableau 6.3 présente la couverture disponible pour les différentes machines à états.

TABLE 6.3 – Couverture de test des machines à états

FSM	Transitions testées	Constat	Outil
AcademicYear	Transitions valides et invalides du cycle complet	Testée au niveau backend et intégrée dans un scénario End-to-End couvrant le cycle de vie	pytest, Playwright
Assignment	Validation et publication	Transitions testées côté API. La transition d'annulation n'est pas couverte par une interface ou un endpoint dédié	pytest
ComplementaryMobilityDeclaration	Déclaration, validation et rejet	La déclaration est couverte par des tests API et E2E. La validation et le rejet sont couverts côté API	pytest, Playwright

La machine à états associée à l'année universitaire bénéficie de la couverture la plus complète, puisqu'elle est testée à la fois au niveau backend et dans un parcours utilisateur complet.

6.5 Validation des imports, exports et intégrations

Le système s'appuie sur plusieurs sources de données et propose différents mécanismes d'import et d'export. Leur validation est principalement réalisée au niveau des tests unitaires et des tests d'intégration.

Le tableau 6.4 présente les principaux éléments couverts.

TABLE 6.4 – Validation des imports, exports et intégrations

Fonctionnalité	Constat	Type	Outil
Synchronisation MoveON et Pégase	Tests réalisés avec des doublures des clients HTTP externes	Unitaire	pytest
Import Excel	Validation des données, détection des doublons et traitement des erreurs	Unitaire	pytest
Export CTI VII.D3	Génération et contenu du fichier Excel vérifiés au niveau intégration. Le téléchargement depuis le navigateur n'est pas couvert par un scénario E2E	Intégration	pytest
Export des statistiques	Génération d'un classeur Excel testée sur plusieurs jeux de données	Intégration	pytest

Les mécanismes d'import et d'export sont principalement validés au niveau backend. Les parcours complets impliquant le dépôt ou le téléchargement réel d'un fichier depuis l'interface utilisateur constituent encore un axe de validation complémentaire.

6.6 Alertes et tâches asynchrones

Certaines fonctionnalités du système reposent sur des traitements asynchrones ou sur la génération d'alertes.

Le déclenchement d'une alerte lorsqu'un administrateur tente de lancer une affectation sans vœu disponible est couvert par un scénario End-to-End dédié. Le calcul d'affectation est quant à lui validé à travers les tests de l'algorithme et des endpoints associés.

Les autres traitements asynchrones, notamment ceux liés aux synchronisations et aux recalculs automatiques, sont principalement couverts indirectement par les tests des services qui leur sont associés. Aucun scénario End-to-End spécifique n'a été identifié pour chacun de ces traitements.

6.7 Statistiques et tableaux de bord

Les indicateurs statistiques du système sont calculés à partir des données de mobilité et des résultats d'affectation. Les endpoints responsables de ces agrégations sont couverts par des tests backend.

La génération des exports Excel associés est également testée. Le chargement de la page des tableaux de bord analytiques est inclus dans le parcours administrateur End-to-End. Toutefois, les différents filtres proposés dans l'interface ne sont pas tous couverts par des scénarios dédiés.

Le calcul de la durée cumulée de mobilité CTI suit une approche similaire. Les calculs et les exports associés sont couverts côté API, tandis que les interfaces de consultation et de téléchargement ne disposent pas de la même profondeur de couverture.

6.8 Validation de la sécurité

La sécurité applicative dispose d'une suite de tests spécifique composée de 42 tests. Ces tests couvrent notamment :

- l'accès aux ressources sans authentification ;
- l'utilisation d'un jeton expiré ;
- l'utilisation d'un jeton invalide ou altéré ;
- la tentative d'accès d'un étudiant à des ressources administratives ;
- la tentative d'accès d'un étudiant aux données d'un autre étudiant ;
- l'accès autorisé pour un administrateur ;
- l'authentification reposant sur les cookies ;
- les endpoints volontairement publics.

Ces tests permettent de valider les principaux mécanismes d’authentification et de contrôle d’accès implémentés dans l’application.

En revanche, aucun test spécifique d’injection SQL, de XSS ou de CSRF n’a été identifié dans les artefacts de tests disponibles.

Le pipeline d’intégration continue complète cette validation par un scan de vulnérabilités des images Docker avec Trivy. Les vulnérabilités de sévérité élevée ou critique peuvent bloquer le pipeline.

6.9 Performance et charge

Deux scripts k6 ont été implémentés pour évaluer les performances de l’application.

Le premier correspond à un test de charge ciblant les endpoints les plus sollicités et permettant de mesurer notamment les temps de réponse et les taux d’erreur sous une charge concurrente.

Le second constitue un test plus simple permettant de vérifier le comportement général de l’API.

Ces scripts sont exécutables, mais les résultats détaillés d’une campagne de test complète ne sont pas disponibles dans les éléments analysés. Il n’est donc pas possible de conclure quantitativement au respect des objectifs de performance définis pour le système.

Des tests complémentaires mesurent également certains temps de réponse directement à l’aide du client de test Django. Ces mesures permettent de détecter certaines régressions, mais ne remplacent pas une véritable mesure réalisée sous charge concurrente.

6.10 Intégration continue et conteneurisation

Le pipeline d’intégration continue automatise plusieurs étapes de validation du système.

Il comprend notamment :

- l’analyse statique et la vérification du format du code backend (ruff) ;
- l’exécution des tests backend avec une base PostgreSQL ;
- la vérification du seuil minimal de couverture backend ;
- l’analyse statique et la vérification des types du code frontend (ESLint, TypeScript) ;
- l’exécution des tests frontend ;
- la construction des images Docker ;
- l’analyse des vulnérabilités des images avec Trivy.

La suite End-to-End Playwright reste indépendante de ce pipeline principal et doit être exécutée séparément.

Le déploiement dans l’environnement cible s’appuie sur les conteneurs Docker et comprend notamment la récupération du code, la reconstruction des services, l’application des migrations de la base de données et la collecte des fichiers statiques.

6.11 Analyse de qualité de code

Une instance SonarQube est disponible dans l’environnement de développement, avec une configuration dédiée au projet (périmètre analysé, exclusions, rapports de couverture backend et frontend). Son usage reste local : l’analyse est lancée manuellement, en dehors du pipeline d’intégration continue.

Le typage statique du backend, préparé avec `mypy` et `django-stubs`, est configuré mais n’est pour l’instant appelé ni localement ni dans le pipeline CI.

6.12 Couverture des besoins non fonctionnels

Le tableau 6.5 présente le niveau de validation disponible pour les besoins non fonctionnels définis précédemment.

TABLE 6.5 – Couverture des besoins non fonctionnels

Besoin	Validation réalisée	Outil	Résultat	État
BNF01 – Interopérabilité	Tests des adaptateurs et des mécanismes d’import	pytest	Les différents adaptateurs sont couverts par des tests unitaires	Partiellement validé
BNF02 – Intégrité des données	Tests des pipelines d’import et de validation	pytest	Validation, gestion des doublons et erreurs couvertes	Validé
BNF03 – Sécurité et protection des accès	Suite dédiée à l’authentification et aux autorisations	pytest	42 scénarios de sécurité disponibles	Validé
BNF04 – Performance	Scripts de charge et mesures complémentaires	k6, pytest	Scripts disponibles, mais résultats de charge non disponibles	Non mesuré

Besoin	Validation réalisée	Outil	Résultat	État
BNF05 – Disponibilité	Aucune campagne spécifique identifiée	—	Absence de preuve de validation dédiée	Non testé
BNF06 – Scalabilité	Aucun test spécifique identifié	—	Absence de test de montée en charge distribué	Non testé
BNF07 – Ergonomie	Tests de navigation et composants frontend	Playwright, Vitest	Certains parcours utilisateur sont couverts, mais la couverture reste partielle	Partiellement validé
BNF08 – Traçabilité	Tests liés au journal d’audit	pytest, Vitest	Vérification des événements et informations associées dans les scénarios testés	Validé

6.13 Matrice de couverture fonctionnelle

Le tableau 6.6 synthétise le niveau de validation disponible pour les principales fonctionnalités du système.

TABLE 6.6 – Matrice de couverture fonctionnelle

Fonctionnalité	Unitair	Intégra	Compos	E2E	Observation
Authentification CAS et JWT	—	✓	—	✓	Sessions préparées pour les deux rôles dans les tests E2E.
Contrôle d’accès admin / étudiant	—	✓	—	~	Contrôle d’accès principalement validé côté API.
Cycle de vie de l’année universitaire	✓	—	—	✓	Cycle métier couvert à plusieurs niveaux.
Référentiels de base	—	✓	—	—	Suite de tests API dédiée.

Fonctionnalité	Unitair	Intégra	Compos	E2E	Observation
Accords et quotas	✓	—	—	—	Méthode de répartition testée au niveau service.
Import des étudiants	—	✓	—	—	Import validé côté backend.
Vœux et affectation	✓	✓	—	✓	Algorithme et principaux parcours testés.
Validation et publication des affectations	—	✓	—	✓	Couverte au niveau API et dans le cycle global.
Mobilités entrantes	—	—	—	~	Navigation disponible, sans scénario métier complet identifié.
Stages internationaux	—	—	—	~	Navigation couverte de manière limitée.
Mobilités complémentaires : déclaration	—	✓	—	✓	Parcours étudiant bien couvert.
Mobilités complémentaires : validation	—	✓	—	—	Couverture backend disponible.
Durée et export CTI	—	✓	—	—	Calcul et export validés côté API.
Statistiques et tableaux de bord	—	✓	—	~	Endpoints testés et chargement de page couvert partiellement.
Journal d'audit	—	✓	✓	—	Couverture backend et composants disponibles.
Alertes système	—	—	—	✓	Scénario End-to-End dédié.

Conclusion

La validation montre une couvverture solide sur le cœur du système (cycle de vie de l'année universitaire, algorithme d'affectation, contrôle d'accès) et plus partielle sur les flux réels d'upload et de téléchargement ainsi que sur les besoins non fonctionnels de performance

et de disponibilité.

Conclusion générale

Bibliographie

- [8] Mehdi ELAHI et al. « Developing and Evaluating a University Recommender System ». In : *Frontiers in Artificial Intelligence* (2022). URL : <https://pmc.ncbi.nlm.nih.gov/articles/PMC8848746/> (visit  le 09/03/2026).
- [27] David GALE et Lloyd S. SHAPLEY. « College Admissions and the Stability of Marriage ». In : *The American Mathematical Monthly* 69.1 (1962), p. 9-15. DOI : 10.2307/2312726.
- [28] Atila ABDULKADIROĞLU et Tayfun SÖNMEZ. « School Choice : A Mechanism Design Approach ». In : *American Economic Review* 93.3 (2003), p. 729-747. DOI : 10.1257/000282803322157061.
- [29] David R. COX. « The Regression Analysis of Binary Sequences ». In : *Journal of the Royal Statistical Society : Series B* 20.2 (1958), p. 215-242. DOI : 10.1111/j.2517-6161.1958.tb00292.x.
- [30] Tianqi CHEN et Carlos GUESTRIN. « XGBoost : A Scalable Tree Boosting System ». In : *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, p. 785-794. DOI : 10.1145/2939672.2939785.

Netographie

- [1] *L'ENSEEIH*T – *Présentation*. URL : <https://www.enseeiht.fr/fr/l-enseeiht.html> (visité le 18/03/2026).
- [2] *International* – *Toulouse INP-ENSEEIH*T. URL : <https://www.enseeiht.fr/fr/international.html> (visité le 18/03/2026).
- [3] *Mobilité sortante* – *Toulouse INP-ENSEEIH*T. URL : <https://www.enseeiht.fr/fr/international/mobilite-sortante.html> (visité le 18/03/2026).
- [4] *Mobilité entrante* – *Toulouse INP-ENSEEIH*T. URL : <https://www.enseeiht.fr/fr/international/mobilite-entrante.html> (visité le 18/03/2026).
- [5] *Programmes courts* – *Toulouse INP-ENSEEIH*T. URL : <https://www.enseeiht.fr/fr/international/programmes-courts.html> (visité le 18/03/2026).
- [6] *MoveON* – *QS Solutions*. URL : <https://www.qs.com/solutions/moveon> (visité le 09/03/2026).
- [7] *Définition CRM* – *Eudonet*. URL : <https://www.eudonet.com/fr/ressources/blog/definition-crm/> (visité le 09/03/2026).
- [9] *React* – *The library for web and native user interfaces*. URL : <https://react.dev/> (visité le 17/03/2026).
- [10] *Angular* – *The web development framework*. URL : <https://angular.dev/> (visité le 17/03/2026).
- [11] *Next.js* – *The React Framework for the Web*. URL : <https://nextjs.org/> (visité le 17/03/2026).
- [12] *Node.js* – *Run JavaScript Everywhere*. URL : <https://nodejs.org/> (visité le 18/03/2026).
- [13] *FastAPI* – *Modern, fast web framework for building APIs with Python*. URL : <https://fastapi.tiangolo.com/> (visité le 18/03/2026).
- [14] *Django* – *The web framework for perfectionists with deadlines*. URL : <https://www.djangoproject.com/> (visité le 18/03/2026).
- [15] *Django REST Framework* – *Web APIs for Django*. URL : <https://www.django-rest-framework.org/> (visité le 18/03/2026).
- [16] *Django Ninja* – *Fast Django REST Framework*. URL : <https://django-ninja.dev/> (visité le 18/03/2026).
- [17] *MySQL* – *The world's most popular open source database*. URL : <https://www.mysql.com/> (visité le 17/03/2026).

-
- [18] *MongoDB – The developer data platform*. URL : <https://www.mongodb.com/> (visité le 17/03/2026).
 - [19] *PostgreSQL – The world’s most advanced open source relational database*. URL : <https://www.postgresql.org/> (visité le 17/03/2026).
 - [20] *Celery – Distributed Task Queue*. URL : <https://docs.celeryq.dev/> (visité le 19/03/2026).
 - [21] *Redis – The Real-time Data Platform*. URL : <https://redis.io/docs/> (visité le 19/03/2026).
 - [22] *Django Q2 – A multiprocessing task queue for Django*. URL : <https://django-q2.readthedocs.io/> (visité le 19/03/2026).
 - [23] *MinIO – High Performance Object Storage*. URL : <https://min.io/docs/> (visité le 24/03/2026).
 - [24] *Nginx – High Performance Load Balancer, Web Server, & Reverse Proxy*. URL : <https://nginx.org/en/docs/> (visité le 27/03/2026).
 - [25] *Traefik – The Cloud Native Application Proxy*. URL : <https://doc.traefik.io/traefik/> (visité le 27/03/2026).
 - [26] *Règlement (UE) 2016/679 du Parlement européen et du Conseil du 27 avril 2016 relatif à la protection des personnes physiques à l’égard du traitement des données à caractère personnel (RGPD)*. URL : <https://eur-lex.europa.eu/eli/reg/2016/679/oj/fra> (visité le 04/05/2026).
 - [31] *Mistral AI – La plateforme pour les modèles de langage*. URL : <https://docs.mistral.ai/> (visité le 30/03/2026).
 - [32] *LangChain – Build context-aware reasoning applications*. URL : <https://python.langchain.com/docs/> (visité le 30/03/2026).
 - [33] *LlamaIndex – Python Framework Documentation*. URL : <https://developers.llamaindex.ai/python/framework/> (visité le 30/03/2026).
 - [34] *Haystack – The open-source AI framework*. URL : <https://docs.haystack.deepset.ai/> (visité le 30/03/2026).